

计算机组成原理

重点难点

课程概貌（40学时授课+16学时实验）

- 前序课程

- C语言程序设计
- 数字逻辑设计（+Verilog HDL）

- 主要内容

- 基本部件的结构和组织方式
- 基本运算的操作原理
- 基于RISC-V的CPU设计

- 一. 概论
- 二. 计算机的运算方法
- 三. RISC-V汇编及其指令系统
- 四. 处理器（CPU）设计
- 五. 流水线处理器
- 六. 存储器
- 七. 总线、输入输出系统

第1章 概论

1.1 计算机发展历史

1.2 计算机体系结构中的7个伟大思想

1.3 计算机层次结构

1.4 计算机性能指标

电子计算机发展历程

	第一代	第二代	第三代	第四代
年代	1946~	1958~	1965~	1971~
基础器件	电子管	晶体管	集成电路	超大规模集成电路
运算速度	几千~几万次/s	几万~几万次/s	几十万~几百万次/s	MIPS->GIPS->TIPS
存储器	水银延迟线、磁鼓、纸带、卡片	磁芯 磁盘、磁带	半导体 磁盘	半导体 磁盘
特征	机器语言 汇编语言	算法语言 FORTRAN、 ALGOL-60、 COBOL 操作系统	软件技术、 外设发展迅速 小型计算机	微型计算机 多机处理/网络化

速度越来越快、 体积越来越小、 成本越来越低、 功耗越来越低

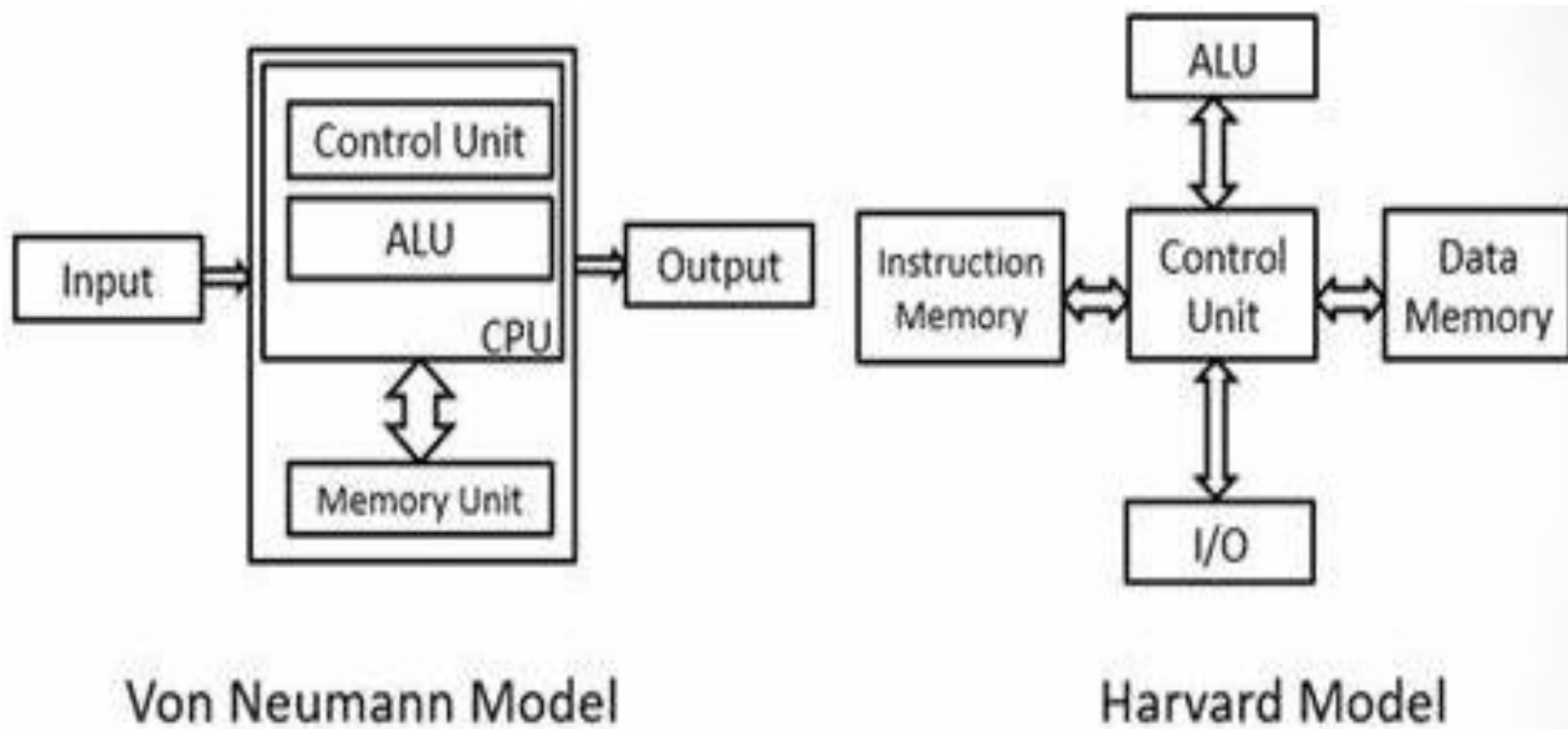
计算机体系结构中的7个伟大思想

- 1) 使用抽象简化设计
- 2) 加速经常性事件
- 3) 通过并行提高性能
- 4) 通过流水线提高性能
- 5) 通过预测提高性能
- 6) 存储层次
- 7) 通过冗余提高可靠性

冯·诺依曼架构计算机的特点

- 计算机由**五大部件**组成
- 指令和数据以同等地位存于存储器，可按地址寻访
- 指令和数据用二进制表示
- 指令由操作码和地址码组成
- **存储程序**
 - 指令在存储器中按照顺序存放，通常是按顺序执行，特定条件下可以根据条件改变执行顺序。
- **以运算器为中心**

冯诺依曼架构 vs. 哈佛架构



哈佛架构和冯诺·依曼架构的区别

- 程序空间和数据空间是否是一体的？
 - 冯·诺依曼结构数据空间和程序空间不分开
 - 哈佛结构数据空间和程序空间是分开的，允许同时取指和取操作数，从而大大提高了运算能力。
- 早期大多采用冯·诺依曼结构，典型代表是X86。
- 改进型哈佛架构（数据、指令分开，且总线复用），典型代表是DSP和ARM。
- 现在的处理器虽然外部总线上看是冯·诺依曼结构的，但是由于内部CACHE的存在，实际上内部来看已经类似改进型哈佛结构。

计算机硬件的主要指标

- 时间指标

- 主频、时钟周期
- CPI、IPC
- MIPS、MFLOPS
- CPU执行时间、响应时间、吞吐率

- 非时间指标

- 机器字长
- 总线宽度
- 主存容量、存储带宽
- CPU内核数

第二章 计算机的运算方法

2.1 计算机中数的表示

2.2 定点运算

2.3 浮点运算

2.1 计算机中数的表示

- 计算机中数的表示

- 无符号数和有符号数

- 机器数与真值；原码/补码/反码/移码表示法

- 定点表示和浮点表示

- IEEE 754标准

- 算数移位与逻辑移位

三种机器数的小结

- 最高位为符号位，**书写上**用 “,” （整数）或 “.” （小数）将数值部分和符号位隔开
- 对于**正数**：符号位为0，原码 = 补码 = 反码
- 对于**负数**：补码 $\langle \text{——} \rangle$ 原码的方法：
 - **除符号位外，每位取反，末位加 1**
 - **自右向左从最先遇到的1左边开始，每位取反（符号位除外）**
- 对于负数：补码=反码+1
反码=原码按位取反（符号位除外）

*例：已知小数 $[y]_{\text{补}}$ ，求 $[-y]_{\text{补}}$

设 $[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$ ，根据符号位 y_0 为0或1分开讨论

<I>

$$[y]_{\text{补}} = 0.y_1y_2 \dots y_n$$

$$[y]_{\text{原}} = 0.y_1y_2 \dots y_n$$

$$-y = \text{---} 0.y_1y_2 \dots y_n$$

$$[-y]_{\text{补}} = 1 + (\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$[-y]_{\text{补}} = 1.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n}$$

<II>

$$[y]_{\text{补}} = 1.y_1y_2 \dots y_n$$

$$[y]_{\text{原}} = 1 + (0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$y = \text{---} (0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$-y = 0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n}$$

$$[-y]_{\text{补}} = 0.\overline{y_1} \overline{y_2} \dots \overline{y_n} + 2^{-n}$$

$[y]_{\text{补}}$ 连同符号位在内，每位取反，末位加1，即得 $[-y]_{\text{补}}$

观察发现 $[-y]_{\text{补}} + [y]_{\text{补}} = 2 \rightarrow 0 \pmod{2}$ $[-y]_{\text{补}} = -[y]_{\text{补}}$

*例：已知整数 $[X]_{\text{补}}$ ，求 $[-X]_{\text{补}}$

设 $[X]_{\text{补}} = X_n, X_{n-1} \cdots X_1 X_0$ ，根据符号位 X_n 为0或1分开讨论

<I>

$$[X]_{\text{补}} = 0, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$X_{\text{原}} = 0, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$[-X]_{\text{原}} = 1, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$[-X]_{\text{补}} = 1, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

<II>

$$[X]_{\text{补}} = 1, X_{n-1} X_{n-2} \cdots X_1 X_0$$

$$X_{\text{原}} = 1, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{原}} = 0, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{补}} = 0, (\bar{X}_{n-1} \bar{X}_{n-2} \cdots \bar{X}_1 \bar{X}_0 + 1)$$

$[X]_{\text{补}}$ 连同符号位在内，每位取反，末位加1，即得 $[-X]_{\text{补}}$

观察发现 $[-X]_{\text{补}} + [X]_{\text{补}} = 2^{n+1} \rightarrow 0 \pmod{2^{n+1}}$ $[-X]_{\text{补}} = -[X]_{\text{补}}$

例：机器数的真值

- 设机器数字长为8位(其中1位为符号位);对于整数,当其分别代表无符号数、原码、补码和反码时, **对应的真值范围**各为多少?

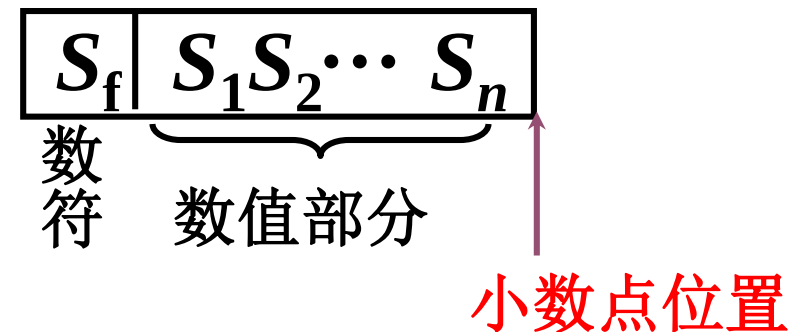
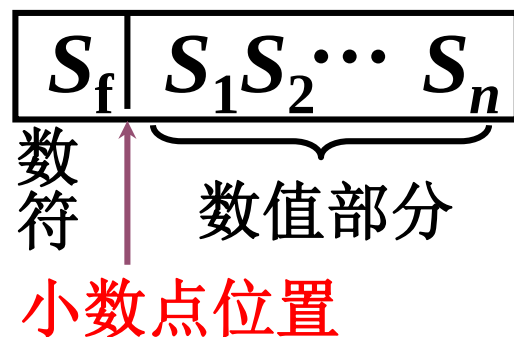
二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	<u>±0</u>	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

真值、补码和移码的对照表

真值 x ($n=5$)	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的 十进制整数
- 1 0 0 0 0	1 0 0 0 0	0 0 0 0 0	0
- 1 1 1 1 1	1 0 0 0 1	0 0 0 0 1	1
- 1 1 1 1 0	1 0 0 0 1 0	0 0 0 0 1 0	2
⋮	⋮	⋮	⋮
- 0 0 0 0 1	1 1 1 1 1 1	0 1 1 1 1 1	31
± 0 0 0 0 0	0 0 0 0 0 0	1 0 0 0 0 0	32
+ 0 0 0 0 1	0 0 0 0 0 1	1 0 0 0 0 1	33
+ 0 0 0 1 0	0 0 0 0 1 0	1 0 0 0 1 0	34
⋮	⋮	⋮	⋮
+ 1 1 1 1 0	0 1 1 1 1 0	1 1 1 1 1 0	62
+ 1 1 1 1 1	0 1 1 1 1 1	1 1 1 1 1 1	63

定点表示

- 小数点按约定方式标出



定点机

小数定点机

整数定点机

原码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

补码

$$-1 \sim +(1 - 2^{-n})$$

$$-2^n \sim +(2^n - 1)$$

反码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

浮点表示

$N = S \times r^j$ 浮点数的一般形式

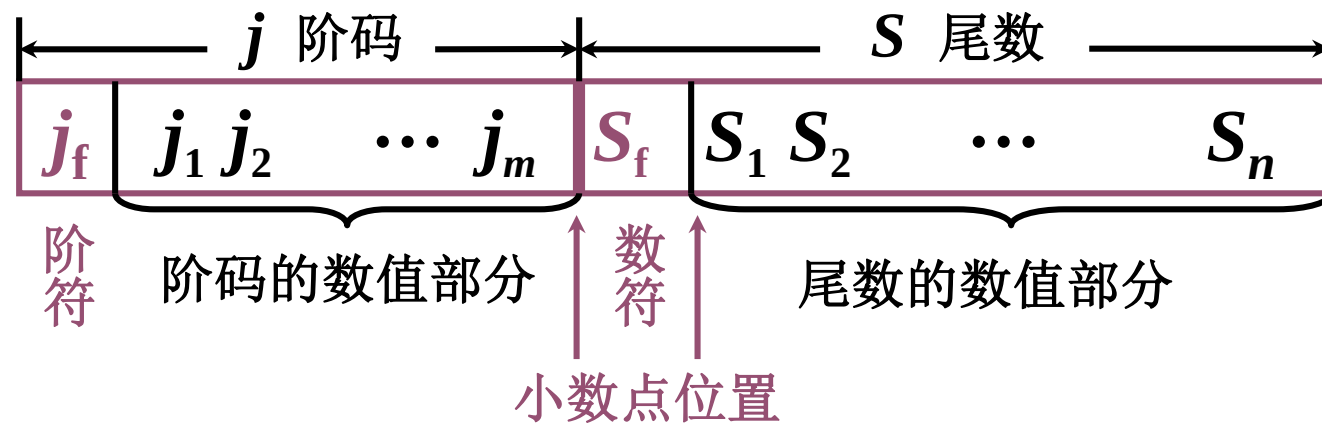
S 尾数 j 阶码 r 基数（基值）

计算机中 r 取 2、4、8、16 等

当 $r = 2$ $N = 11.0101$ 二进制表示
 $\checkmark = 0.110101 \times 2^{10}$ 规格化数
 $= 1.10101 \times 2^1$
 $= 1101.01 \times 2^{-10}$
 $\checkmark = 0.00110101 \times 2^{100}$

计算机中 S 小数、可正可负
 j 整数、可正可负

浮点数的表示形式



S_f

代表浮点数的符号

n

尾数数值位数，反映浮点数的精度

m

阶码数值位数，反映浮点数的表示范围

j_f 和 m

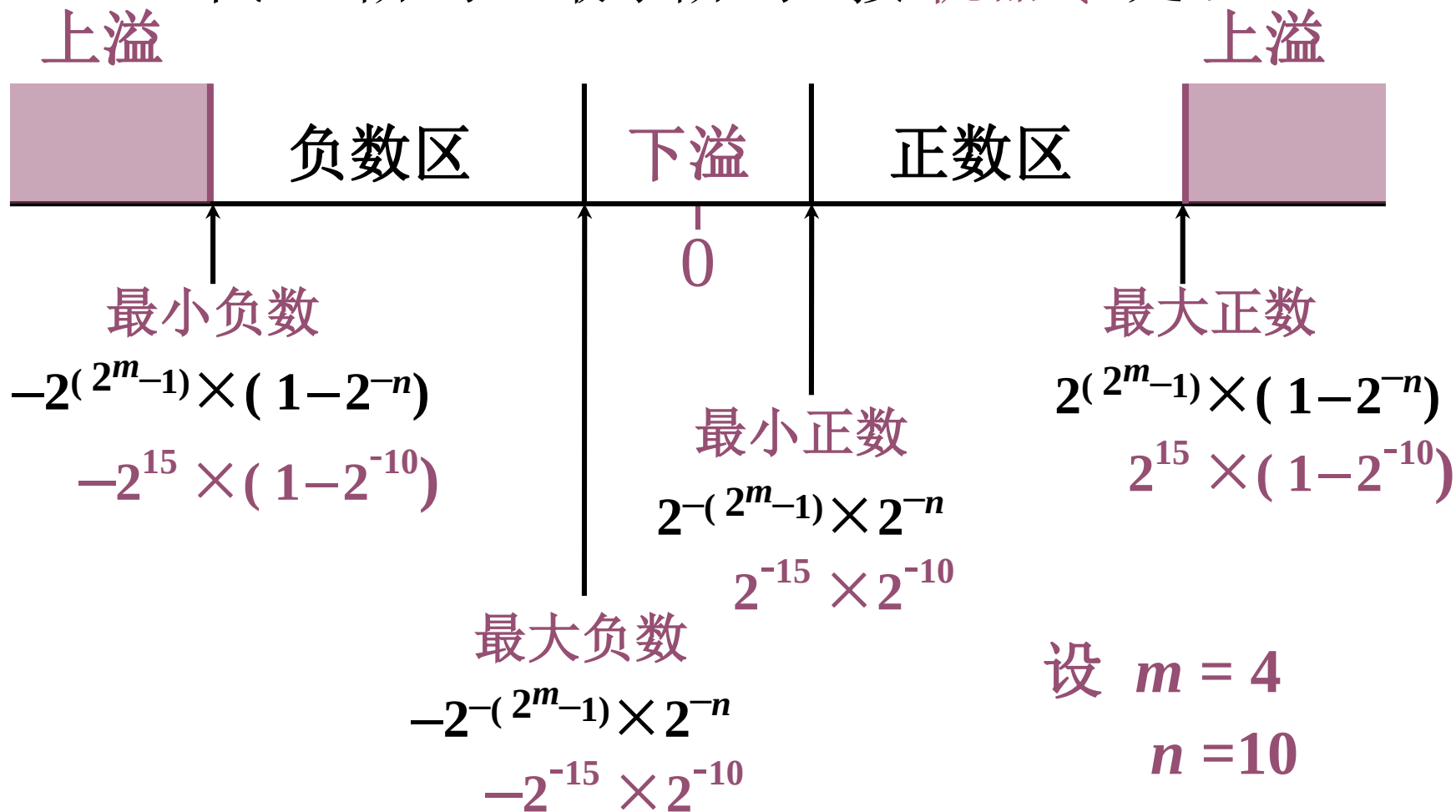
共同表示小数点的实际位置



浮点数的表示范围（真值）

上溢 阶码 > 最大阶码

下溢 阶码 < 最小阶码 按 机器零 处理



二进制数规格化

(1) 规格化数的定义

$$r = 2 \quad \frac{1}{2} \leq |S| < 1$$

(2) 规格化数的判断

$S > 0$	规格化形式	$S < 0$	规格化形式
真值	$0.1 \times \times \dots \times$	真值	$-0.1 \times \times \dots \times$
原码	0.1 $\times \times \dots \times$	原码	1.1 $\times \times \dots \times$
补码	0.1 $\times \times \dots \times$	补码	1.0 $\times \times \dots \times$
反码	$0.1 \times \times \dots \times$	反码	$1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第1数位不同

二进制数规格化的特例

$$S = -\frac{1}{2} = -0.100 \cdots 0$$

$$[S]_{\text{原}} = 1.100 \cdots 0$$

$$[S]_{\text{补}} = \boxed{1.1}00 \cdots 0$$

∴ $[-\frac{1}{2}]_{\text{补}}$ 按照定义是规格化数，而机器判断不是规格化数

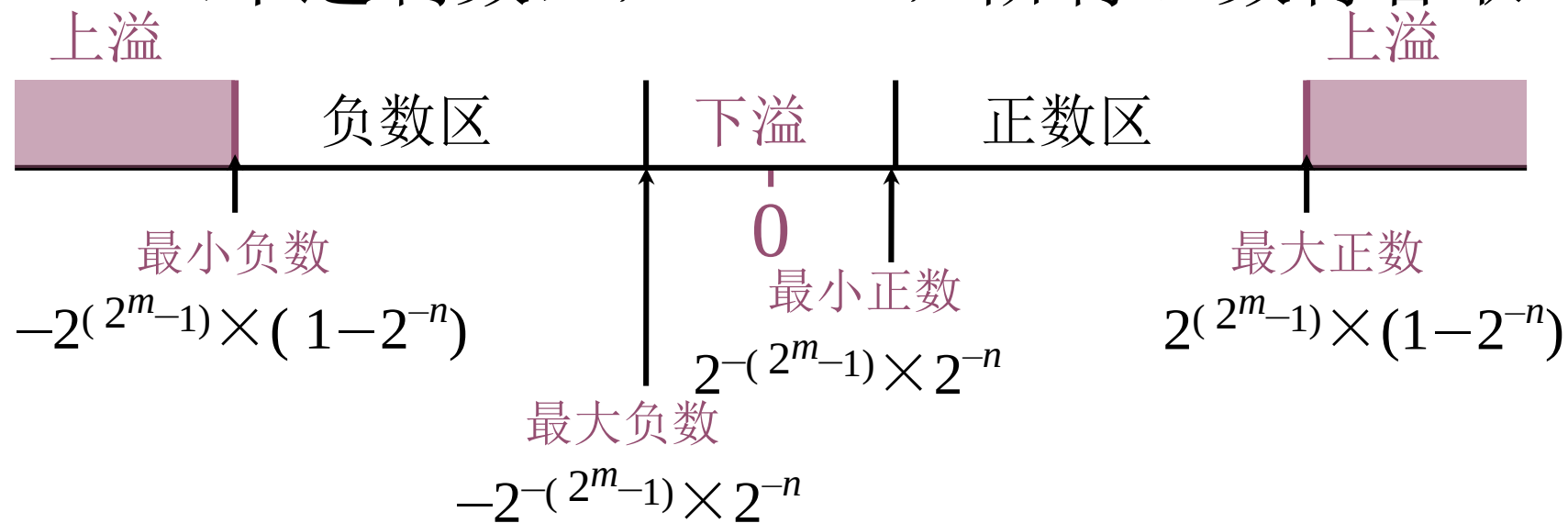
$$S = -1$$

$$[S]_{\text{补}} = \boxed{1.0}00 \cdots 0$$

∴ $[-1]_{\text{补}}$ 按照定义不是规格化数，而机器判断是规格化数

例：写出真值所对应的浮点数补码表示

- 设 $n = 10$ （十进制数）， $m = 4$ ，阶符、数符各取1位。



解：

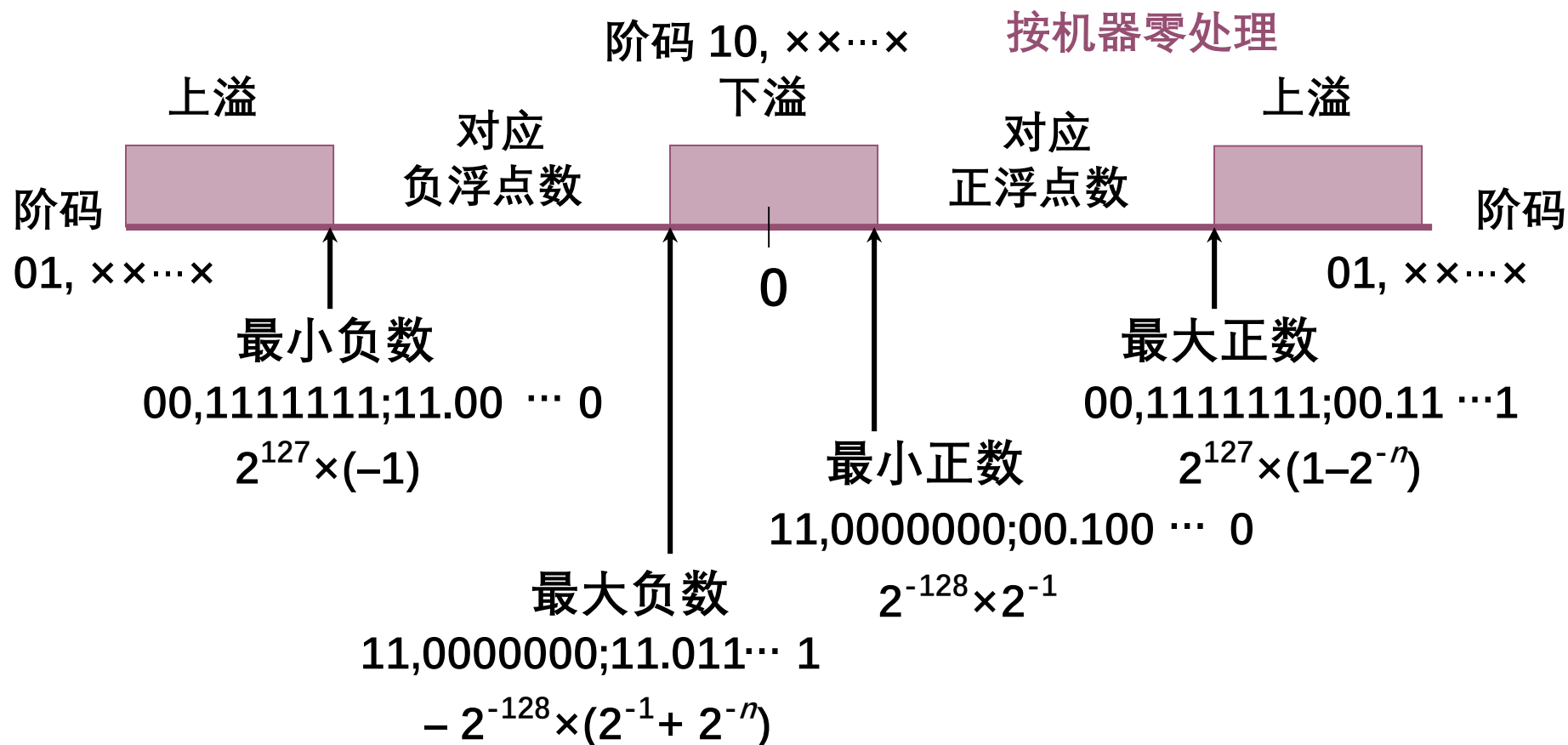
真值

补码

最大正数	$2^{15} \times (1-2^{-10})$	0,1111; 0.1111111111
最小正数	$2^{-15} \times 2^{-10}$	1,0001; 0.0000000001
最大负数	$-2^{-15} \times 2^{-10}$	1,0001; 1.1111111111
最小负数	$-2^{15} \times (1-2^{-10})$	0,1111; 1.0000000001

补码表示的浮点数范围

- 设机器数为补码，尾数为 **规格化形式**，并假设阶符取 2 位，阶码数值位为 7 位，数符取 2 位，尾数数值位取 n 位，则该 **补码** 在数轴上的表示为：



机器零

- 当浮点数**尾数为 0** 时，不论其阶码为何值，按机器零处理
- 当浮点数阶码**小于**它所采用的机器数表示的最小数时，不论尾数为何值，按机器零处理。

如 $m = 4$ $n = 10$

当阶码和尾数都用**补码**表示时，机器零为

$\times, \times \times \times \times; \quad 0.00 \dots 0$

或（阶码 < -16 ）； $\times.\times \times \dots \times$

当阶码用移码，尾数用补码表示时，机器零为

$0, 0000; \quad 0.00 \dots 0$

有利于机器中“判 0”电路的实现



IEEE 754浮点数标准

- 单精度 (32-bit)

31	30	29	28	27	26	25	24	23	22 ~ 0										
s	8位指数（无符号数）								23位尾数（无符号数）										

- 双精度 (64-bit)

63	62	61	60	59	58	57	56	55	54	53	52	51~0													
s	11位指数 (无符号数)											52位尾数 (无符号数)													

IEEE 754浮点数：单精度为例



指数	尾数	表示对象	换算方法
0	0	0	规定（符号位不同，存在+0.0和-0.0）
0	非0	正负非规格化数	正负非规格化数 = $(-1)^S * (\text{尾数}_2) * 2^{(0 - 126)}$ (S代表符号位，1为负数，0为正数)
[1: 254]	任意	正负浮点数	正负浮点数 = $(-1)^S * (1 + \text{尾数}_2) * 2^{(\text{指数} - 127)}$
255	0	正负无穷 (inf)	规定
255	非零	NaN	规定

算术移位规则

- 符号位不变
- 机器数的移位 与 真值移位 一致

真值	机器数表示	空位补？
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移 补 0
		右移 补 1
	反 码	1

$$y = 0.y_1y_2\dots y_n100\dots00$$

$$y = -0.y_1y_2\dots y_n100\dots00$$

$$[y]_{\text{原}} = 1.y_1y_2\dots y_n100\dots00$$

$$\text{右移1位: } 1.\textcolor{red}{0}y_1y_2\dots y_n100\dots0$$

$$\text{左移1位: } 1.y_2y_3\dots y_n100\dots00\textcolor{red}{0}$$

$$[y]_{\text{反}} = 1.\overline{y}_1\overline{y}_2\dots\overline{y}_n011\dots11$$

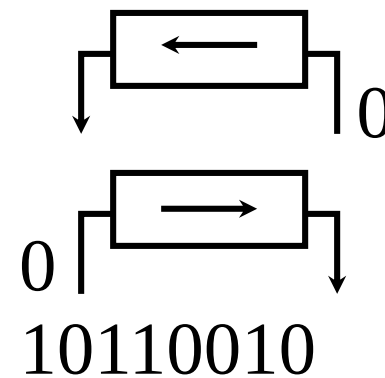
$$\begin{aligned}
 [y]_{\text{补}} &= [y]_{\text{反}} \text{ 的末位} + \textcolor{red}{1} \\
 &= 1.\overline{y}_1\overline{y}_2\dots\overline{y}_n100\dots00
 \end{aligned}$$

算术移位和逻辑移位的区别

- 算术移位：有符号数的移位
- 逻辑移位：无符号数的移位

逻辑左移 低位添 0，高位丢弃

逻辑右移 高位添 0，低位丢弃



例：补码表示的机器数 10110010

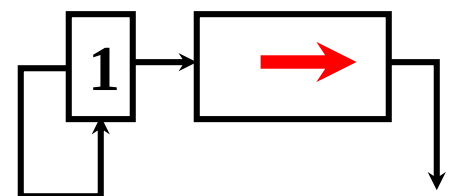
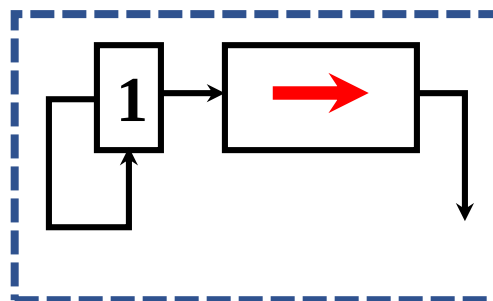
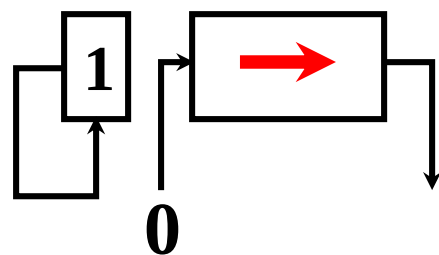
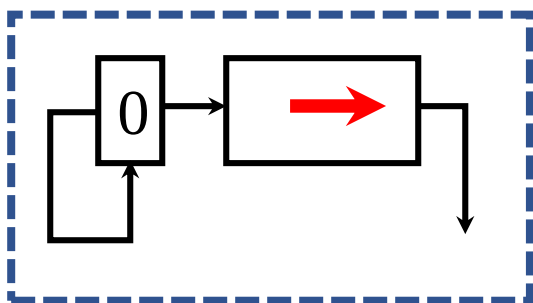
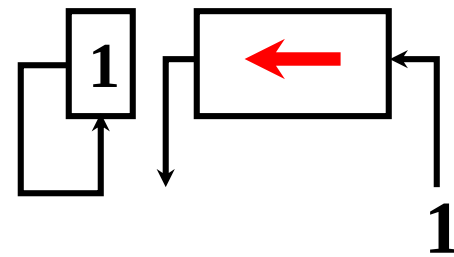
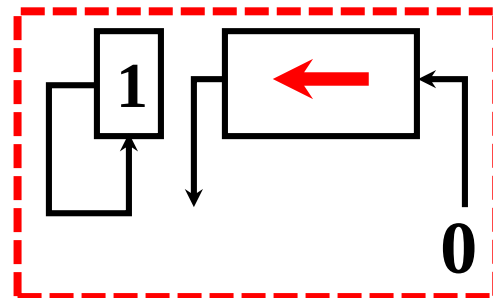
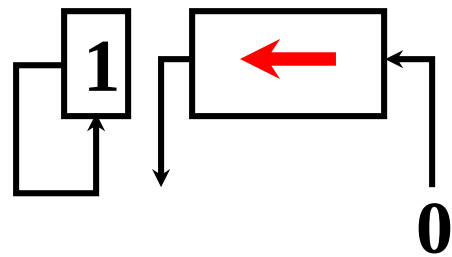
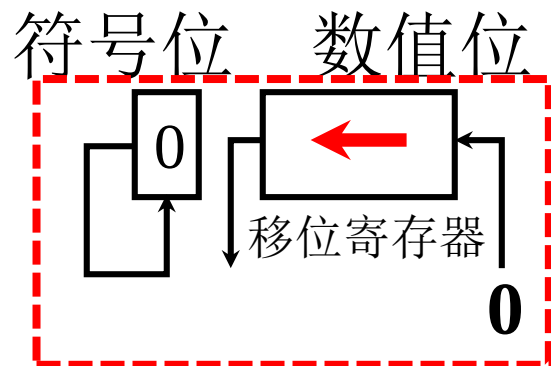
逻辑左移 0110010**0**

逻辑右移 **0**1011001

算术左移 1110010**0**

算术右移 1**1**011001

算术移位的硬件实现



a 真值为正
← 丢 1 出错
→ 丢 1 影响精度

b 负数的原码
出错
影响精度

c 负数的补码
正确
影响精度

d 负数的反码
正确
正确

第二章 计算机的运算方法

2.1 计算机中数的表示

2.2 定点运算

- 加减法运算
- 一位乘法运算
- Booth算法

2.3 浮点运算

加減法运算

- 补码加減运算公式

(1) 加法

整数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2}$

(2) 減法

$$A-B = A+(-B)$$

整数 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2}$

连同符号位一起相加，符号位产生的进位自然丢掉

原码一位乘运算规则

整数： 设 $[X]_{\text{原}} = \textcolor{red}{X}_n X_{n-1} \dots X_1 X_0$, $[Y]_{\text{原}} = \textcolor{red}{Y}_n Y_{n-1} \dots Y_1 Y_0$

$$\begin{aligned}[X \times Y]_{\text{原}} &= (X_n \oplus Y_n)(\textcolor{blue}{0} X_{n-1} \dots X_1 X_0 \times \textcolor{blue}{0} Y_{n-1} \dots Y_1 Y_0) \\ &= (X_n \oplus Y_n)(|X| \times |Y|)\end{aligned}$$

小数： 设 $[x]_{\text{原}} = \textcolor{red}{x}_0 . x_1 x_2 \dots x_n$, $[y]_{\text{原}} = \textcolor{red}{y}_0 . y_1 y_2 \dots y_n$

$$\begin{aligned}[x \times y]_{\text{原}} &= (x_0 \oplus y_0) . (0 . x_1 x_2 \dots x_n \times 0 . y_1 y_2 \dots y_n) \\ &= (x_0 \oplus y_0) . (|x| \times |y|)\end{aligned}$$

乘积的符号位单独处理, 数值部分为绝对值相乘

整数原码一位乘递推公式

$$\begin{aligned}
 |X| \times |Y| &= |X|(\mathbf{0}Y_{n-1} \dots Y_1Y_0) = 2^{n-1}Y_{n-1}|X| + 2^{n-2}Y_{n-2}|X| + \dots + 2^1Y_1|X| + 2^0Y_0|X| \\
 &= \mathbf{2}^n * (2^{-1}Y_{n-1}|X| + 2^{-2}Y_{n-2}|X| + \dots + 2^{-n+1}Y_1|X| + 2^{-n}Y_0|X|) \\
 &= \mathbf{2}^n * (2^{-1} * (Y_{n-1}|X| + 2^{-1}Y_{n-2}|X| + \dots + 2^{-n+2}Y_1|X| + 2^{-n+1}Y_0|X|)) \\
 &= \mathbf{2}^n * (2^{-1} * (Y_{n-1}|X| + 2^{-1}(Y_{n-2}|X| + \dots + 2^{-n+3}Y_1|X| + 2^{-n+2}Y_0|X|)))
 \end{aligned}$$

$$\begin{aligned}
 &\dots\dots\dots \dots\dots\dots \dots\dots\dots \dots\dots\dots \dots\dots\dots \dots\dots\dots \\
 &= 2^{-1} * (\underbrace{Y_{n-1}|X| * \mathbf{2}^n}_{Z_n} + 2^{-1}(\underbrace{Y_{n-2}|X| * \mathbf{2}^n + 2^{-1}(\dots + 2^{-1}(\underbrace{Y_1|X| * \mathbf{2}^n + 2^{-1}(Y_0|X| * \mathbf{2}^n + 0)}_{Z_0}))}_{Z_1})) \dots)
 \end{aligned}$$

$$Z_0 = 0$$

$$Z_1 = 2^{-1}(Y_0|X| * \mathbf{2}^n + Z_0)$$

$$Z_2 = 2^{-1}(Y_1|X| * \mathbf{2}^n + Z_1)$$

...

$$Z_n = 2^{-1}(Y_{n-1}|X| * \mathbf{2}^n + Z_{n-1})$$

原码乘法小结

- 乘积的符号位 = 被乘数的符号位 $\oplus$ 乘数的符号位
- 数值部分按绝对值相乘

特点：

- 绝对值运算
- 用移位的次数判断乘法是否结束
- 逻辑移位

补码一位乘法（小数）

- 补码一位乘运算规则

① 被乘数任意，乘数为正

$$[x \times y]_{\text{补}} = [x]_{\text{补}} \times y$$

② 被乘数任意，乘数为负

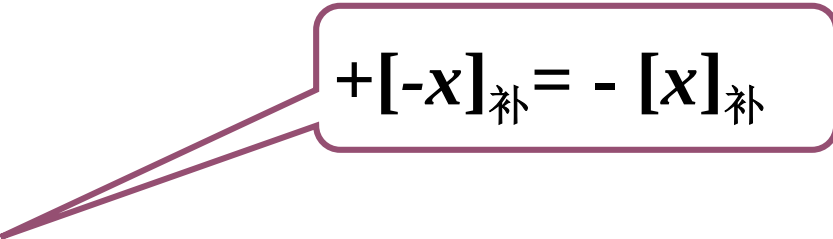
$$[x \times y]_{\text{补}} = [x]_{\text{补}} \times (0.y_1y_2 \dots y_n) + [-x]_{\text{补}}$$

$$[x]_{\text{补}} = x_0.x_1x_2 \dots x_n$$

$$[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$$

统一算法

$$\begin{aligned} [x \times y]_{\text{补}} &= [x]_{\text{补}} \times (0.y_1y_2 \dots y_n) + [-x]_{\text{补}} \times y_0 \\ &= [x]_{\text{补}} \times (0.y_1y_2 \dots y_n) - [x]_{\text{补}} \times y_0 \end{aligned}$$


$$+[-x]_{\text{补}} = -[x]_{\text{补}}$$

补码一位乘法运算规则（整数）

设 被乘数 $[X]_{\text{补}} = X_n \dots X_1 X_0$, 乘数 $[Y]_{\text{补}} = Y_n \dots Y_1 Y_0$

● 被乘数任意, 乘数为正 ($[X \times Y]_{\text{补}} = [X]_{\text{补}} \times [Y]_{\text{补}} \quad Y_n = 0$)

● 类似原码乘, 加和移位按补码规则, 积的符号自然形成

● 被乘数任意, 乘数为负

● 乘数 $[Y]_{\text{补}}$, 去掉符号位, 其他操作同上, 最后加 $2^n[-X]_{\text{补}}$ (校正)

$$\begin{aligned}[X \times Y]_{\text{补}} &= [X]_{\text{补}} \times [10_1 \dots 0_n + 0Y_{n-1} \dots Y_1 Y_0]_{\text{补}} \\ &= [X]_{\text{补}} \times [0Y_{n-1} \dots Y_1 Y_0]_{\text{补}} + (-2^n)[X]_{\text{补}} \\ &= [X]_{\text{补}} \times [0Y_{n-1} \dots Y_1 Y_0]_{\text{补}} + 2^n[-X]_{\text{补}}\end{aligned}$$

统一算法

$$\begin{aligned}[X \times Y]_{\text{补}} &= [X]_{\text{补}} \times (0Y_{n-1} \dots Y_1 Y_0) + 2^n[-X]_{\text{补}} \times Y_n \\ &= [X]_{\text{补}} \times (0Y_{n-1} \dots Y_1 Y_0) - 2^n[X]_{\text{补}} \times Y_n\end{aligned}$$

Booth 算法（小数、被乘数、乘数符号任意）

$$\text{设 } [x]_{\text{补}} = x_0.x_1x_2 \cdots x_n \quad [y]_{\text{补}} = y_0.y_1y_2 \cdots y_n$$

$$[x \times y]_{\text{补}}$$

$$= [x]_{\text{补}} (0.y_1 \cdots y_n) - [x]_{\text{补}} \cdot y_0$$

$$= [x]_{\text{补}} (y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n}) - [x]_{\text{补}} \cdot y_0$$

$$2^{-1} = 2^0 - 2^{-1}$$

$$= [x]_{\text{补}} (-y_0 + y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n})$$

$$2^{-2} = 2^{-1} - 2^{-2}$$

$$= [x]_{\text{补}} [-y_0 + (y_1 - y_1 2^{-1}) + (y_2 2^{-1} - y_2 2^{-2}) + \cdots + (y_n 2^{-(n-1)} - y_n 2^{-n})]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_n - y_{n-1}) 2^{-(n-1)} + (0 - y_n) 2^{-n}]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_{n+1} - y_n) 2^{-n}]$$

附加位 y_{n+1}

$$y'_1 2^{-1} + \cdots + y'_n 2^{-n}$$

Booth算法递推公式

$$[z_0]_{\text{补}} = 0 \quad y_{n+1} = 0$$

$$[z_1]_{\text{补}} = 2^{-1} \{ (y_{n+1} - y_n) [x]_{\text{补}} + [z_0]_{\text{补}} \}$$

$$[z_2]_{\text{补}} = 2^{-1} \{ (y_n - y_{n-1}) [x]_{\text{补}} + [z_1]_{\text{补}} \}$$

.....

$$[z_i]_{\text{补}} = 2^{-1} \{ (y_{n+2-i} - y_{n+1-i}) [x]_{\text{补}} + [z_{i-1}]_{\text{补}} \}$$

.....

$$[z_n]_{\text{补}} = 2^{-1} \{ (y_2 - y_1) [x]_{\text{补}} + [z_{n-1}]_{\text{补}} \}$$

$$[x \times y]_{\text{补}} = (y_1 - y_0) [x]_{\text{补}} + [z_n]_{\text{补}} \quad \text{最后一步不移位}$$

y_i	y_{i+1}	$y_{i+1} - y_i$	操作
0	0	0	$[z_{n-i}]_{\text{补}} \rightarrow 1$
0	1	1	$[z_{n-i}]_{\text{补}} + [x]_{\text{补}} \rightarrow 1$
1	0	-1	$[z_{n-i}]_{\text{补}} + [-x]_{\text{补}} \rightarrow 1$
1	1	0	$[z_{n-i}]_{\text{补}} \rightarrow 1$

Booth算法（整数，被乘数、乘数符号任意）

设 $[X]_{\text{补}} = X_n X_{n-1} \dots X_0$, $[Y]_{\text{补}} = Y_n Y_{n-1} \dots Y_0$ 。

$$\begin{aligned}
 [X \times Y]_{\text{补}} &= [X]_{\text{补}} \times (0Y_{n-1} \dots Y_1 Y_0) - 2^n [X]_{\text{补}} \times Y_n \\
 &= 2^n * ([X]_{\text{补}} (Y_{n-1} 2^{-1} + Y_{n-2} 2^{-2} + \dots + Y_0 2^{-n}) + [X]_{\text{补}} \times (-Y_n)) \quad \text{2进制转换并提取} 2^n \\
 &= 2^n * ([X]_{\text{补}} (-Y_n + Y_{n-1} 2^{-1} + Y_{n-2} 2^{-2} + \dots + Y_0 2^{-n})) \quad \underline{2^{-i+1} - 2^{-i} = 2 * 2^{-i} - 2^{-i} = 2^{-i}} \\
 &= 2^n * ([X]_{\text{补}} [-Y_n + (Y_{n-1} 2^0 - Y_{n-1} 2^{-1}) + (Y_{n-2} 2^{-1} - Y_{n-2} 2^{-2}) + \dots + (Y_0 2^{-(n-1)} - Y_0 2^{-n})]) \\
 &= 2^n * ([X]_{\text{补}} [(Y_{n-1} - Y_n) + (Y_{n-2} - Y_{n-1}) 2^{-1} + (Y_{n-3} - Y_{n-2}) 2^{-2} + \dots + (Y_0 - Y_1) 2^{-(n-1)} + \\
 &\quad (Y_{-1} - Y_0) 2^{-n}]) \quad \text{附加位 } Y_{-1} = 0 \\
 &= 2^n * (Y_{n-1} - Y_n) [X]_{\text{补}} + 2^{-1} \{ 2^n * (Y_{n-2} - Y_{n-1}) [X]_{\text{补}} + 2^{-1} \{ 2^n * (Y_{n-3} - Y_{n-2}) [X]_{\text{补}} \dots + \\
 &\quad 2^{-1} \{ 2^n * (Y_{-1} - Y_0) [X]_{\text{补}} + 0 \} \dots \} \} \\
 &\quad [Z_1]_{\text{补}} \quad Z_0 = 0
 \end{aligned}$$

Booth整数算法递推公式

$$[Z_0]_{\text{补}} = 0 \quad Y_{-1} = 0$$

$$[Z_1]_{\text{补}} = 2^{-1} \{ 2^n * (Y_{-1} - Y_0) [X]_{\text{补}} + [Z_0]_{\text{补}} \}$$

$$[Z_2]_{\text{补}} = 2^{-1} \{ 2^n * (Y_0 - Y_1) [X]_{\text{补}} + [Z_1]_{\text{补}} \}$$

.....

$$[Z_{i+1}]_{\text{补}} = 2^{-1} \{ 2^n * (Y_{i-1} - Y_i) [X]_{\text{补}} + [Z_i]_{\text{补}} \}$$

.....

$$[Z_n]_{\text{补}} = 2^{-1} \{ 2^n * (Y_{n-2} - Y_{n-1}) [X]_{\text{补}} + [Z_{n-1}]_{\text{补}} \}$$

$$[X \times Y]_{\text{补}} = 2^n * (Y_{n-1} - Y_n) [X]_{\text{补}} + [Z_n]_{\text{补}} \quad \text{最后一步不移位}$$

$Y_i Y_{i-1}$	$Y_{i-1} - Y_i$	操作
0 0	0	$[Z_i]_{\text{补}} \rightarrow 1$
0 1	1	$[Z_i]_{\text{补}} + 2^n * [X]_{\text{补}} \rightarrow 1$
1 0	-1	$[Z_i]_{\text{补}} + 2^n * [-X]_{\text{补}} \rightarrow 1$
1 1	0	$[Z_i]_{\text{补}} \rightarrow 1$

$X=+0011, Y=-1011$, 用booth算法求 $[X \times Y]_{补}$

部分积(乘数暂存)		丢弃	说明
$0, 0000$	$1, 0101$	0	附加位 $Y_{-1}=0, Z_0=0$
$+1, 1101$			$Y_0Y_{-1}=10$, 加 $2^n*[-X]_{补}$
$1, 1101$	10101		算术右移 $\rightarrow 1$
$1, 1110$	11010	10	$[Z_1]_{补}$
$+0, 0011$			$Y_1Y_0=01$, 加 $2^n*[X]_{补}$
$0, 0001$	11010		算术右移 $\rightarrow 1$
$0, 0000$	11101	010	$[Z_2]_{补}$
$+1, 1101$			$Y_2Y_1=10$, 加 $2^n*[-X]_{补}$
$1, 1101$	11101		算术右移 $\rightarrow 1$
$1, 1110$	11110	1010	$[Z_3]_{补}$
$+0, 0011$			$Y_3Y_2=01$, 加 $2^n*[X]_{补}$
$0, 0001$	11110		算术右移 $\rightarrow 1$
$0, 0000$	11111	01010	$[Z_4]_{补}$
$+1, 1101$			$Y_4Y_3=10$, 加 $2^n*[-X]_{补}$
$1, 1101$	1111		结束(最后一步不移位)

$[X]_{补} = 0,0011$
 $[Y]_{补} = 1,0101$
 $[-X]_{补} = 1,1101$

Y_iY_{i-1}	$Y_{i-1}-Y_i$	操作
0 0	0	$[Z_i]_{补} \rightarrow 1$
0 1	1	$[Z_i]_{补} + 2^n*[X]_{补} \rightarrow 1$
1 0	-1	$[Z_i]_{补} + 2^n*[-X]_{补} \rightarrow 1$
1 1	0	$[Z_i]_{补} \rightarrow 1$

$\therefore [X \times Y]_{补}$
 $= 1,11011111$

乘法小结

- 整数乘法与小数乘法基本相同
 - 用 逗号 代替小数点
 - 整数有整体移位操作，部分积最开始保存在积的高位
- 符号位：原码乘需要单独处理，补码乘 自然形成
- 原码乘去掉符号位运算，即为无符号数乘法
- 不同的乘法运算需有不同的硬件支持

第二章 计算机的运算方法

2.1 计算机中数的表示

2.2 定点运算

2.3 浮点运算

浮点四则运算

• 一、浮点加减运算

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

1. 对阶

(1) 求阶差

$$\Delta j = j_x - j_y = \begin{cases} = 0 & j_x = j_y & \text{已对齐} \\ > 0 & j_x > j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \leftarrow 1, j_x - 1 \\ y \text{ 向 } x \text{ 看齐} & \checkmark S_y \rightarrow 1, j_y + 1 \end{cases} \\ < 0 & j_x < j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & \checkmark S_x \rightarrow 1, j_x + 1 \\ y \text{ 向 } x \text{ 看齐} & S_y \leftarrow 1, j_y - 1 \end{cases} \end{cases}$$

(2) 对阶原则

小阶向大阶看齐

• 例: $x = 0.1101 \times 2^{01}, y = (-0.1010) \times 2^{11}$, 求 $x + y$

解: $[x]_{\text{补}} = 00, 01; 00.1101$ $[y]_{\text{补}} = 00, 11; 11.0110$

1. 对阶

$$\textcircled{1} \text{ 求阶差 } [\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 00, 01$$

$$\begin{array}{r} + \quad 11, 01 \\ \hline 11, 10 \end{array}$$

阶差为负 (-2) $\therefore S_x \rightarrow 2 \quad j_x + 2$

$$\textcircled{2} \text{ 对阶 } [x]_{\text{补}'} = 00, 11; 00.0011$$

2. 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}'} = 00.0011 \quad \text{对阶后的}[S_x]_{\text{补}'} \\ + \quad [S_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \\ \therefore [x+y]_{\text{补}} = 00, 11; 11.1001 \end{array}$$

左规和右规

- 原码规格化判断方法：不论正数、负数，第一数位为1
- 补码规格化判断方法：符号位和第一数位不同
- 补码左规：尾数每左移1位，阶码减1，直到数符与第一数位不同。

$$[x+y]_{\text{补}} = 00, 11; 11. 1001$$

$$\text{左规后 } [x+y]_{\text{补}} = 00, 10; 11. 0010$$

$$\therefore x + y = (-0.1110) \times 2^{10}$$

- 右规：当尾数溢出（>1）时，需右规

例如：双符号位的尾数出现 01. $\times \dots \times$ 或 10. $\times \dots \times$ 时，
尾数每右移一位，阶码加 1。

- 例. $x = (-\frac{5}{8}) \times 2^{-5}$, $y = (\frac{7}{8}) \times 2^{-4}$, 求 $x - y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $x = (-0.101000) \times 2^{-101}$ $y = (0.111000) \times 2^{-100}$

$$[x]_{\text{补}} = 11, 011; 11. 011000 \quad [y]_{\text{补}} = 11, 100; 00. 111000$$

① 对阶

$$\begin{aligned} [\Delta j]_{\text{补}} &= [j_x]_{\text{补}} - [j_y]_{\text{补}} = 11, 011 \\ &\quad + 00, 100 \\ &\hline &\quad 11, 111 \end{aligned}$$

$$\text{阶差为 } -1 \quad \therefore S_x \longrightarrow 1, j_x + 1$$

$$\therefore [x]_{\text{补}'} = 11, 100; 11. 101100$$

② 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}'} = 11.101100 \\ + [-S_y]_{\text{补}} = 11.001000 \\ \hline 110.110100 \end{array}$$

③ 右规

$$[x - y]_{\text{补}} = 11, 100; 10.110100$$

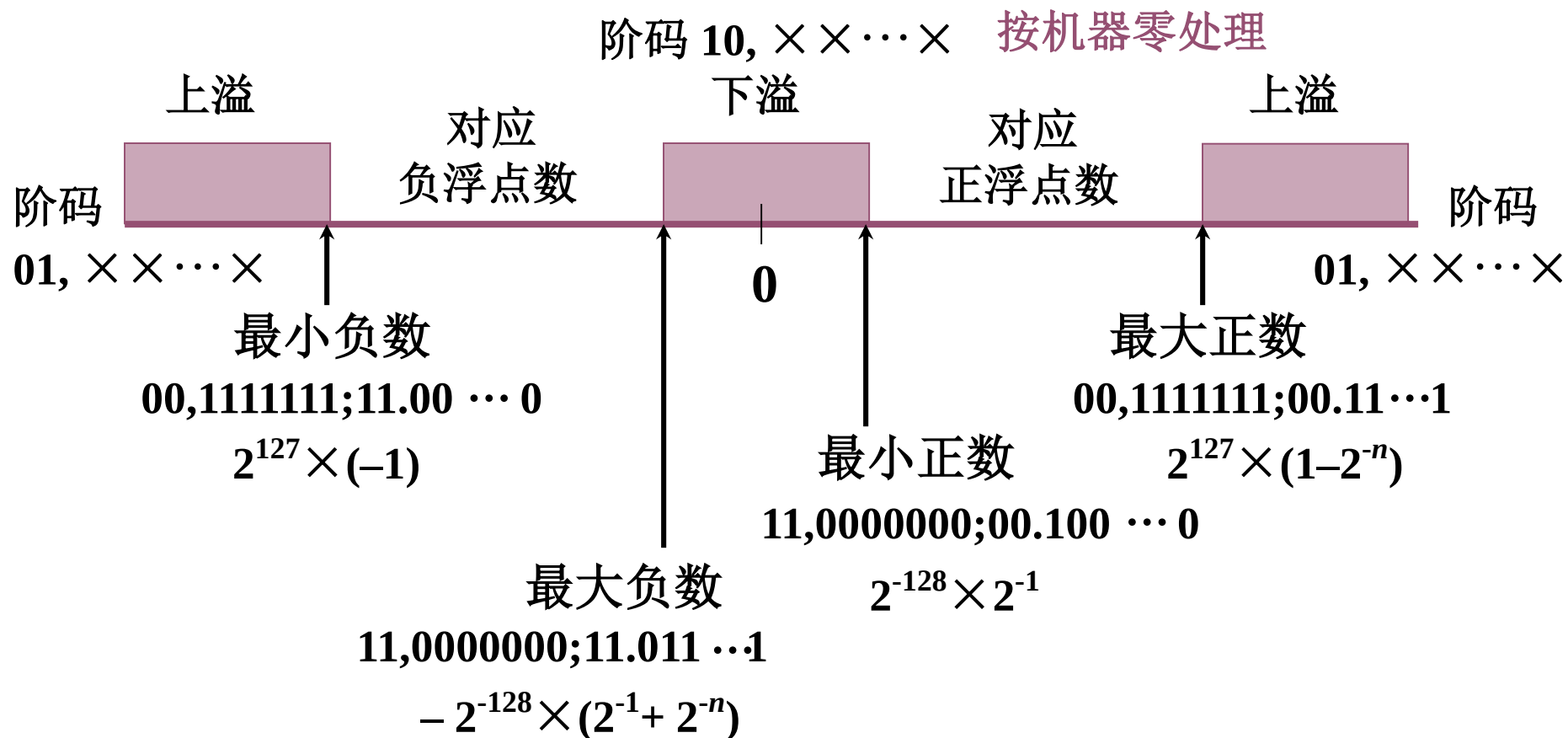
右规后

$$[x - y]_{\text{补}} = 11, 101; 11.011010$$

$$\begin{aligned} \therefore x - y &= (-0.100110) \times 2^{-11} \\ &= \left(-\frac{19}{32}\right) \times 2^{-3} \end{aligned}$$

溢出判断

- 设机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码的数值部分取 7 位，数符取 2 位，尾数取 n 位，则该补码在数轴上的表示为



第三章 RISC-V汇编及其指令系统

- **RISC-V概述**
- RISC-V汇编语言
- RISC-V指令表示
- 案例分析

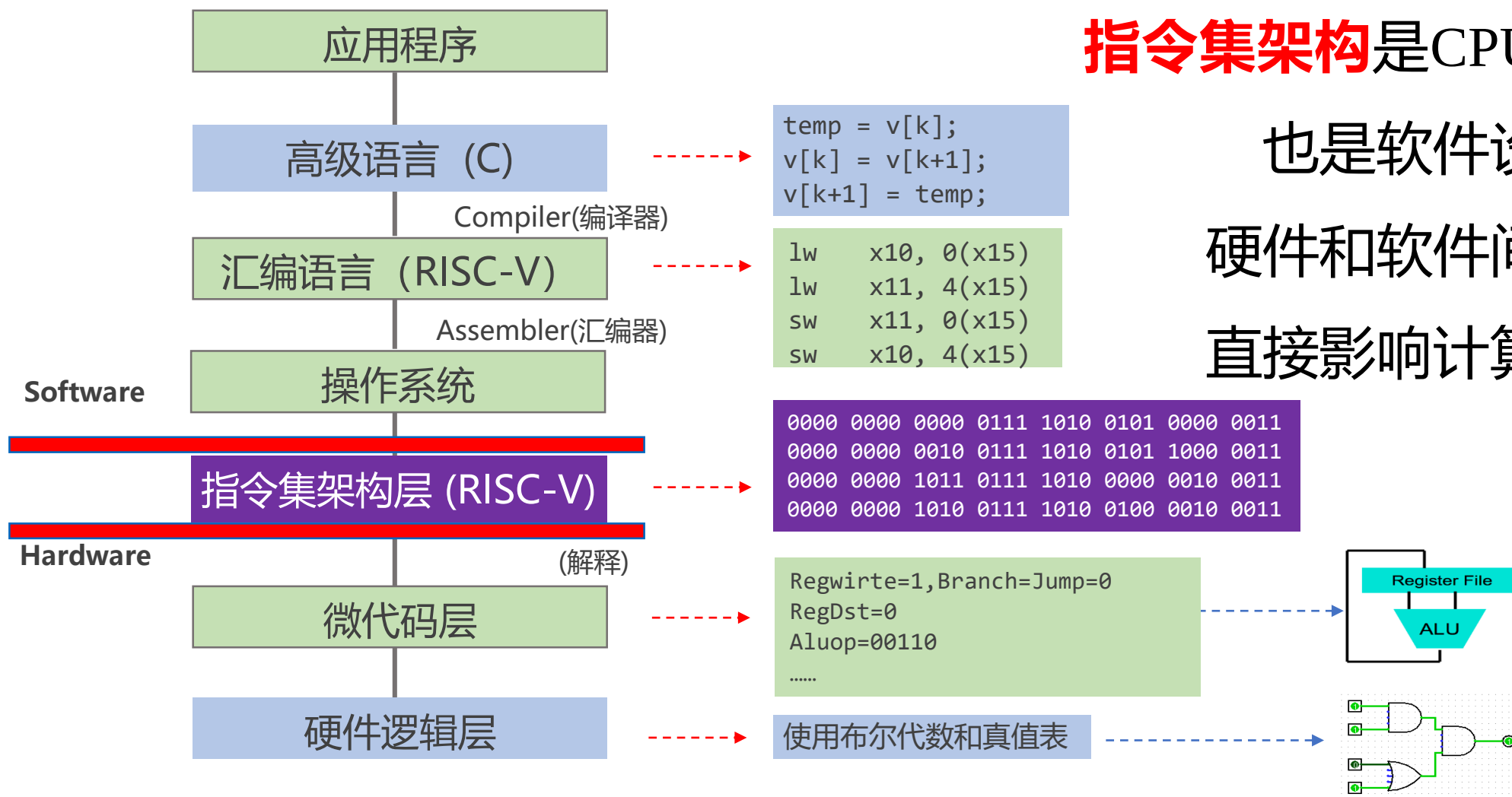
第三章 RISC-V汇编及其指令系统

- **RISC-V概述**
 - 指令系统的基本概念
 - 主流指令集及发展方向
 - RISC-V指令集

指令系统基本概念

- 机器指令（指令）
 - 计算机能直接识别、执行的某种操作命令，它是一堆二进制代码
- 指令系统（指令集，**IS: Instruction Set**）
 - 一台计算机中所有机器指令的集合
- 指令集系统架构（**ISA: Instruction Set Architecture**）
 - 包含了程序员正确编写二进制机器语言程序所需的全部信息。
 - 具体来说，它规定了如何使用硬件、指令格式，操作种类、操作数所能存放的寄存器组 and 结构，包括每个寄存器名称、编号、长度和用途等。
- 系列机
 - 基本指令系统相同，基本系统结构相同的计算机
 - IBM 360 是第一个将ISA与其实现分离的系列机
 - 解决软件兼容的问题给定一个ISA，可以有不同的实现方式；例如AMD/Intel CPU 都是X86-64指令集。ARM ISA 也有不同的实现方式

计算机指令系统层次



指令集架构是CPU设计的依据、
也是软件设计的基础、
硬件和软件间的分界面、
直接影响计算机系统性能

指令系统的评价

- 指令系统的评价

- 方便硬件设计，方便编译器实现，性能更优，成本功耗更低

- 性能要求

- 完备性：指令丰富，功能齐全，使用方便
 - 高效性：程序占空间小，执行速度快
 - 规整性：RISC-V指令长度是32位和16位的压缩指令
(关于规整性，X86中还包括对称性、匀齐性、一致性的定义)
 - 兼容性：系列机软件向上兼容

精减指令系统(RISC)

- 指令条数少，保留使用频率最高的简单指令，指令定长
 - 便于硬件实现，用软件实现复杂指令功能
- Load/Store架构
 - 只有存/取数指令才能访问存储器，其余指令的操作都在寄存器之间进行，便于硬件实现
- 指令长度固定，指令格式简单、寻址方式简单
 - 便于硬件实现
- CPU设置大量寄存器（32~192）
 - 便于编译器实现
- RISC CPU采用硬布线控制，CISC采用微程序
- 一个时钟周期完成一条机器指令（单周期模型）

CISC & RISC

- ISA的功能设计
 - 任务：确定硬件支持哪些操作
 - 方法：统计的方法
 - 两种类型：CISC和RISC
- CISC（Complex Instruction Set Computer）
 - 目标：强化指令功能，减少运行的指令条数，提高系统性能
 - 方法：面向目标程序的优化，面向高级语言和编译器的优化
- RISC（Reduced Instruction Set Computer）
 - 目标：通过简化指令系统，用高效的方法实现最常用的指令
 - 方法：充分发挥流水线的效率，降低（优化）CPI

第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**
 - 汇编语言简介
 - RISC-V汇编指令概览
 - RISC-V常用汇编指令
 - 函数调用及栈的使用
 - 编译工具介绍

汇编语言是什么？

- 汇编语言（**Assembly Language**）是一种“低级”语言，直接接触最底层的硬件，需要对底层硬件非常熟悉才能编写出高效的汇编程序。
- 汇编语言属于第二代计算机语言，用一些容易理解和记忆的字母，单词来代替一个特定的指令。在汇编语言中，用助记符代替机器指令的操作码，用地址符号或标号代替指令或操作数的地址。
- 例如：用“ADD”代表数字逻辑上的加减，“MOV”代表数据传递等等。

汇编语言简介

- C语言程序在翻译成可以在计算机上运行的机器语言程序，大致需要四个步骤。
 - 首先将高级语言程序编译成汇编语言程序
 - 然后用机器语言组装成目标模块
 - 链接器将多个模块与库程序组合在一起以解析所有引用
 - 最后加载器将机器代码放入适当的存储器位置以供处理器执行

注：所有程序不一定严格按照这个四个步骤执行。为了加快转换过程，可以跳过或将一些步骤组合到一起。一些编译器直接生成目标模块，一些系统使用链接加载器执行最后两个步骤。

RISC-V 汇编指令格式

- 通用指令格式

op dst, src1, src2

- 1个操作码，3个操作数
- op 操作的名字（**o**peration）
- dst 目标寄存器（destination）
- src1 第一个源操作数寄存器（source）
- src2 第二个源操作数寄存器
- 通过一些限制来保持硬件简单

RISC-V汇编指令操作对象

- 寄存器:

- 32个通用寄存器, $x0 \sim x31$ (**注意: 仅涉及 RV32I 的通用寄存器组**)
- 在 RISC-V 中, 算术逻辑运算所操作的数据必须直接来自寄存器
- $x0$ 是一个特殊的寄存器, 只用于全零
- 每一个寄存器都有别名便于软件使用, 实际硬件并没有任何区别
- **RV32I指令集通用寄存器是32位整数寄存器, RV64I指令集, 通用寄存器是64位寄存器**

- 内存:

- 可以执行在寄存器和内存之间的数据读写操作
- 读写操作使用字节 (Byte) 为基本单位进行寻址
- RV64可以访问最多 2^{64} 个字节的内存空间, 即 2^{61} 个存储单元

汇编语言的变量---寄存器

- 汇编语言不能使用变量（C、JAVA可以）
 - `int a; float b;`
 - 寄存器变量没有数据类型
- 汇编语言的操作对象以寄存器为主
 - 好处：寄存器是最快的数据单元
 - 缺陷：寄存器数量有限，需要充分和高效地使用各寄存器

32个RISC-V寄存器

寄存器	助记符	备注
x0	zero	固定值为0
x1	ra	返回地址(Return Address)
x2	sp	栈指针(Stack Pointer)
x3	gp	全局指针(Global Pointer)
x4	tp	线程指针(Thread Pointer)
x5-x7	t0-t2	临时寄存器 (temporary register)
x8	s0/fp	save寄存器/帧指针(Frame Pointer)
x9	s1	save寄存器 (Save Register)
x10-x11	a0-a1	函数参数 / 函数返回值(Return Value)
x12-x17	a2-a7	函数参数(Function Argument)
x18-x27	s2-s11	save寄存器
x28-x31	t3-6	临时寄存器

RISC-V汇编语言指令分类

- 算术运算: add、sub、add*i*、mul、div 后缀*i* = immediate
- 逻辑运算: and、or、xor、and*i*、ori、xori
- 移位操作: sll、srl、sra、sll*i*、srli、srai 后缀i = unsigned
- 数据传输: ld、sd、lw、sw、lwu、lh、lhi、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、slti、sltii
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

伪指令: 方便程序员使用的更加直观的指令，以上指令的组合。

伪指令 vs. 硬件指令

- 硬件指令
 - 所有指令都是硬件可以直接执行的指令，在硬件中直接实现了
- 伪指令
 - 汇编语言程序员可以使用的指令（加上了伪指令）
 - 每一条伪指令指令会被翻译为1条或者多条硬件指令

字节排布

- ▶ 大端机：最高的字节在最低的地址，字的地址等于最高字节的地址
- ▶ 小端机：最低的字节在最低的地址，字的地址等于最低字节的地址

$*(x1) = 0x00000180$ #代表x1寄存器中存的数值

高地址  低地址

80	01	00	00
----	----	----	----

Big Endian

00	00	01	80
----	----	----	----

Little Endian

RISC-V 是小端机

函数调用

- 函数（过程）调用的6个步骤
 - 将参数放在过程函数可以访问到的位置（x10~x17）
 - 将控制转交给函数
 - 获取函数所需的存储资源
 - 执行过程中的操作
 - 将结果值放在调用程序可以访问到的寄存器
 - 返回调用位置（x1中保存的地址）

函数调用

- 调用子程序包含两个参与者
 - 调用者（caller）

准备函数参数，跳转到被调用者子程序
 - 被调用者（callee）

使用调用者提供的参数，然后运行
运行结束保存返回值
将控制（如跳回）还给调用者。

栈的使用

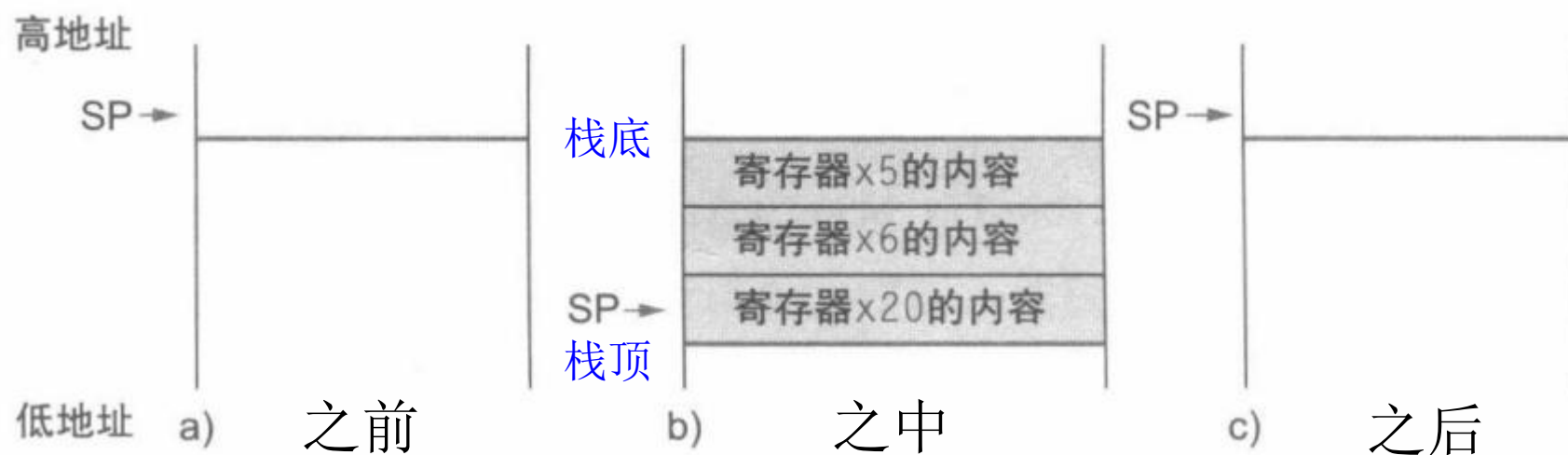
- ◆ 高级语言函数体中一般使用局部变量
- ◆ RISC-V汇编子程序使用寄存器（全局变量）
- ◆ 调用者函数和被调用函数可能使用相同寄存器
 - 造成数据破坏
 - 被调用函数需要保存可能被破坏的寄存器（现场）
 - 哪些寄存器属于现场？

栈的使用

- 在调用函数之前需要一个保存旧值的地方，在返回时还原它们，然后删除。在RISC-V中使用栈来完成这个功能。
- 堆栈：后进先出（LIFO）队列
 - push: 将数据放入堆栈
 - pop: 从堆栈中删除数据

栈的使用

- 下图展示了在函数调用之前、之中和之后栈的情况



- sp (x2寄存器) 是RISC-V中的栈指针寄存器，用来保存栈顶的地址。
- 按**历史惯例**，RISC-V中栈按照从高到低的地址顺序增长（即栈底位于高地址，栈顶位于低地址）。
- push 向下移动栈指针(减 sp), pop向上移动栈指针(增 sp)。

栈的使用

- 栈中需要保存那些数据？栈中存储的数据包括：
 - 需要保存的参数/save寄存器的值。
 - 函数执行完成后，返回下一条“指令”的地址。
 - 其他局部变量（例如函数中使用的数组或结构体）的空间
- 堆栈帧（一次过程调用中产生的数据）占据连续的内存块，栈指针指示堆栈帧底部（栈顶）的位置
- 当过程结束时，堆栈帧从栈中弹出，为将来的堆栈帧释放内存。

栈的使用

- 当被调用函数从执行中返回时，调用函数需要知道哪些寄存器可能已经更改，哪些寄存器保持不变。
- RISC-V函数调用**约定**将寄存器分为两类：
 - 在函数调用中保留 `sp, gp, tp, x8-x9, x18-x27`
 - 在函数调用中不保留 `x10-x17, x1(ra), x5-x7, x28-x31`

寄存器	助记符	注解
x0	zero	固定值为0
x1	<u>ra</u>	返回地址(Return Address)
x2	<u>sp</u>	栈指针(Stack Pointer)
x3	<u>gp</u>	全局指针(Global Pointer)
x4	<u>tp</u>	线程指针(Thread Pointer)
x5-x7	t0-t2	临时寄存器
x8	s0/ <u>fp</u>	save寄存器/帧指针(Frame Pointer)
x9	s1	save寄存器
x10-x11	a0-a1	函数参数 / 函数返回值
x12-x17	a2-a7	函数参数
x18-x27	s2-s11	save寄存器
x28-x31	t3-6	临时寄存器

第三章 RISC-V汇编及其指令系统

- RISC-V概述
- RISC-V汇编语言
- **RISC-V指令表示**
- 案例分析

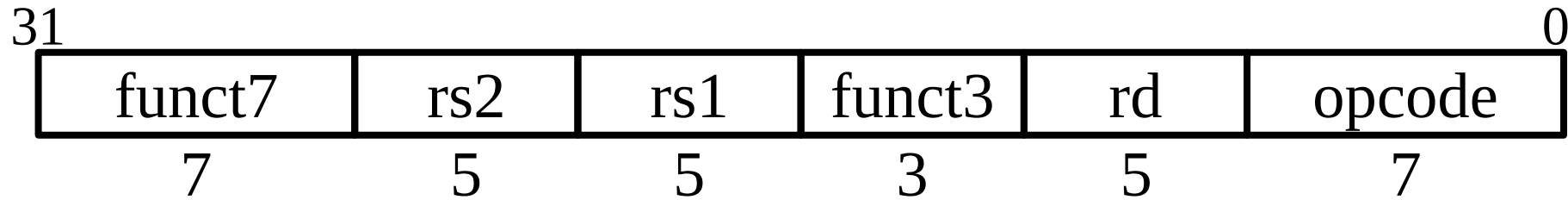
RISC-V指令表示

- 一个字是32位，所以把指令字分成“**字段**”
- 每个字段都告诉处理器一些关于指令的信息
- 理论上可以为**每条指令定义不同的字段**
- RISC-V定义了六种基本类型的**指令格式**:
 - **R型**指令 用于**寄存器 - 寄存器**操作
 - **I型**指令 用于**短立即数**和**取数 load** 操作
 - **S型**指令 用于**存数 store** 操作
 - **B型**指令 用于**条件跳转**操作
 - **U型**指令 用于**长立即数**操作
 - **J型**指令 用于**无条件跳转**操作

RISC-V指令格式

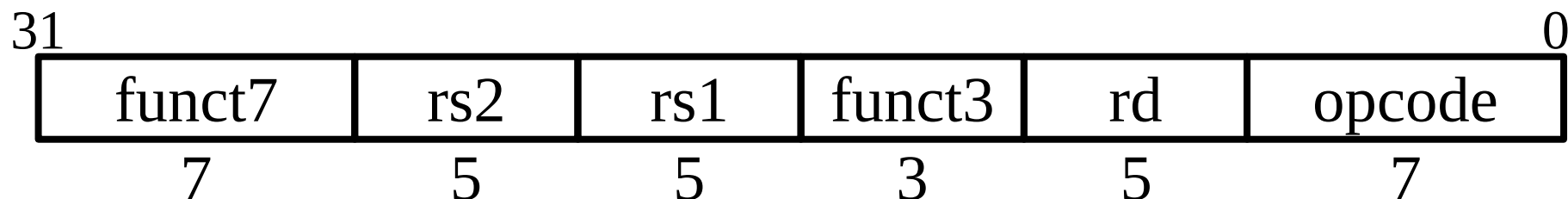
R 型	funct7	rs2	rs1	funct3	rd	opcode
I 型	imm[11:0]		rs1	funct3	rd	opcode
S 型	Imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B 型	Imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode
J 型	Imm[20,10:1,11,19:12]				rd	opcode
U 型	Imm[31:12]				rd	opcode

R型指令



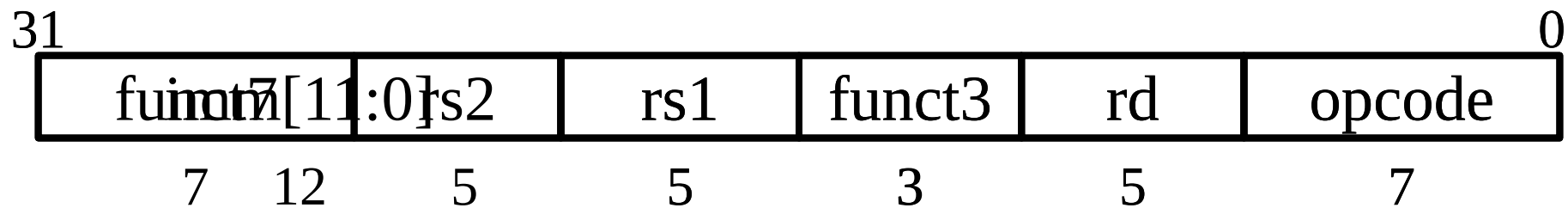
- 32位指令字，分为6个不同位数的字段
- **opcode(操作码)**: 指定它是什么指令
- **funct7+funct3**: 这两个字段结合操作码描述要执行的操作

R型指令



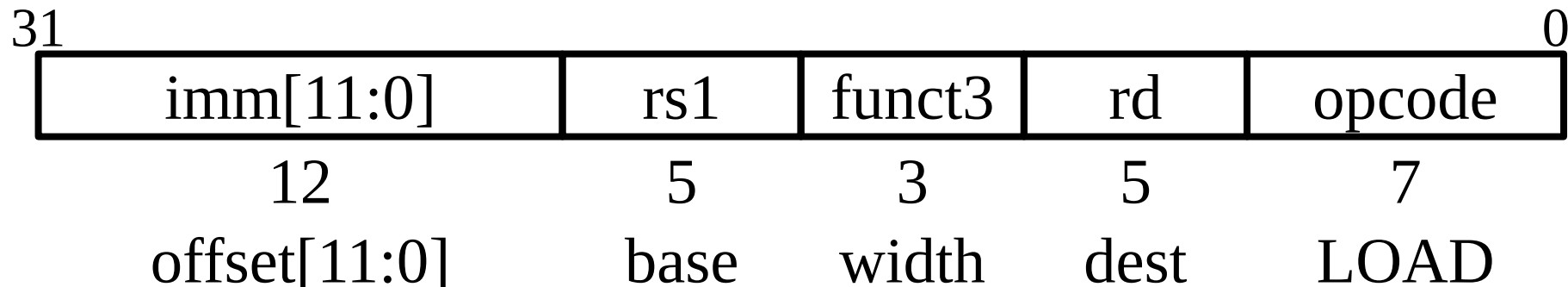
- **rs1**（源寄存器1）：指定第一个寄存器操作数
- **rs2**（源寄存器2）：指定第二个寄存器操作数
- **rd**（目的寄存器）：指定接收计算结果的寄存器
- 每个字段保存一个5位无符号整数（0-31），对应于一个寄存器号（x0-x31）（寄存器约定详见P75图2-14）

I型指令



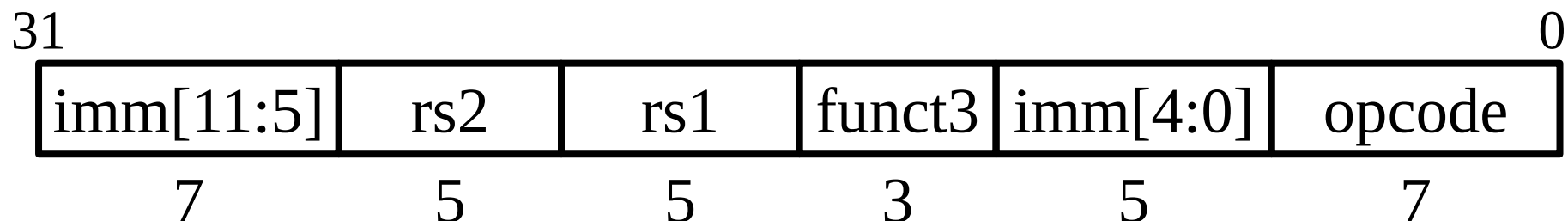
- 只有一个字段与R型指令不同，rs2和funct7被12位有符号立即数imm[11:0]替换
- 其余字段（rs1、funct3、rd、opcode）与之前相同
- 在算术运算前，立即数需符号扩展到32位（RV32）
- 如何处理大于12位的立即数？

I型指令



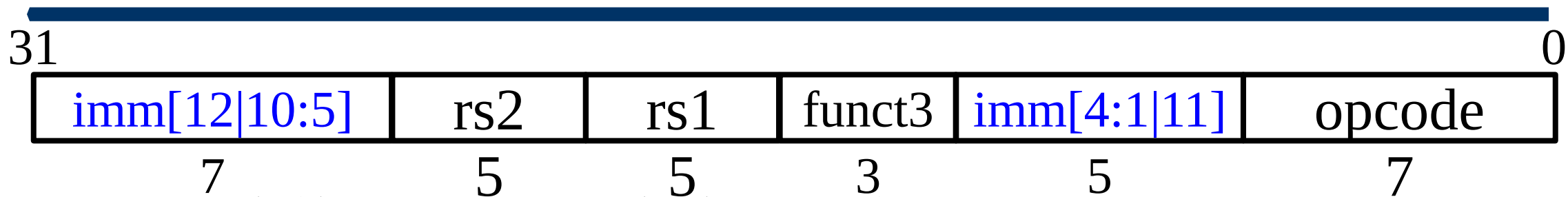
- **Load**指令也是I型指令
- **12位有符号**立即数加**寄存器rs1**的基址，形成**内存地址**
 - 这与add immediate操作非常相似，但用于**创建地址**
- 从**内存读取**到的值**存储**在寄存器rd中

S型指令



- 存储需要读取两个寄存器，**rs1**为**内存地址**，**rs2**为要**写入的数据**，以及立即数**偏移量offset**
- 12位immediate是分开的，要保证rs2**位置不变**。
- S型指令不会将值写入寄存器，所以没有rd
- RISC-V的设计决定是将**低位的5位**立即数移到其他指令中的**rd字段**所在的位置——保持rs1/rs2字段在**同一位置**

B型指令



- **B型**指令格式与**S型**指令格式基本相同
- PC相对寻址：将立即数字段作为相对PC的**补码偏移量**
- 立即数字段以**2字节**为增量表示**-4096**到**+4094**的偏移量
- 用**12位**立即数字段表示**13位有符号字节地址**的偏移量
 - 偏移量的**最低位**始终为**零**，因此无需存储

B型指令

- 为什么不使用字节作为PC的偏移量单位？
 - 因为指令是32位（4字节）
- 将偏移量视为以字为单位，在相对寻址前，先将偏移量乘以4,得到字节偏移量，再加上PC中的基地址
 - 可以访问到相对PC地址前后 $2^{12} \times 32$ 位范围内的所有指令
 - 比使用字节偏移量大四倍的转移范围

B型指令

- 基于RISC-V的扩展指令集支持16位压缩编码指令，也支持16位长度倍数的可变长度指令
- 为了实现对可能的扩展指令的支持，在实际的RISC-V指令设计中，分支转移指令的偏移量都是以2字节为单位
- RISC-V条件分支指令实际上只能访问相对PC地址前后 $2^{12} \times 16$ 位范围内的指令

U型指令



U-immediate[31:12]

dest

LUI

AUIPC

- ADDI指令中的立即数**最大**是多少？
- U型指令进行长立即数操作
 - **高20**位的立即数字段
 - 5位的目的地寄存器字段
- 两个指令
 - LUI(Load Upper Immediate)——将长立即数写入目的寄存器
 - AUIPC——将PC与长立即数相加结果写入目的寄存器

U型指令LUI的应用

- LUI将长立即数写入目的寄存器的高20位，并清除低12位。
- LUI与ADDI一起可在寄存器中创建任何32位值

LUI x10, 0x87654 # x10 = 0x87654000

ADDI x10, x10, 0x321 # x10 = 0x87654321

U型指令LUI的应用

- 如何创建 0xDEADBEEF?

LUI x10, 0xDEAD**B** # x10 = 0xDEADB000

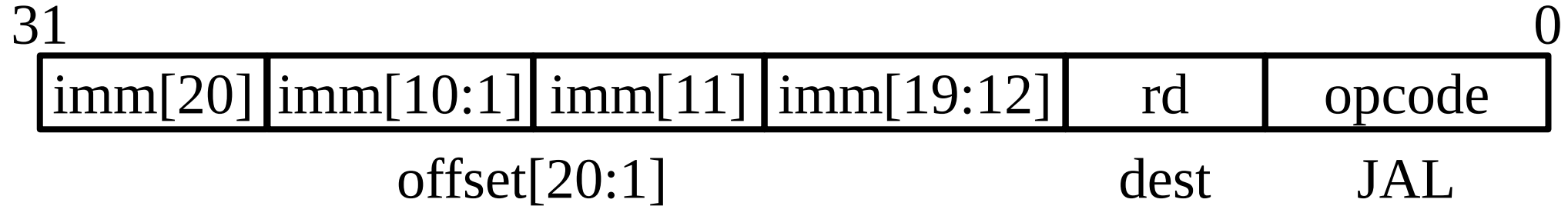
ADDI x10, x10, -0x111 (0xFFFFFEEF) # x10 = 0xDEAD**A**EEF?

- ADDI 立即数总是进行符号扩展，如果高位为1，将从高位的20位中减去1

LUI x10, 0xDEAD**C** # x10 = 0xDEADC000

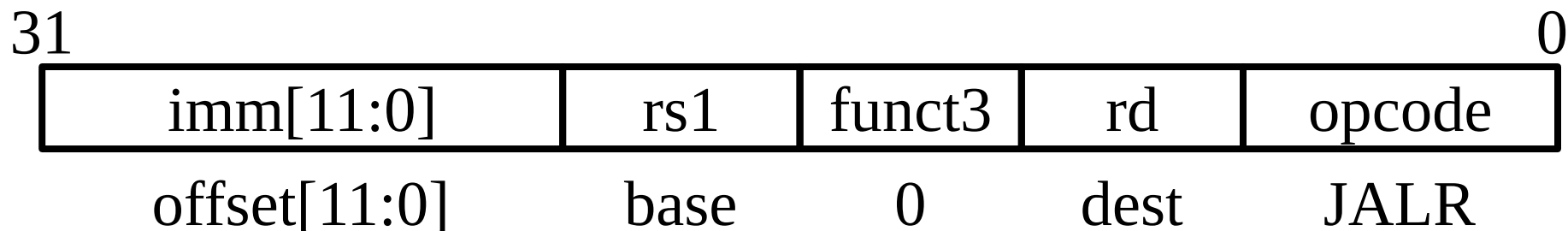
ADDI x10, x10, -0x111 (0xFFFFFEEF) # x10 = 0xDEAD**B**EEF

J型指令



- JAL将PC+4写入目的寄存器rd中
 - J是伪指令，等同于JAL,但设置rd = x0不保存返回地址
- $PC = PC + offset$ （PC相对寻址）
- 访问相对PC,以2字节为单位的 $\pm 2^{19}$ 范围内的地址空间
 - $\pm 2^{19}$ 16-bit指令空间
- 立即数字段编码的优化类似于B型指令，以降低硬件成本

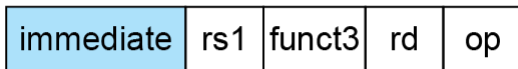
jalr指令



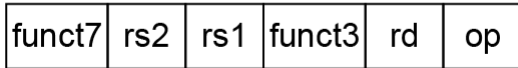
- JALR属于**I型**指令
- 将PC + 4保存在rd中
- 设置 $PC = rs + immediate$
- 与load指令得到地址的方式类似
- 与B型指令和jal不同，**不**将立即数 $\times 2$ 字节

RISC-V的寻址模式

1. Immediate addressing



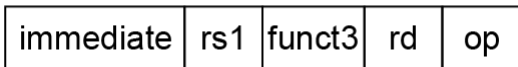
2. Register addressing



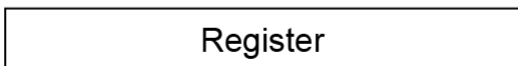
Registers

Register

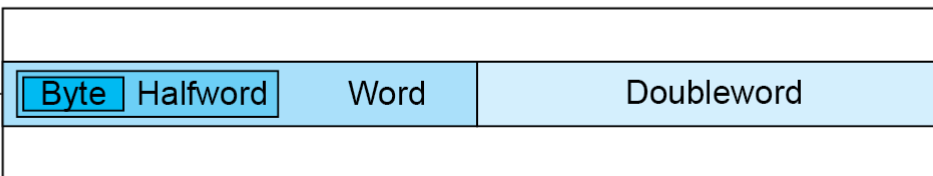
3. Base addressing



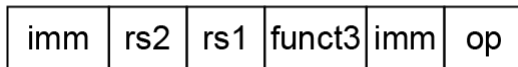
Memory



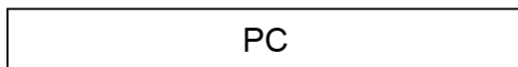
+



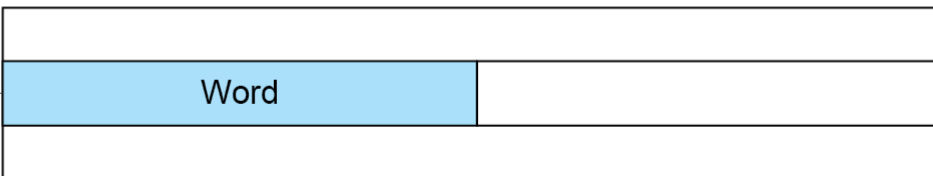
4. PC-relative addressing



Memory



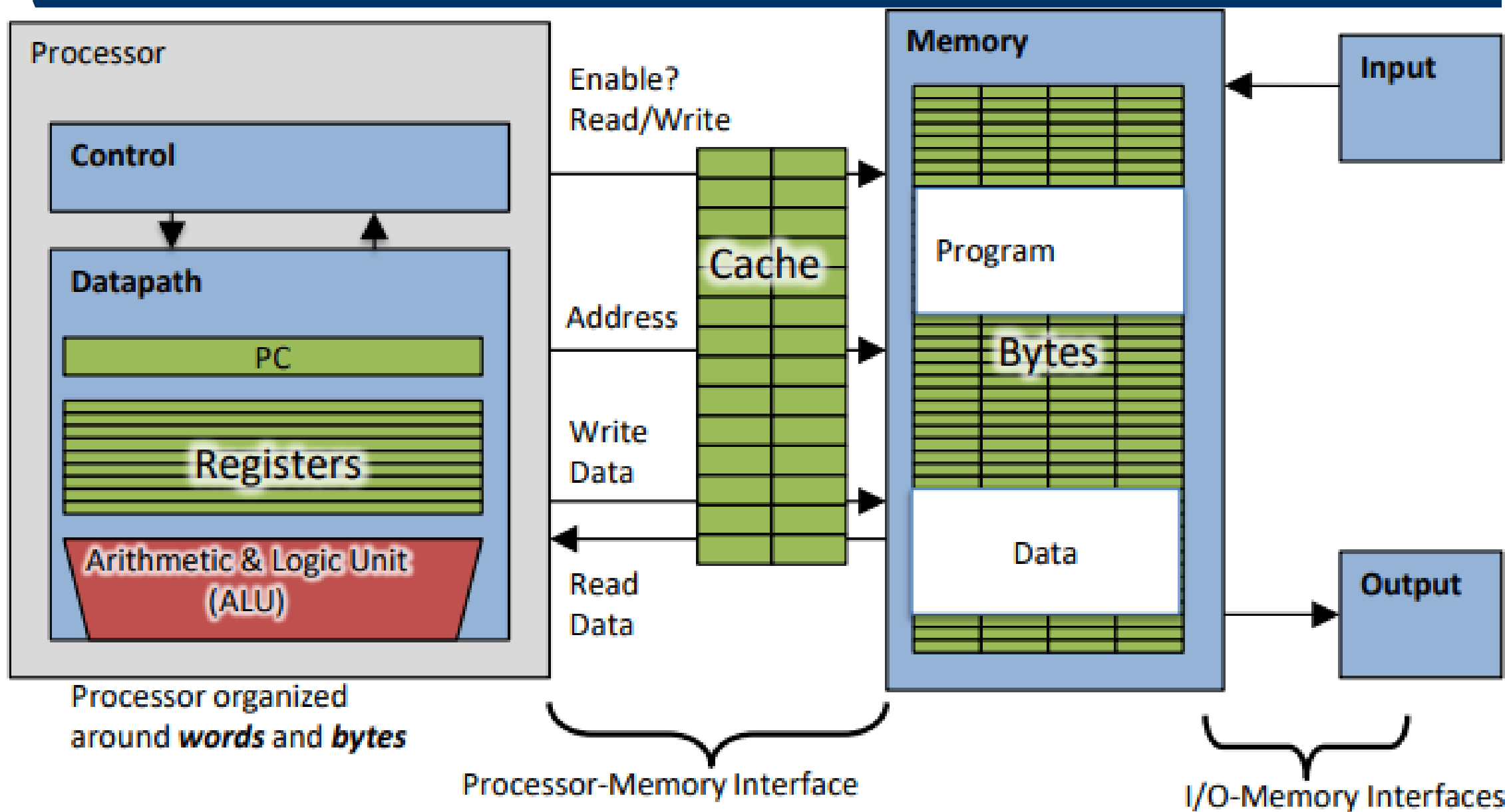
+



第4章 处理器设计

- 处理器设计重点
 - 数据通路与控制器的概念
 - 数据通路的组件（能看懂32位ALU的电路图）
 - 逐步设计数据通路，完整的RISC-V数据通路
 - 指令执行过程(每一个数据组件的作用，寄存器堆，立即数生成器等)，关键路径和指令时延
 - 如何基于ROM实现RISC-V的控制器（重点关注每条指令的控制信号真值表）

单核计算机系统



CPU功能层面的定义

- 构建一个能够通过输入的机器码，执行相应操作、并保持相应状态的 **数字电路**

RISC-V机器码
(例如: `addi t0, x0, 6`)

0x00600293

数字电路
(逻辑门、
寄存器)

执行完毕这条指令后，
寄存器t0的值为6

CPU的组成部分

- **数据通路**是处理器中执行所需操作的硬件

- 执行控制器的操作（例如，控制器告诉数据通路，执行add指令，则数据通路就会将操作数传递给加法器）

≈≈处理器的四肢

- **控制器**是对数据通路要做什么操作进行调度的硬件结构

- 告诉数据通路：需要执行什么操作？需要读内存吗？需要写寄存器吗？
写哪个寄存器？读哪个寄存器？

≈≈处理器的大脑

第四章 处理器设计

- 处理器设计的需求分析
- **RISC-V数据通路的组件选择**
- RISC-V部分指令的数据通路设计
- RISC-V控制器

存储组件——数据存储器

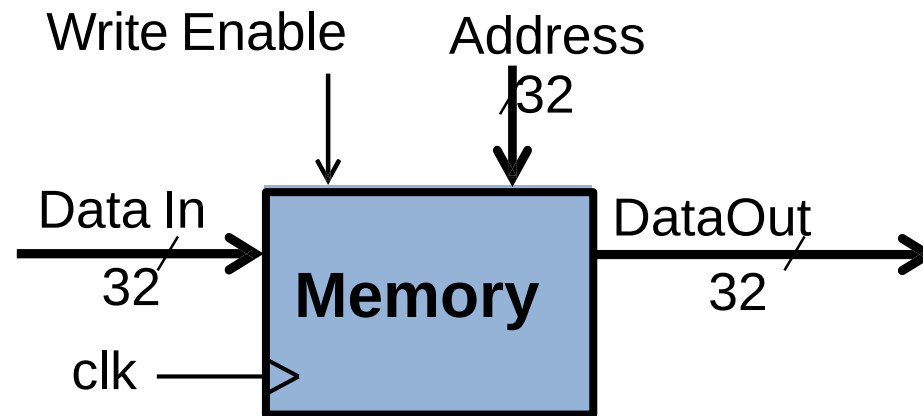
- 时序逻辑 **vs** 组合逻辑

- 数据接口信号

- Data In: 数据输入
- Data Out: 数据输出

- 读写控制

- Address: 指定一个存储单元，将其内容送到数据输出端
- WriteEnable: 写使能。在时钟信号（clk）的上升沿，如果写使能信号有效，将数据输入信号的内容存入地址信号指定存储单元

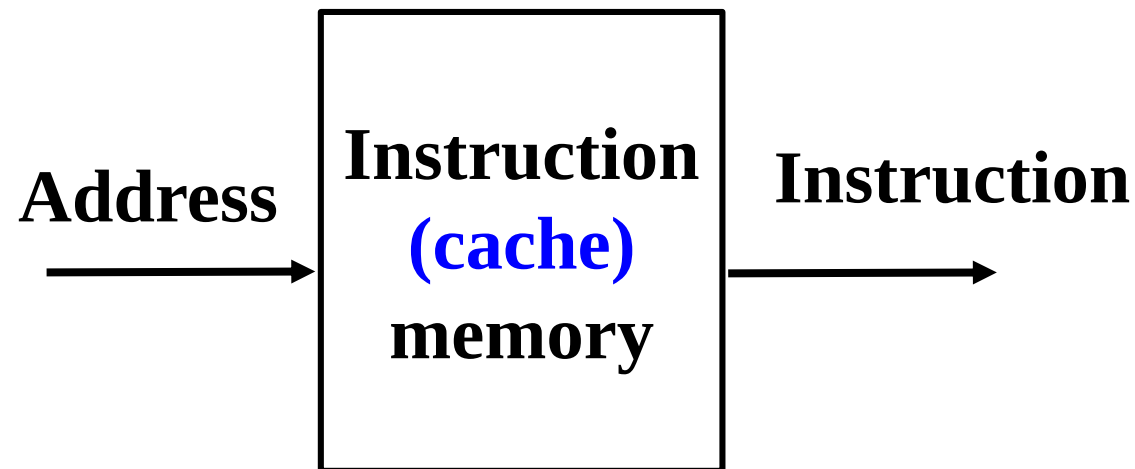


注：存储器的读操作不受时钟控制

存储组件——指令存储器

- 指令存储器

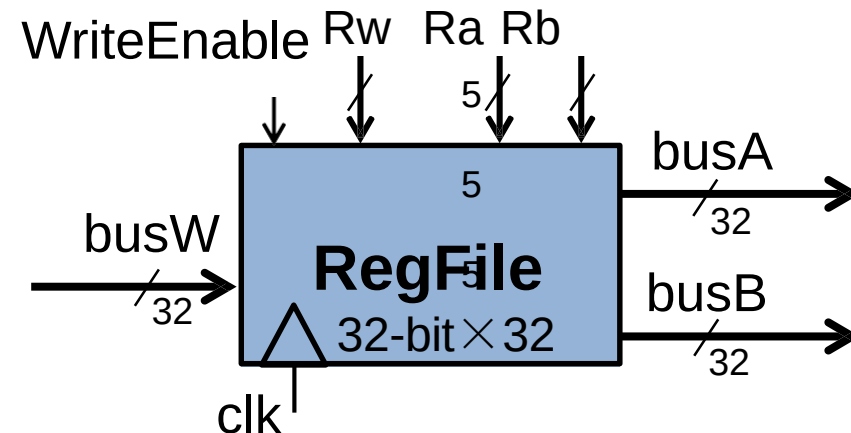
- 输入总线: Instruction Address
- 输出总线: Instruction



- 数据通路中**不会写**指令存储器
- 读操作时，指令存储器可以看做是**组合逻辑**电路

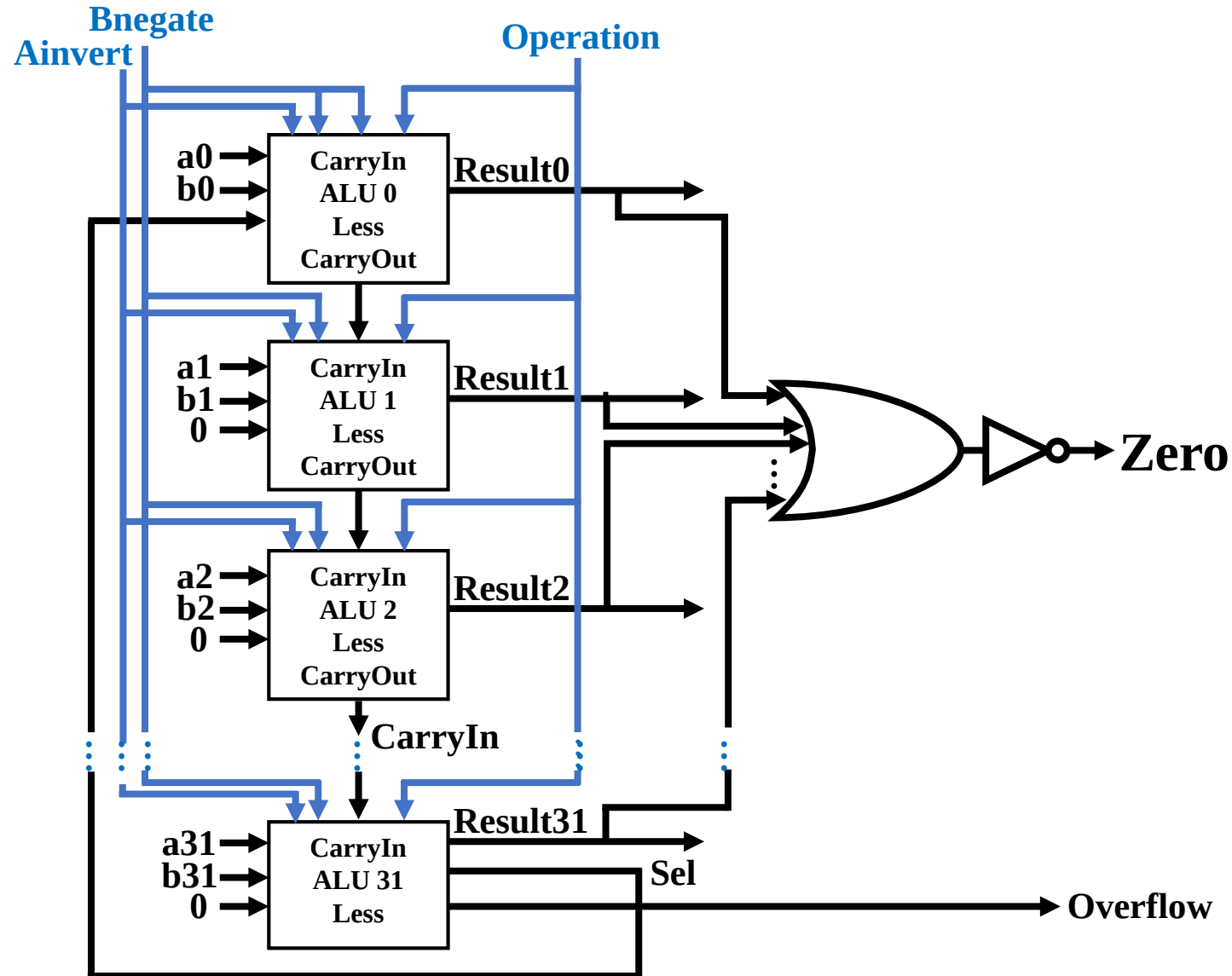
存储组件——寄存器堆

- 内部构成：32个寄存器
- 数据接口信号
 - busA, busB: 两个数据输出
 - busW: 一个数据输入
- 读写控制
 - Ra(5位): 选中对应编号的寄存器，将其内容放到busA(读)
 - Rb(5位): 选中对应编号的寄存器，将其内容放到busB(读)
 - Rw(5位): 选中对应编号的寄存器，在时钟信号（clk）的上升沿，如果写使能信号（WriteEnable）有效，将busW的内容存入该寄存器



注：寄存器堆的读操作不受时钟控制，组合逻辑电路。

32bit ALU

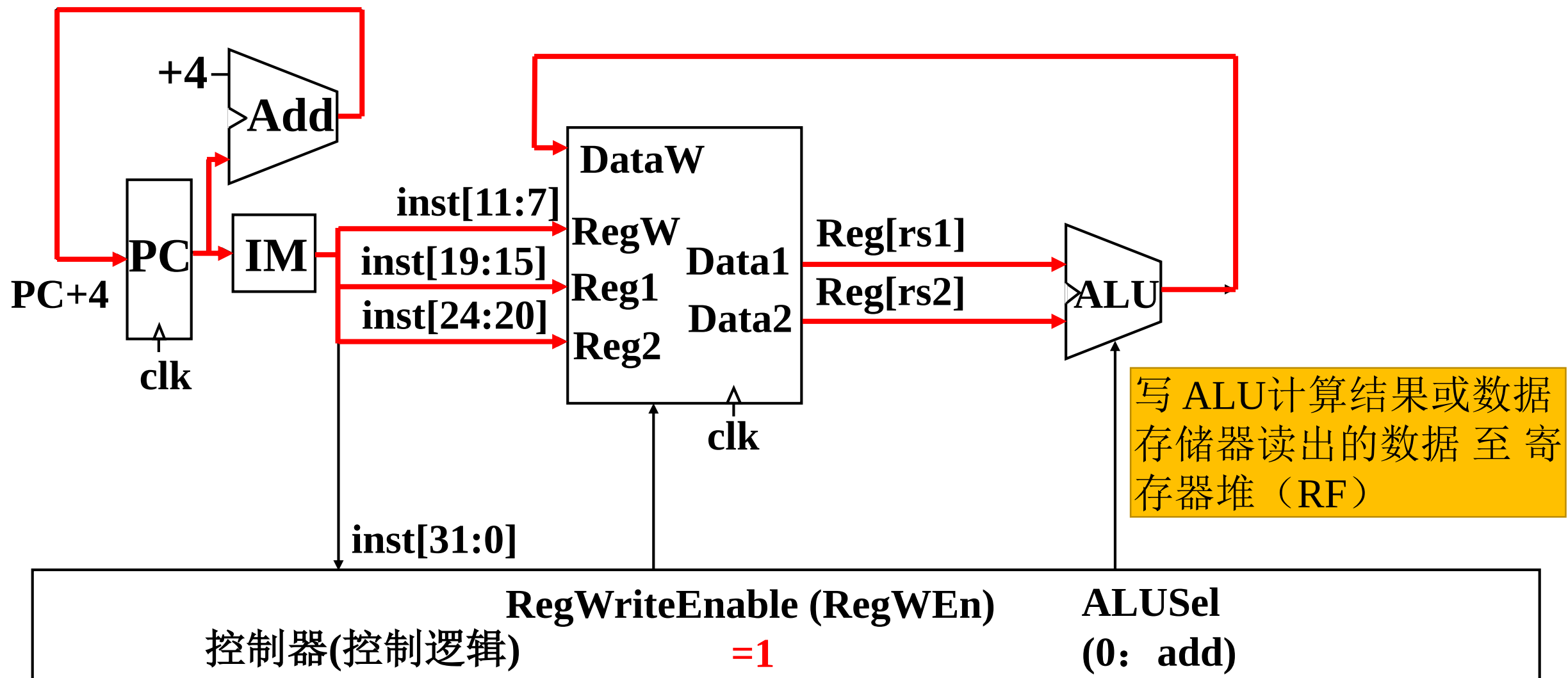


ALU控制	Funciton
0000	and
0001	or
0010	add
0110	subtract
0111	set-on-less-than
1100	nor

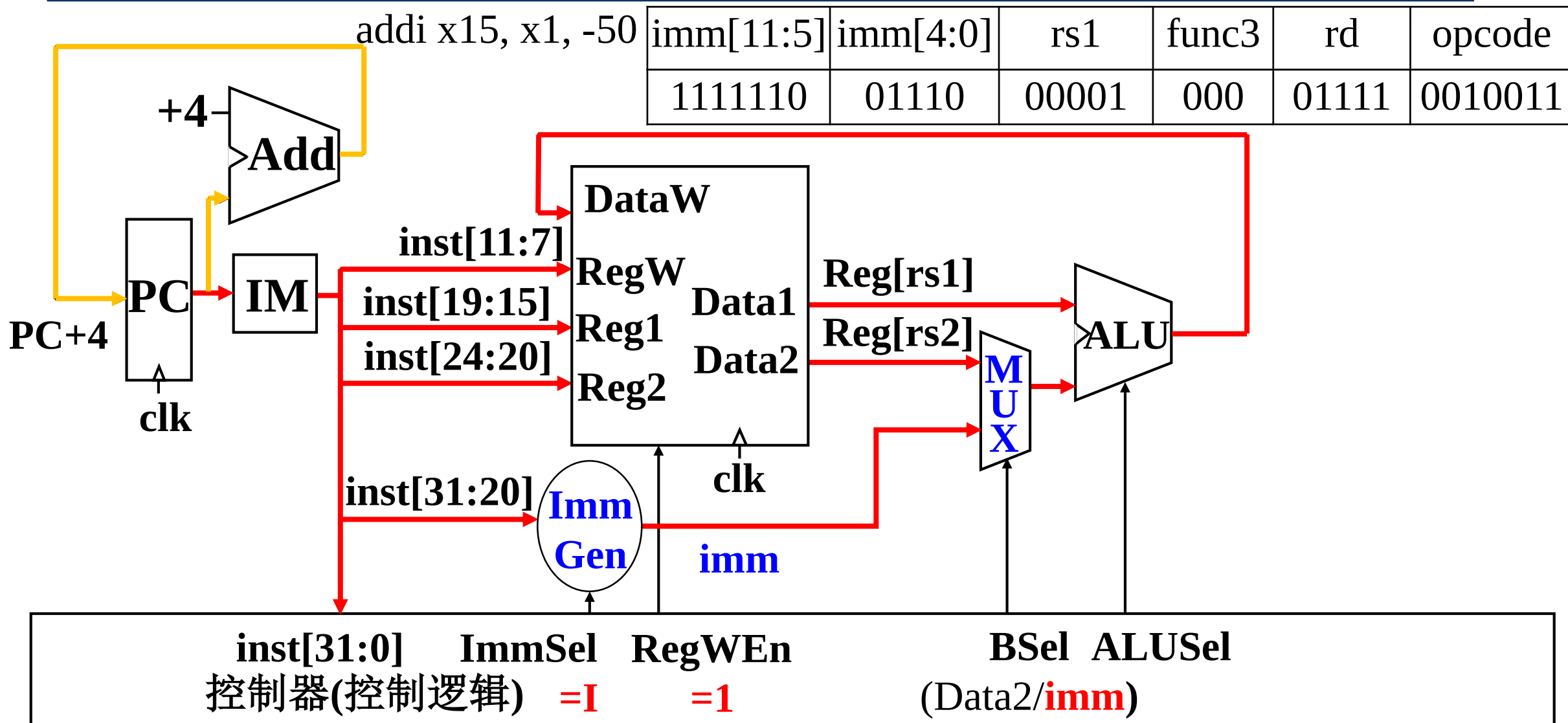
第4章 处理器设计

- 处理器设计的需求分析
- RISC-V数据通路的组件选择
- RISC-V部分指令的数据通路设计
- RISC-V控制器

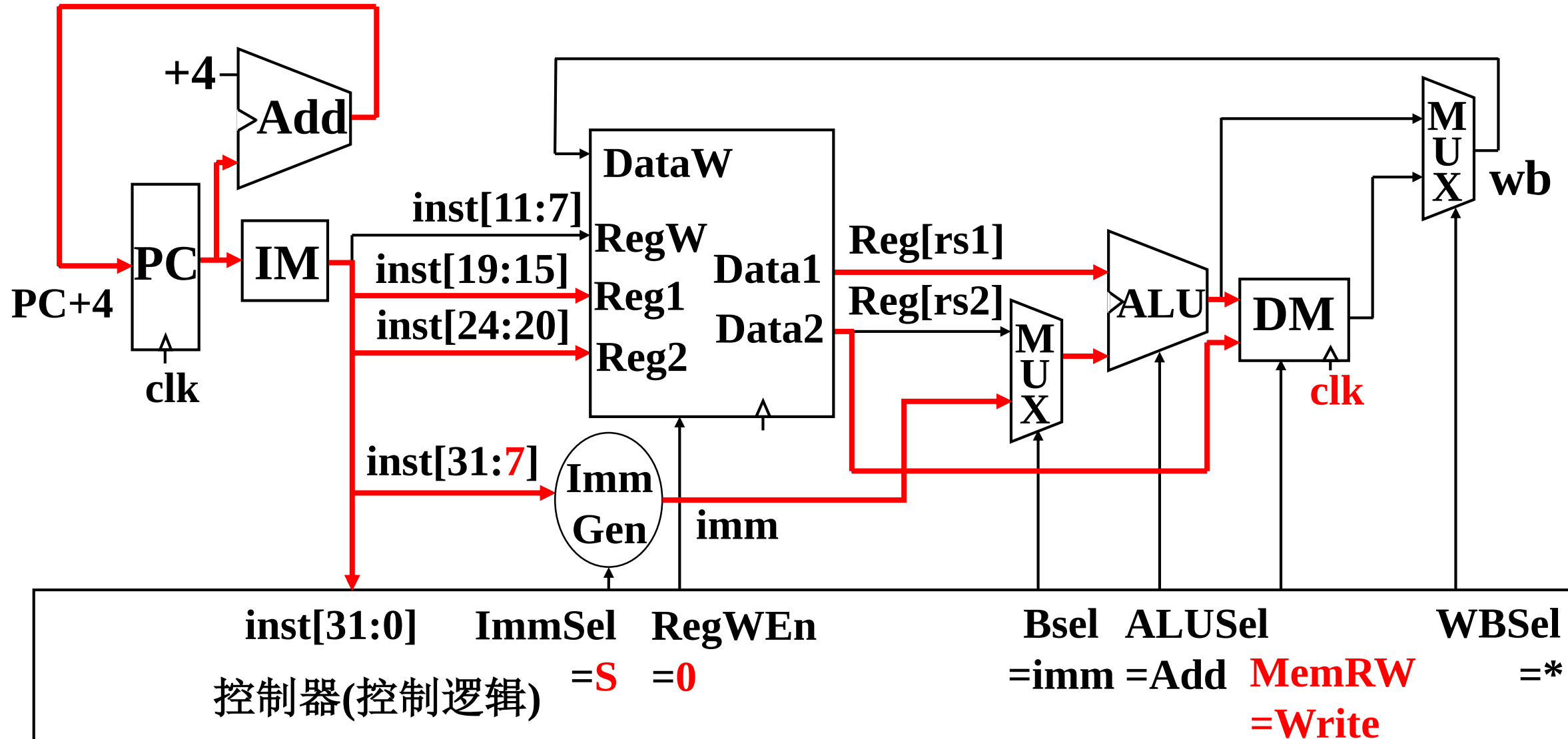
add指令的数据通路——写回寄存器



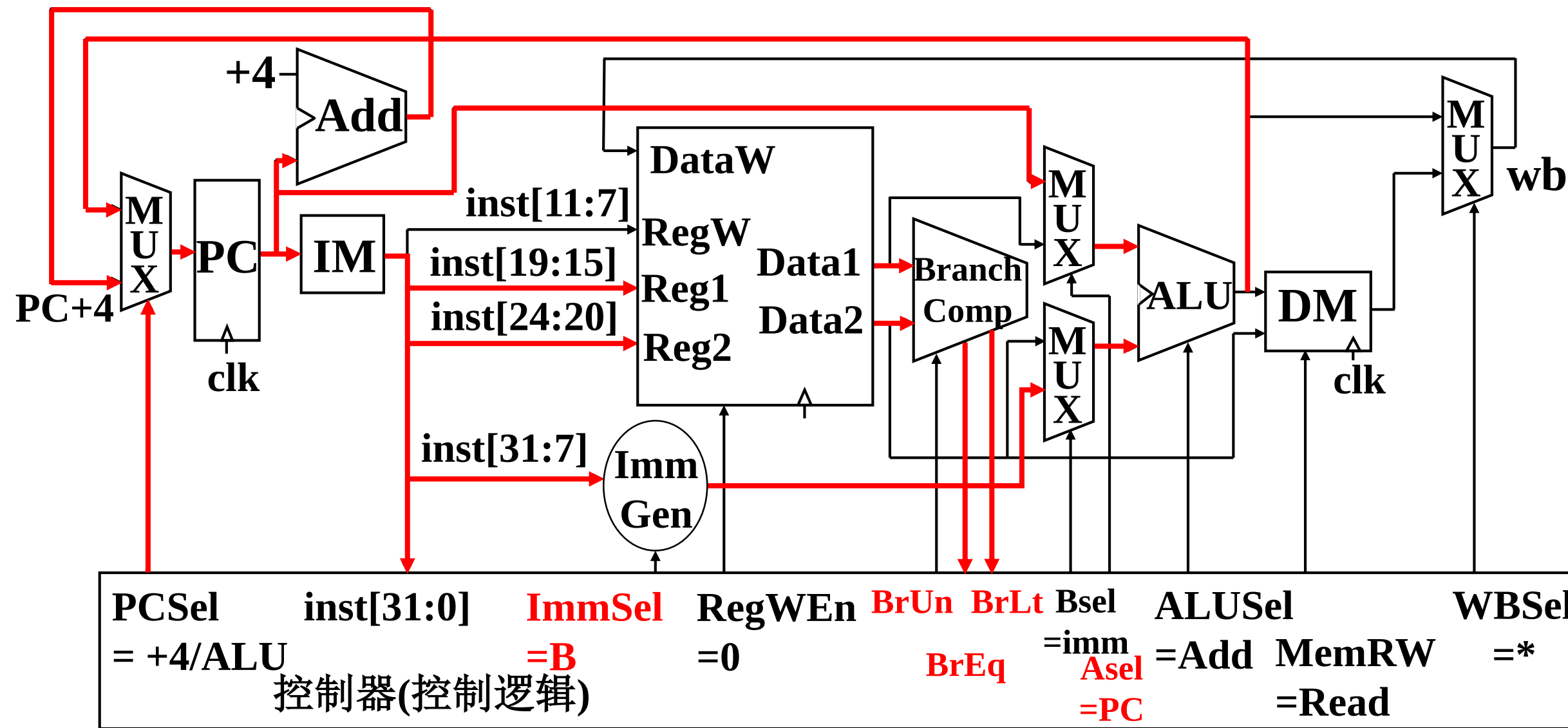
addi指令的数据通路



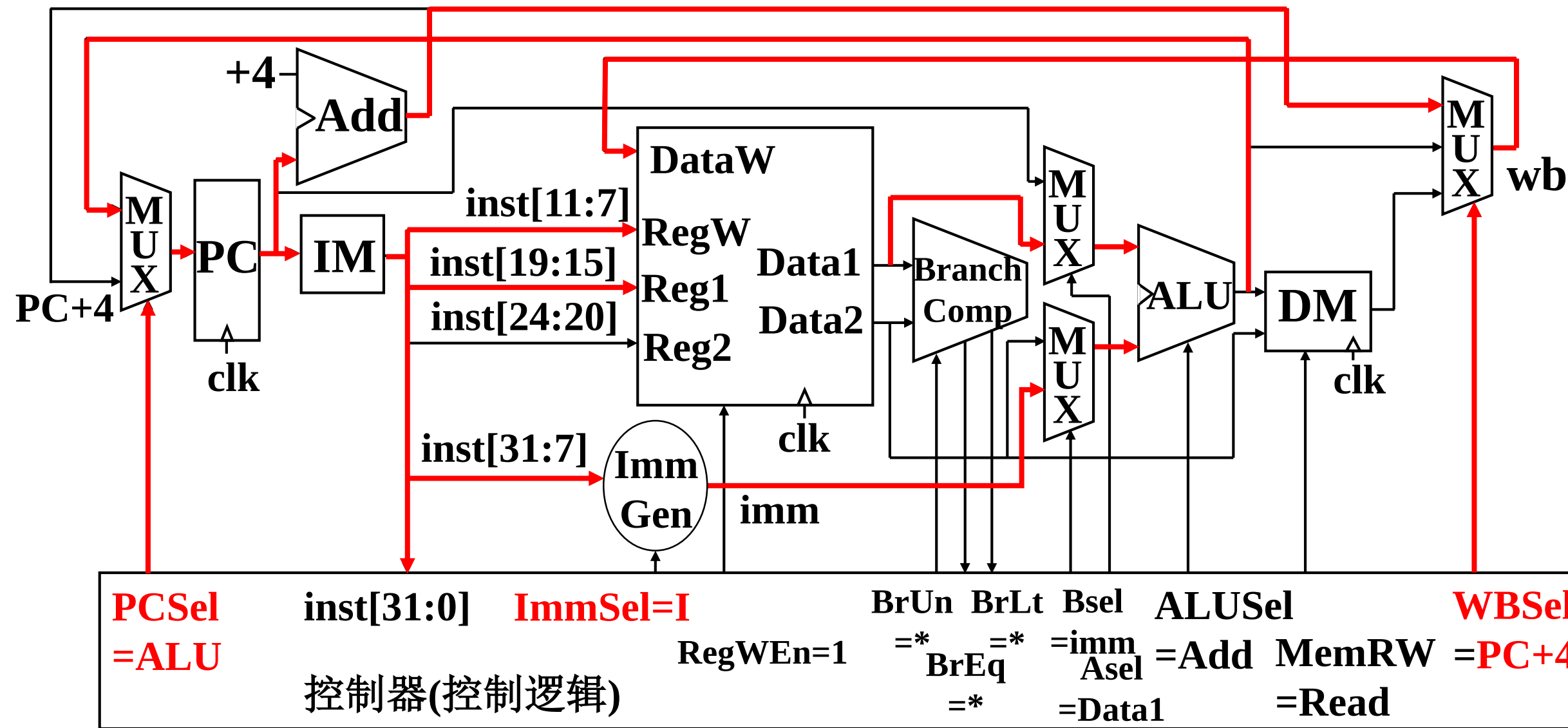
sd指令的数据通路



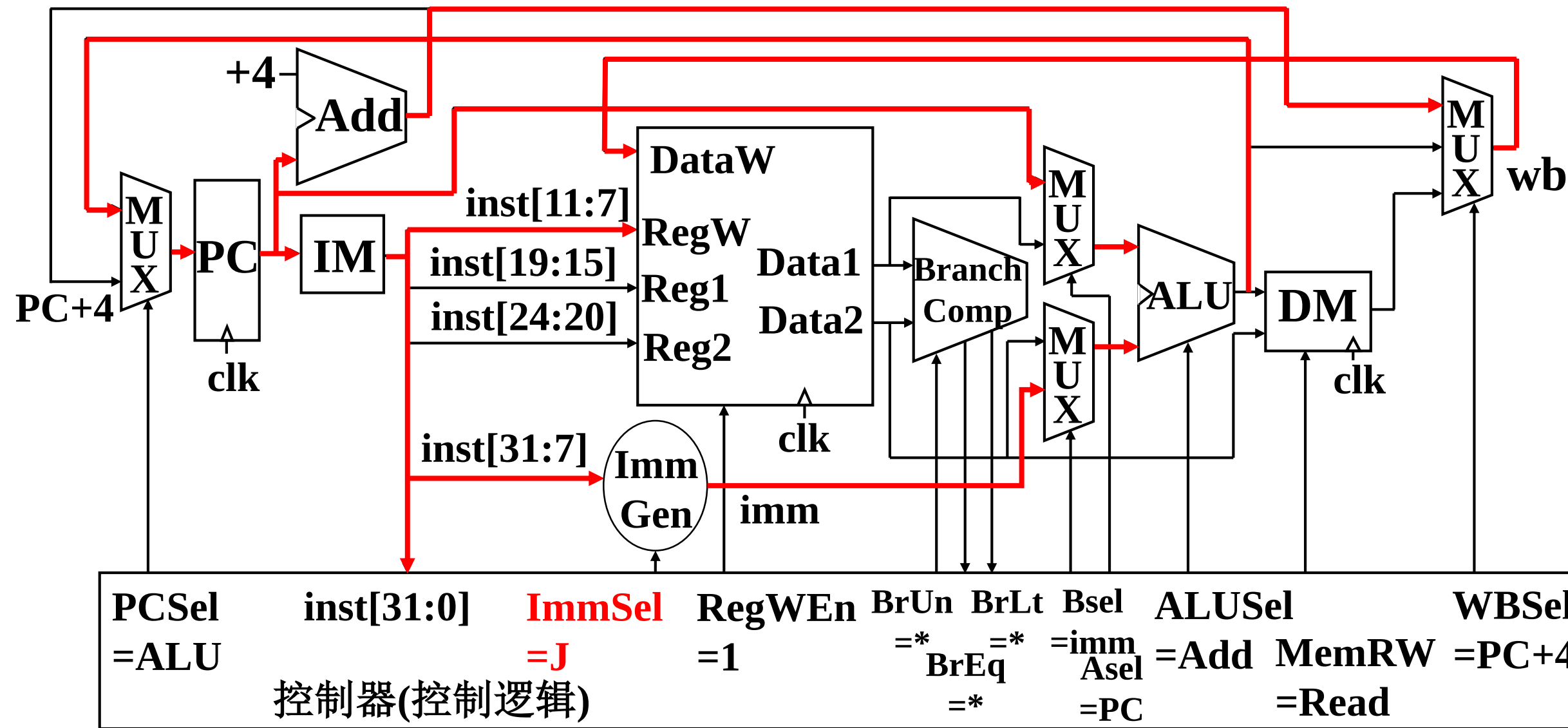
B型指令的数据通路



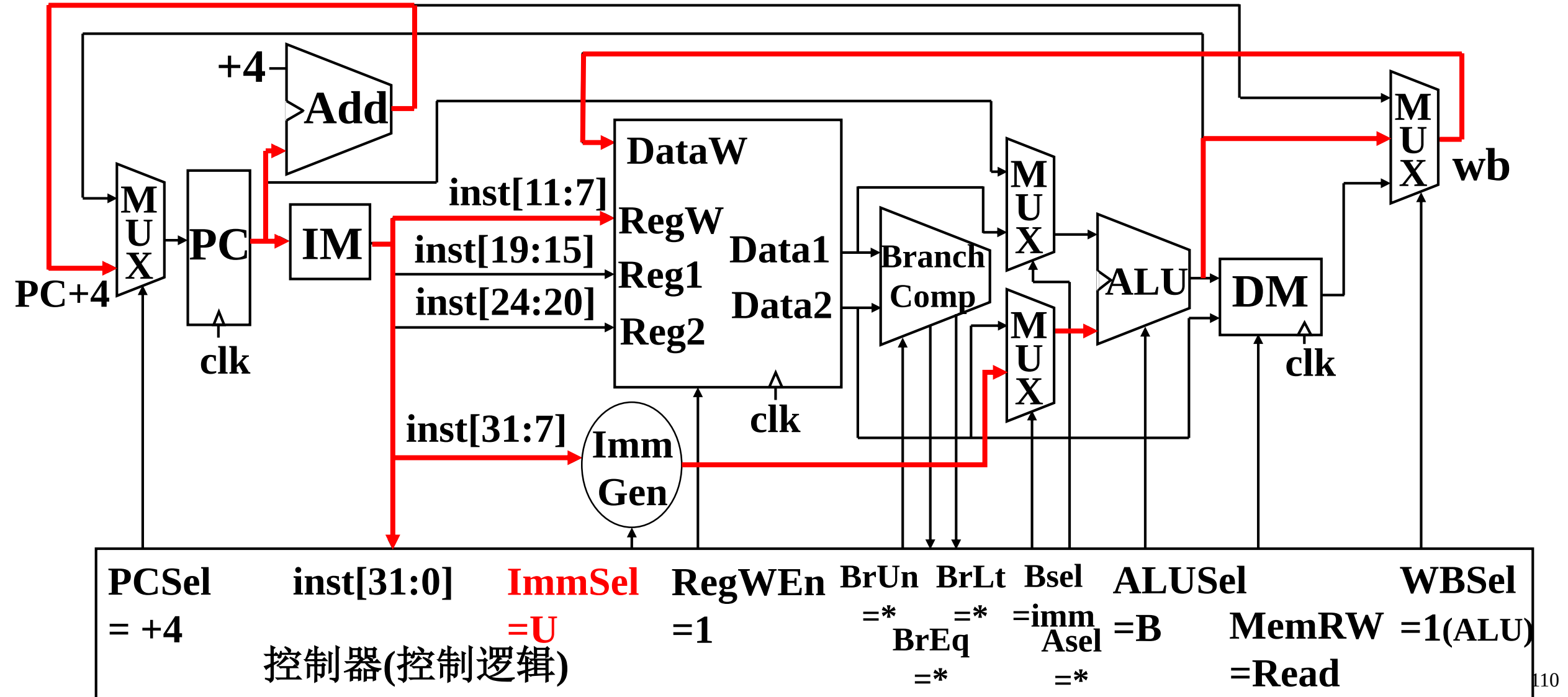
jalr指令的数据通路



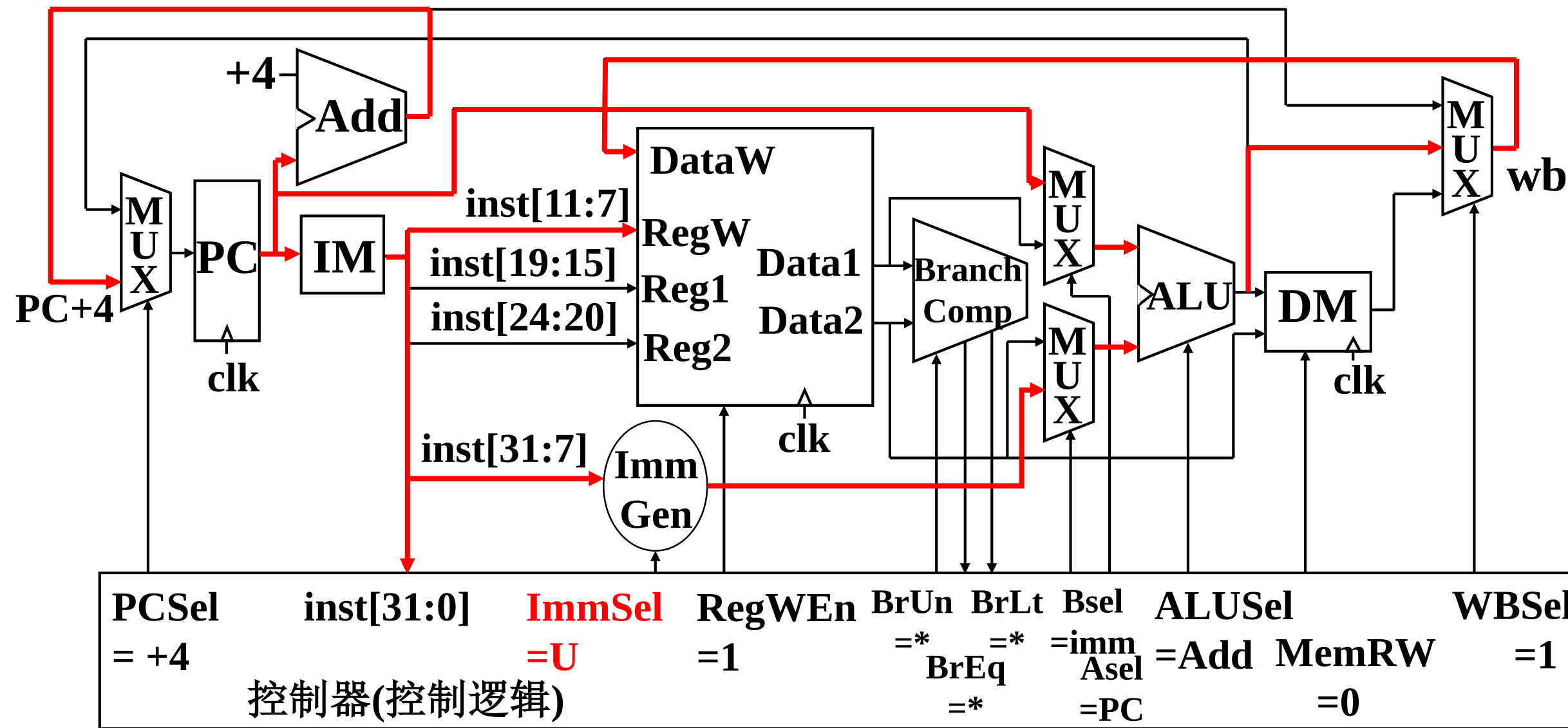
jal指令的数据通路



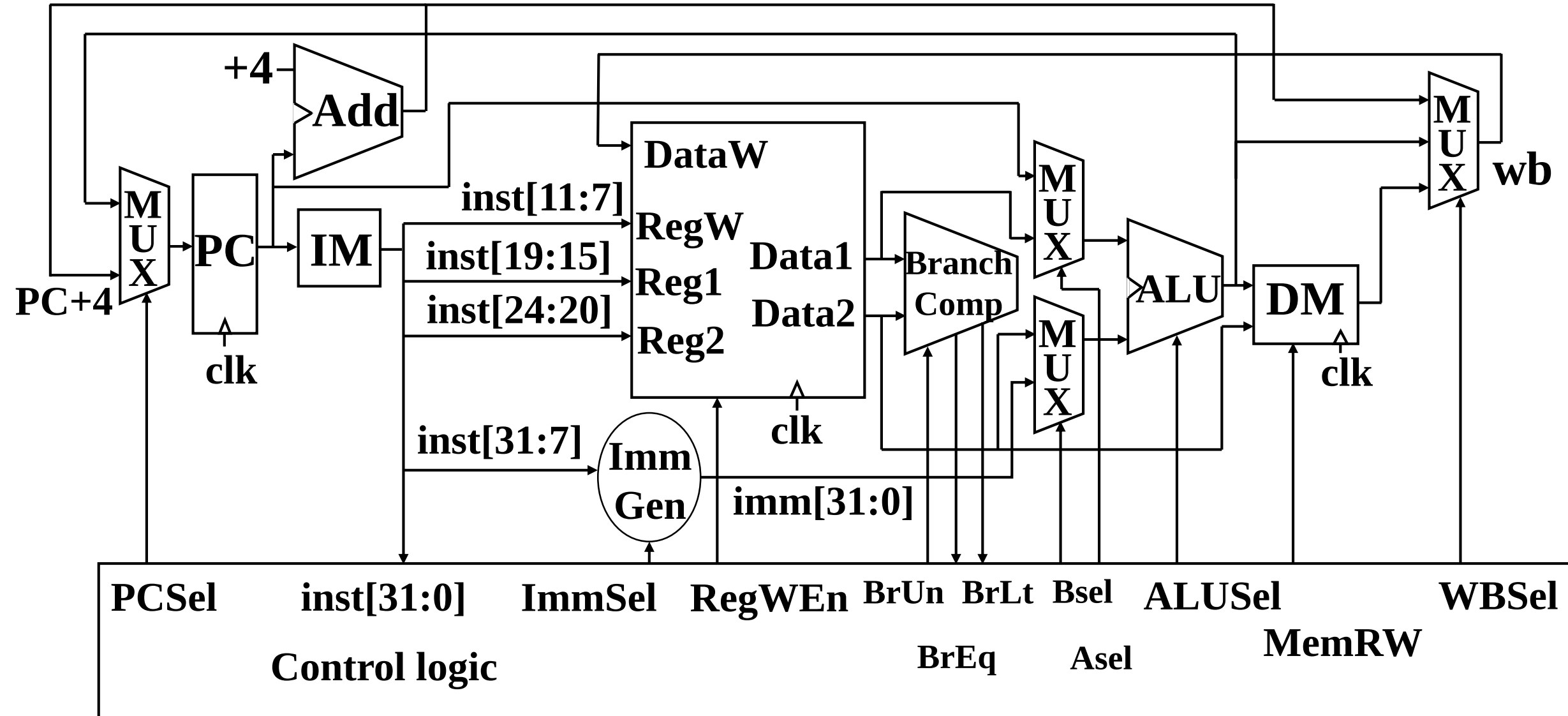
U型指令lui的数据通路



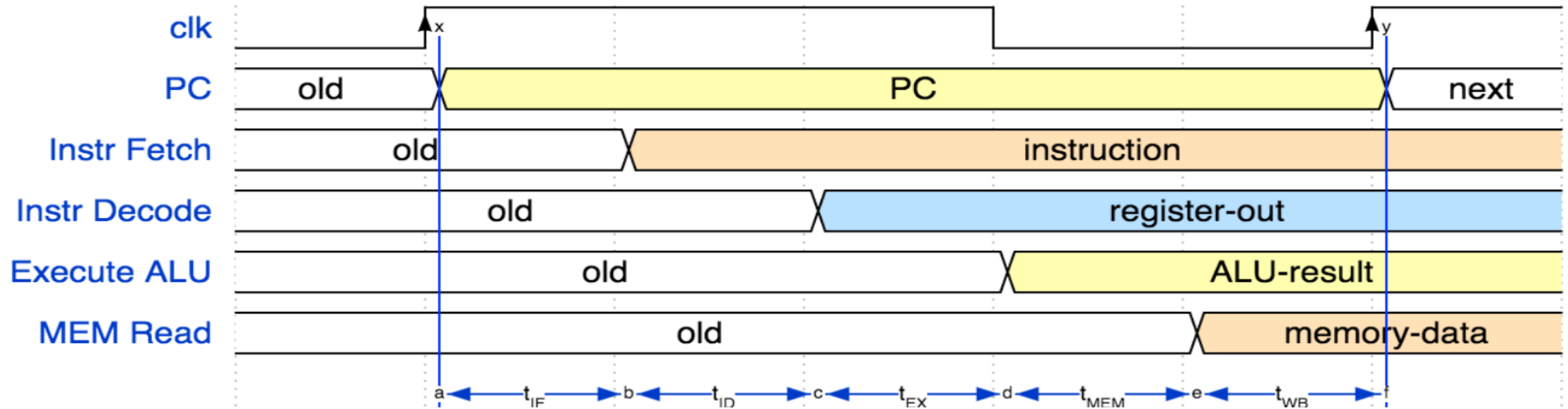
U型指令auiipc的数据通路



RISC-V 的数据通路



指令时延



IF	ID	EX	MEM	WB	Total
I-MEM	Reg Read	ALU	D-MEN	Reg W	
200ps	100ps	200ps	200ps	100ps	800ps

指令时延

Instr	IF=200ps	ID=100ps	ALU=200ps	MEM=200ps	WB=100ps	Total
add	√	√	√		√	600ps
beq	√	√	√			500ps
jal	√	√	√		√	600ps
load	√	√	√	√	√	800ps
store	√	√	√	√		700ps

- 最大时钟频率 $f_{\max, \text{ALU}} = 1/(800\text{ps}) = 1.25\text{GHz}$
- 有些段多数时间处于空闲状态
- 单个段最大时钟频率 $f_{\max, \text{ALU}} = 1/(200\text{ps}) = 5\text{GHz}$

数据通路的五个阶段

- 取指: Instruction Fetch (IF)
- 译码: Instruction Decode (ID)
- 执行: EXecute (EX) - ALU (Arithmetic-Logic Unit)
- 访存: MEMory access (MEM)
- 写回: Write Back to register (WB)

第四章 处理器设计

- 处理器设计的需求分析
- RISC-V数据通路的组件选择
- RISC-V部分指令的数据通路设计
- **RISC-V控制器**

RISC-V 编码

- 从前述的RISC-V 指令集编码可以看出，实际上用来区分某条指令种类的编码只有inst[30]、inst[14:12]、inst[6:2]

	int[31:25]	int[24:20]	int[19:15]	int[14:12]	int[11:7]	inst[6:0]
R型	func7	rs2	rs1	func3	rd	opcode
I型	imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12,10:5]	rs2	rs1	func3	imm[4:1,11]	opcode
J型	imm[20,10:5]	imm[4:1,11]	imm[19:15]	imm[14:12]	rd	opcode
U型	imm[31:25]	imm[24:20]	imm[19:15]	imm[14:12]	rd	opcode

RISC-V 编码

- inst[30]用于区分移位指令是“逻辑右移”和“算术右移”，R型指令的ADD和SUB。

0000000	rs2	rs1	000	<u>rd</u>	0110011	add
0100000	rs2	rs1	000	<u>rd</u>	0110011	sub
0000000	rs2	rs1	001	<u>rd</u>	0110011	<u>sll</u>
0000000	rs2	rs1	010	<u>rd</u>	0110011	<u>slt</u>
0000000	rs2	rs1	011	<u>rd</u>	0110011	<u>sltu</u>
0000000	rs2	rs1	100	<u>rd</u>	0110011	<u>xor</u>
0000000	rs2	rs1	101	<u>rd</u>	0110011	<u>srl</u>
0100000	rs2	rs1	101	<u>rd</u>	0110011	<u>sra</u>
000000	<u>shamt</u>	rs1	101	<u>rd</u>	0010011	<u>srli</u>
010000	<u>shamt</u>	rs1	101	<u>rd</u>	0010011	<u>srai</u>

RISC-V 编码

- `inst[14:12]`是部分指令32位机器码的`funct3`字段。它主要用于区分一个大类指令下不同的小功能。例如，B型指令下的各种不同的比较方式。`Inst[14:12]`

<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	000	<code>imm[4:1 11]</code>	1100011	BEQ
<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	001	<code>imm[4:1 11]</code>	1100011	BNE
<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	100	<code>imm[4:1 11]</code>	1100011	BLT
<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	101	<code>imm[4:1 11]</code>	1100011	BGE
<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	110	<code>imm[4:1 11]</code>	1100011	BLTU
<code>imm[12 10:5]</code>	<code>rs2</code>	<code>rs1</code>	111	<code>imm[4:1 11]</code>	1100011	BGEU

- $\text{BrUn} = \text{Inst}[14] \cdot \text{Inst}[13] \cdot \text{Branch}$

RISC-V 编码

- inst[6:2]是指令的操作码(Opcode)字段，主要用于区分不同类别的指令。

						Inst[6:2]
R 型	funct7	rs2	rs1	funct3	rd	0110011
I 型(运算类)	imm[11:0]		rs1	funct3	rd	0010011
I 型(lw)	imm[11:0]		rs1	funct3	rd	0000011
I 型(jalr)	imm[11:0]		rs1	funct3	rd	1100111
S 型	Imm[11:5]	rs2	rs1	funct3	imm[4:0]	0100011
B 型	Imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	1100011
J 型(jal)	Imm[20,10:1,11,19:12]				rd	1101111
U 型	Imm[31:12]				rd	0110111

ROM和真值表

- ROM “存储” 了一个n输入、b输出的组合逻辑功能的真值表。
- 一个3输入、4输出的组合功能的真值表，可以被存储在一个 $2^3 * 4$ ($8 * 4$) 的只读存储器中。忽略延迟，ROM的数据输出总是等于真值表中由地址输入所选择的那行输出位。

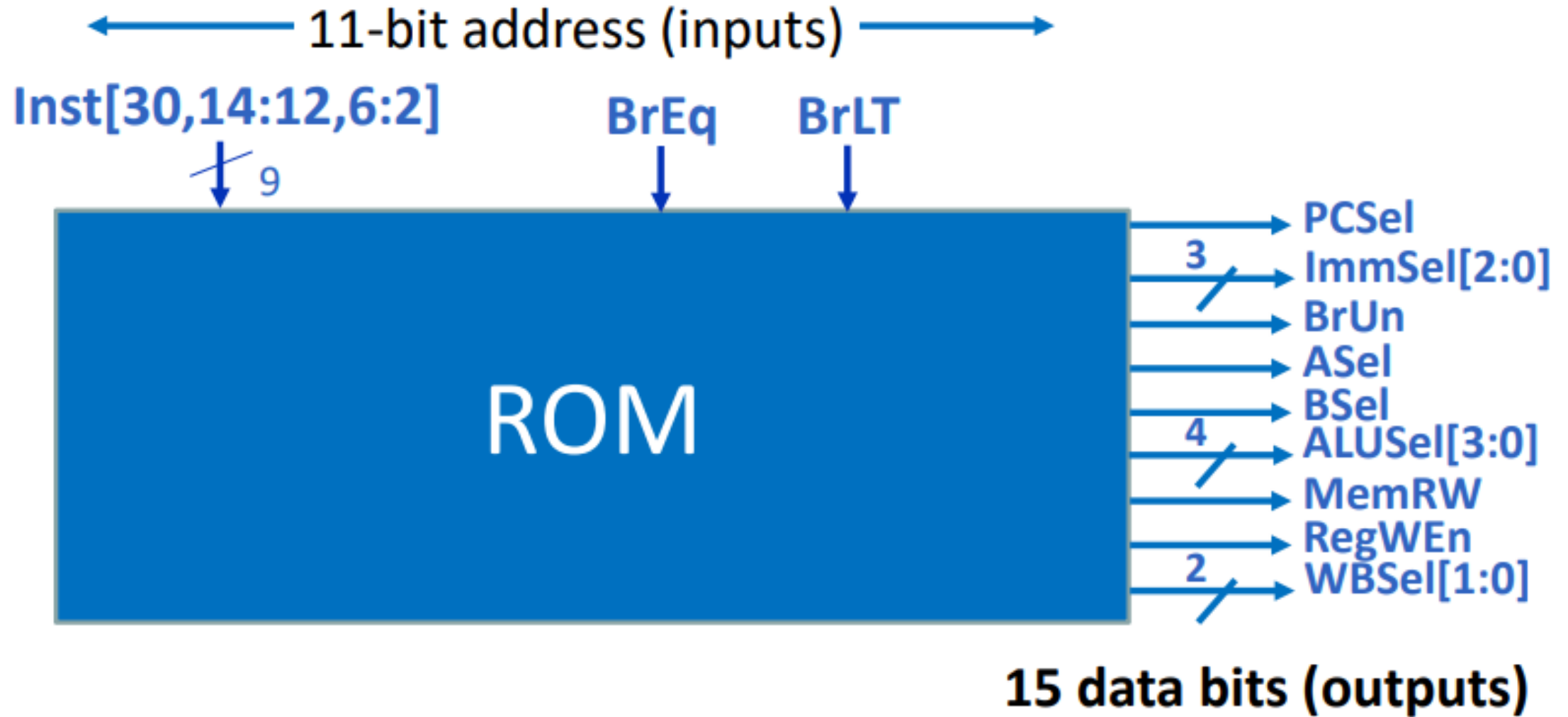
输入			输出			
A2	A1	A0	D3	D2	D1	D0
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

一个3输入4输出组合逻辑函数的真值表
(具有输出极性控制的2-4译码器)

控制信号真值表

Inst[31:0]	BrEq	BrLt	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWen	WBSel
add	*	*	+4	*	*	Reg	Reg	Add	Read	1	ALU
sub	*	*	+4	*	*	Reg	Reg	Add	Read	1	ALU
(R-R Op)	*	*	+4	*	*	Reg	Reg	(Op)	Read	1	ALU
addi	*	*	+4	I	*	Reg	Imm	Add	Read	1	ALU
lw	*	*	+4	I	*	Reg	Imm	Add	Read	1	Mem
sw	*	*	+4	S	*	Reg	Imm	Add	Write	0	*
beq	0	*	+4	B	*	PC	Imm	Add	Read	0	*
beq	1	*	ALU	B	*	PC	Imm	Add	Read	0	*
bne	0	*	ALU	B	*	PC	Imm	Add	Read	0	*
bne	1	*	+4	B	*	PC	Imm	Add	Read	0	*
blt	*	1	ALU	B	0	PC	Imm	Add	Read	0	*
bltu	*	1	ALU	B	1	PC	Imm	Add	Read	0	*
jalr	*	*	ALU	I	*	Reg	Imm	Add	Read	1	PC+4
jal	*	*	ALU	J	*	PC	Imm	Add	Read	1	PC+4
auipc	*	*	+4	U	*	PC	Imm	Add	Read	1	ALU

ROM 控制



第五章 流水线处理器

- 流水线概述
- 流水线数据通路和控制
- 数据冒险：前递与停顿
- 控制冒险
- 例外

第五章 流水线处理器

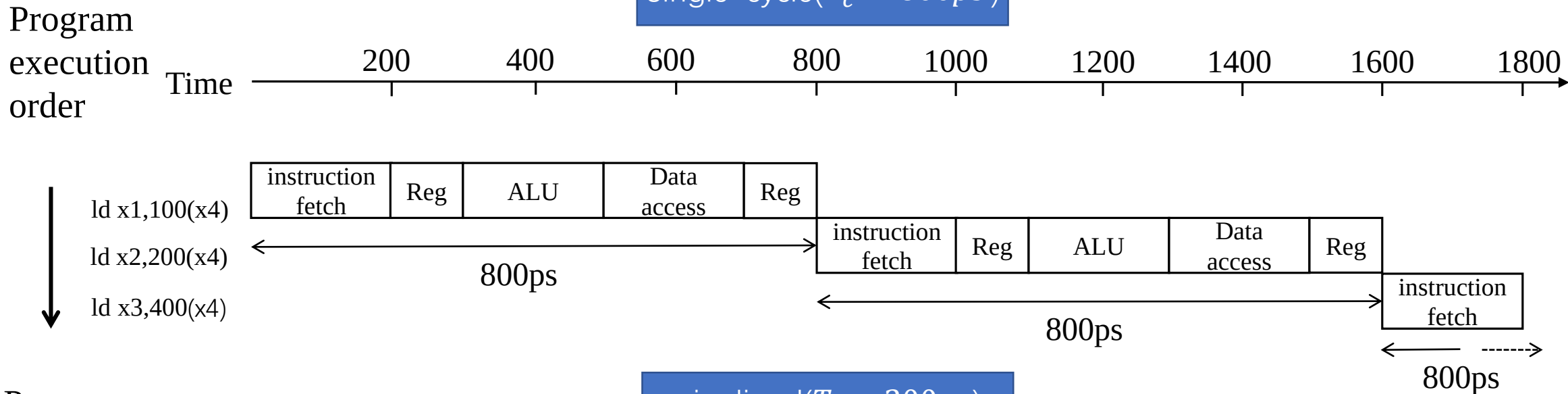
• 流水线处理器重点

- 流水线的思想（会计算加速比和分析流水线性能）
- 流水线数据通路和控制信号。
- 结构冒险的概念。
- 两种数据冒险及解决方法
- 控制冒险和解决方法
- 例外的解决方法

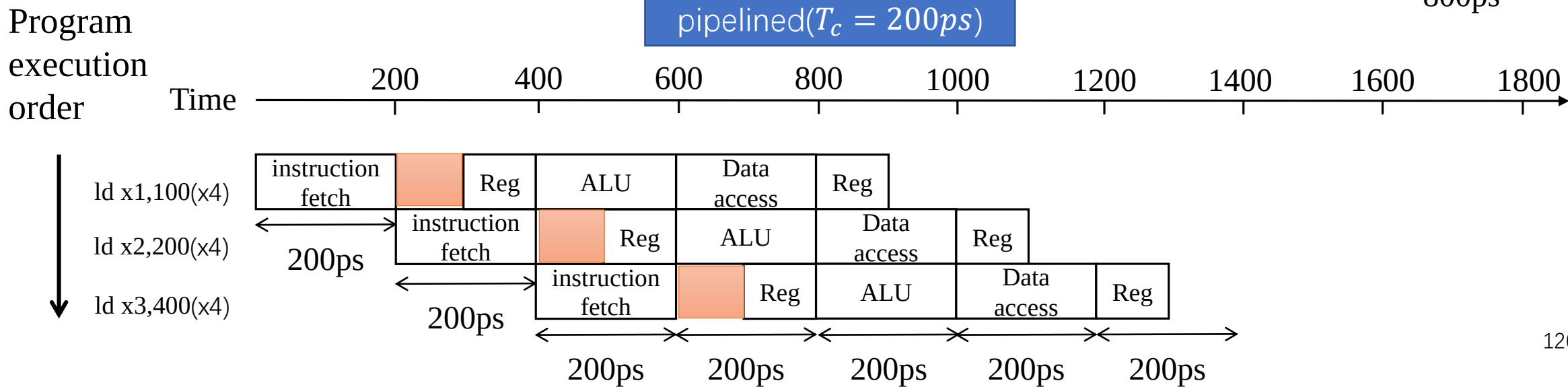
要熟悉为了解决这些问题对于流水线数据通路做的修改：添加了哪些硬件组件？如何将他们连接起来的？

流水线性能

Single-cycle($T_c = 800ps$)



pipelined($T_c = 200ps$)



流水线加速比

- 如果流水线各阶段操作平衡
 - 例如：所有阶段需要相同的时间
 - 则：指令执行时间（流水线） $\approx$ 指令执行时间（非流水线） / 流水线级数
- 若各阶段不完全平衡，加速比会变小
- 通过提高指令吞吐率来提高性能
 - 单个指令的执行时间没有减少

流水线总结

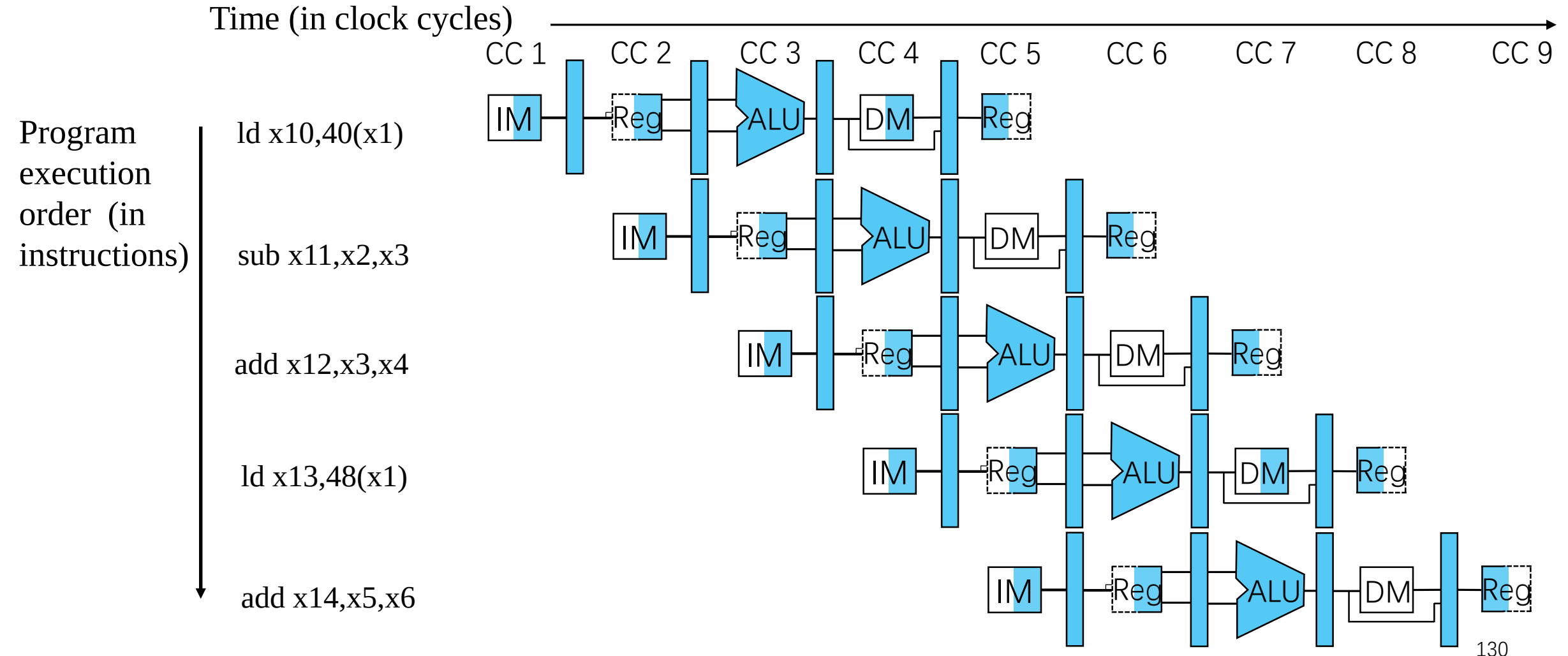
- 流水线通过提高吞吐率来提升性能
 - 并行执行多条指令
 - 每条指令拥有相同的延迟
 - 延迟：执行单条指令的时间
- 会受到冒险的影响
 - 结构、数据、控制
- 指令集的设计影响到流水线实现的复杂度

第五章 流水线处理器

- 流水线概述
- **流水线数据通路和控制**
- 数据冒险：前递与停顿
- 控制冒险
- 例外

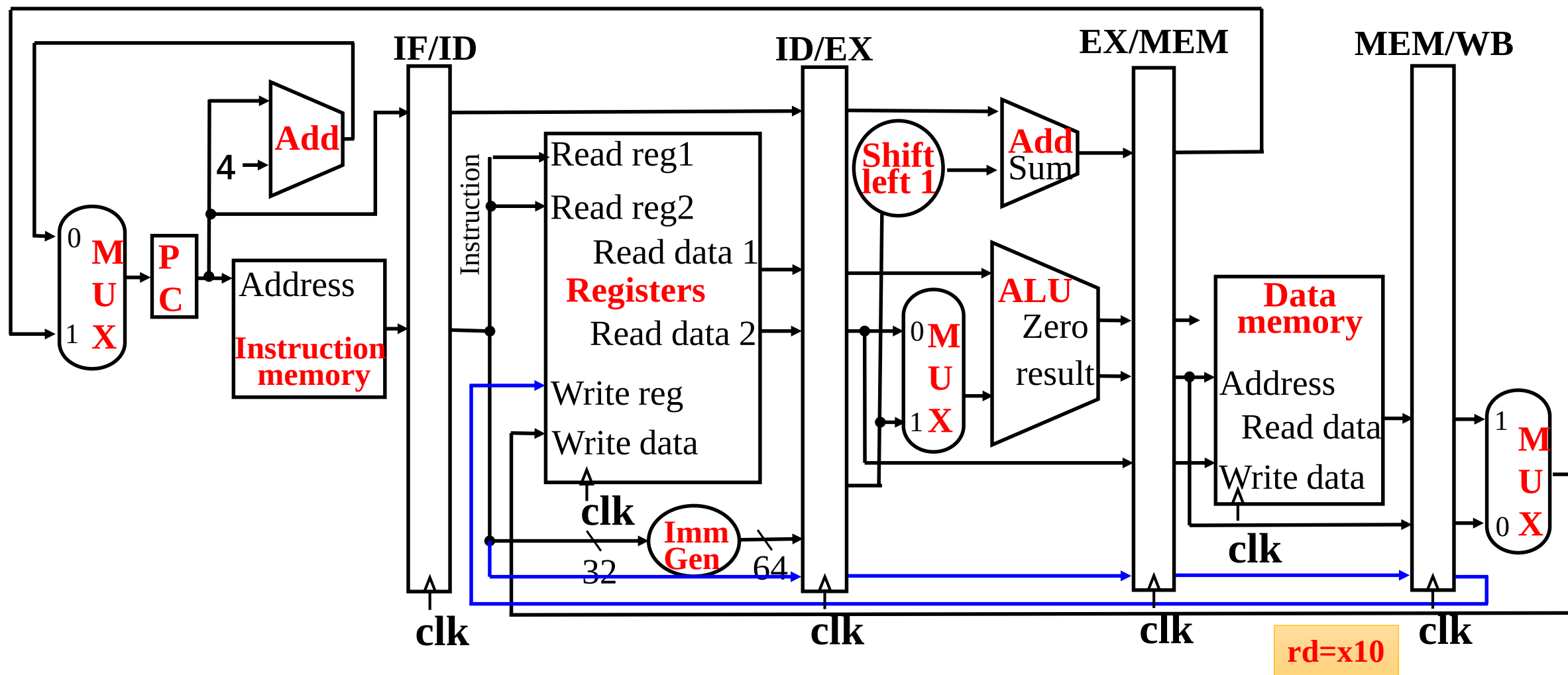
多时钟周期流水线图

- 显示了每个流水线阶段中使用的物理资源



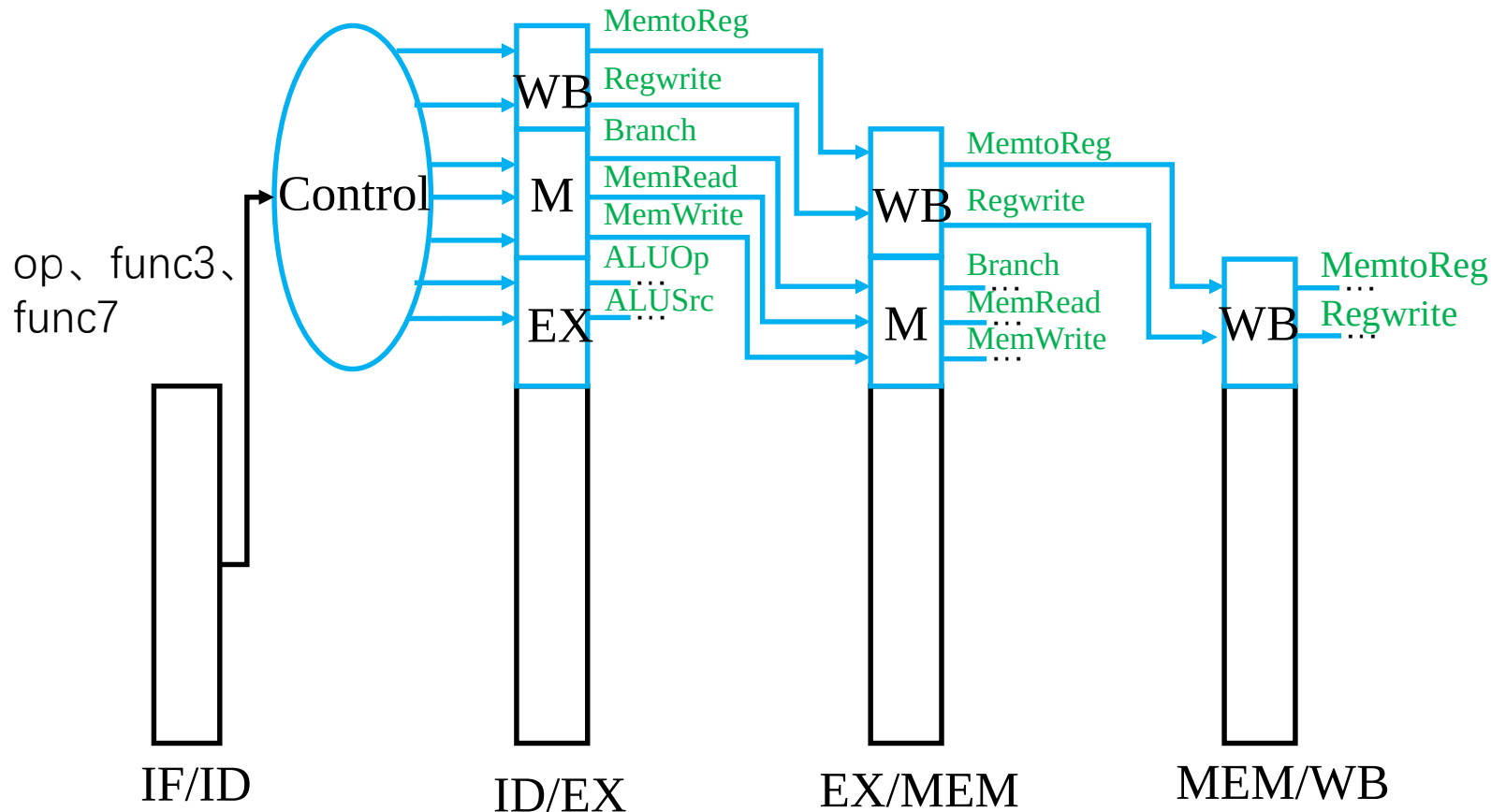
更正之后的数据通路

ld x10,40(x1)



流水线控制

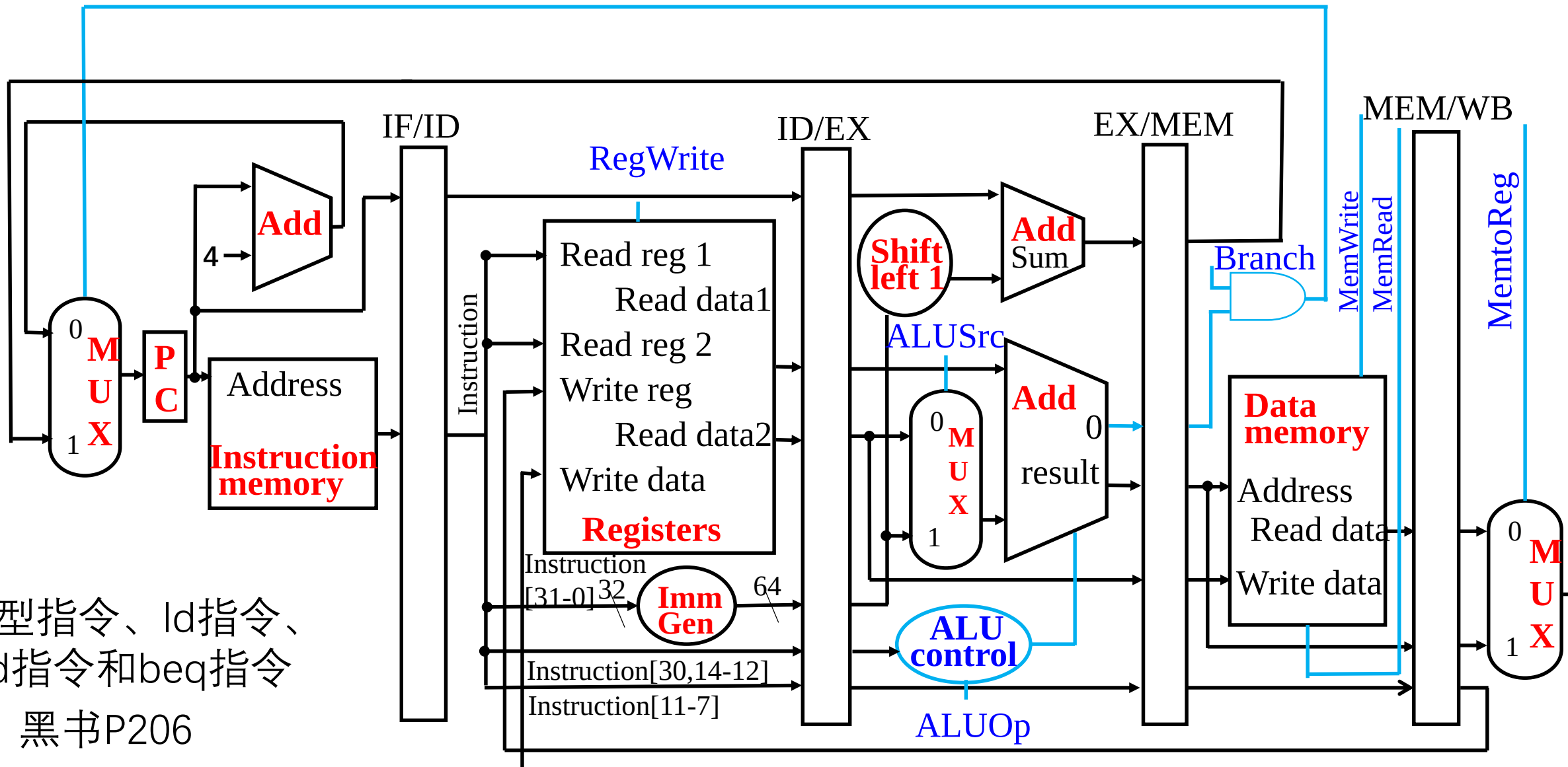
- 控制信号从指令中产生，与单周期实现相似
- 根据流水线阶段将控制线也进行分组
 - IF和ID阶段没有需要特别控制的内容



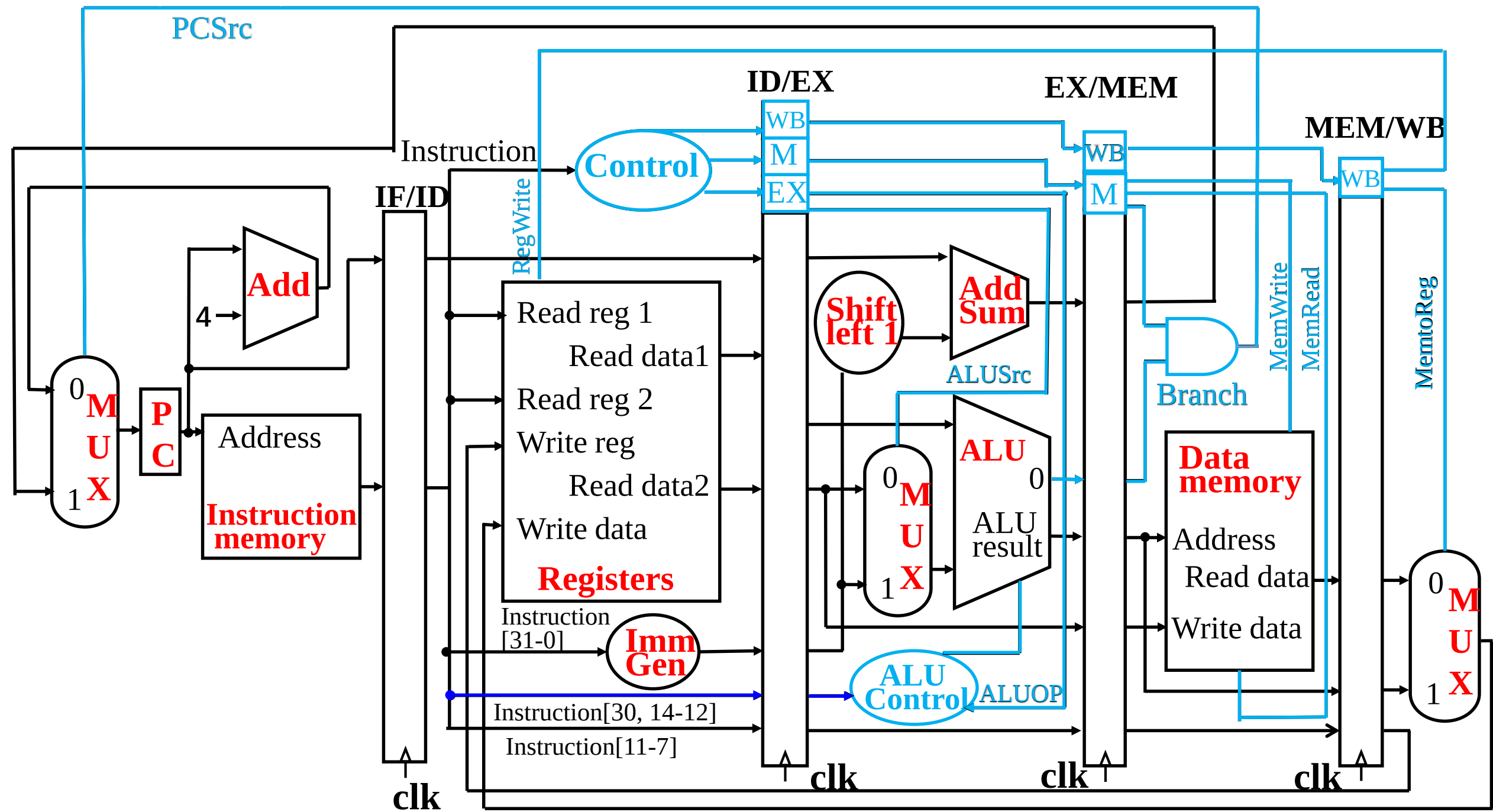
- MemtoReg选择ALU或数据存储器值写入到寄存器堆
- ALUSrc: ALU前的选择器控制信号，用于选择Reg[rs2]或立即数值
- Branch: 分支指令的控制信号

简单的流水线控制

PCSrc



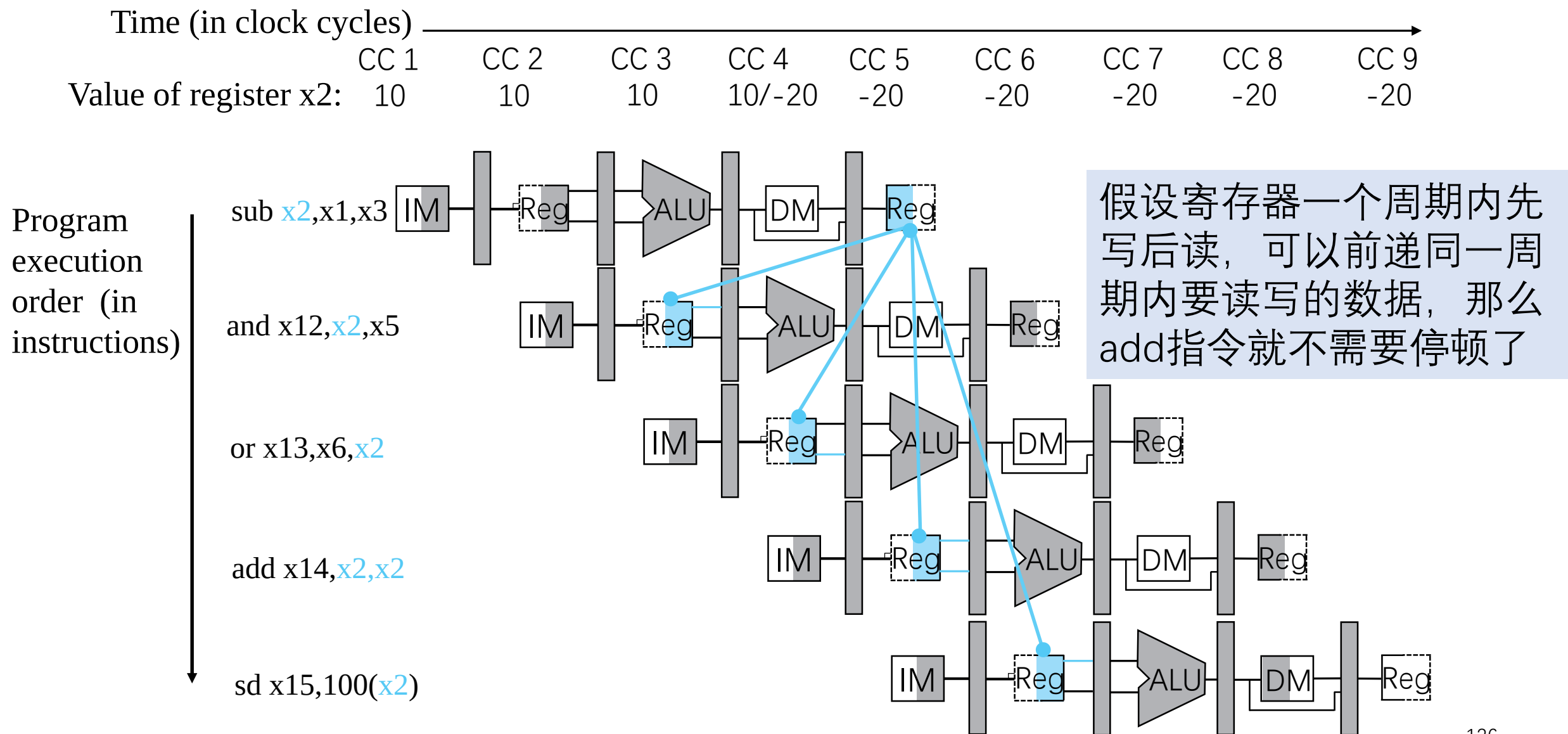
R型指令、ld指令、
sd指令和beq指令
黑书P206



第五章 流水线处理器

- 流水线概述
- 流水线数据通路和控制
- 数据冒险：前递与停顿
- 控制冒险
- 例外

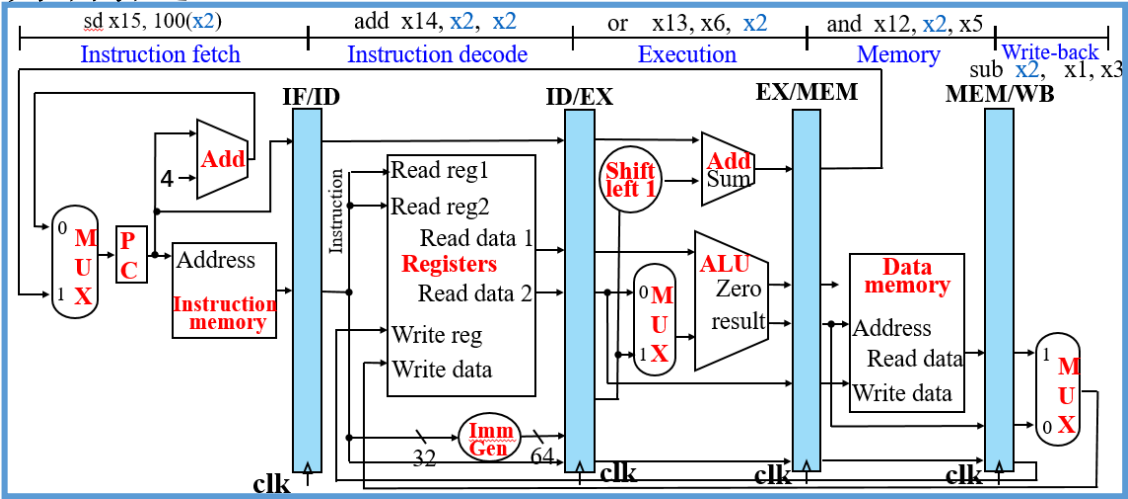
依赖关系与前递



检测前递发生

- 命名流水线寄存器字段
 - 例：ID/EX.RegisterRs1 代表 ID/EX流水线寄存器中，存放寄存器字段的编号Rs1
- EX阶段中，ALU操作数寄存器字段名称分别是：
 - ID/EX.RegisterRs1, ID/EX.RegisterRs2

- 两组数据冒险：EX冒险和MEM冒险
 - 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
 - 1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
 - 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
 - 2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2



sub x2,x1,x3

and x12,x2,x5

sub x2,x1,x3

and x12,x2,x5

or x13,x6,x2

取指令	译码	执行	访存	写回	
	取指令	译码	执行	访存	写回

取指令	译码	执行	访存	写回	
	取指令	译码	执行	访存	写回
		取指令	译码	执行	访存
			取指令	译码	写回

检测前递发生

- 并不是所有指令都会写回寄存器，在前递的时候还应：

- 检查RegWrite信号是否有效

- EX冒险：EX/MEM.RegWrite $\neq 0$
- MEM冒险：MEM/WB.RegWrite $\neq 0$

```
sd    x1, 8(x2)
add   x3, x8, x2
```

- 冒险的检测条件仍然不完全

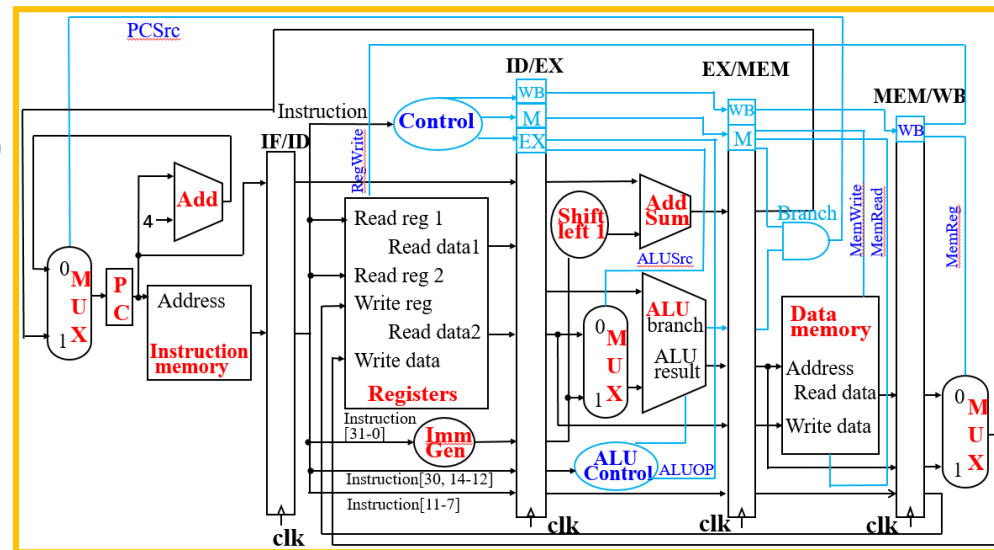
```
sub   x0, x1, x3
and   x12, x0, x5
```

- Rd=x0?

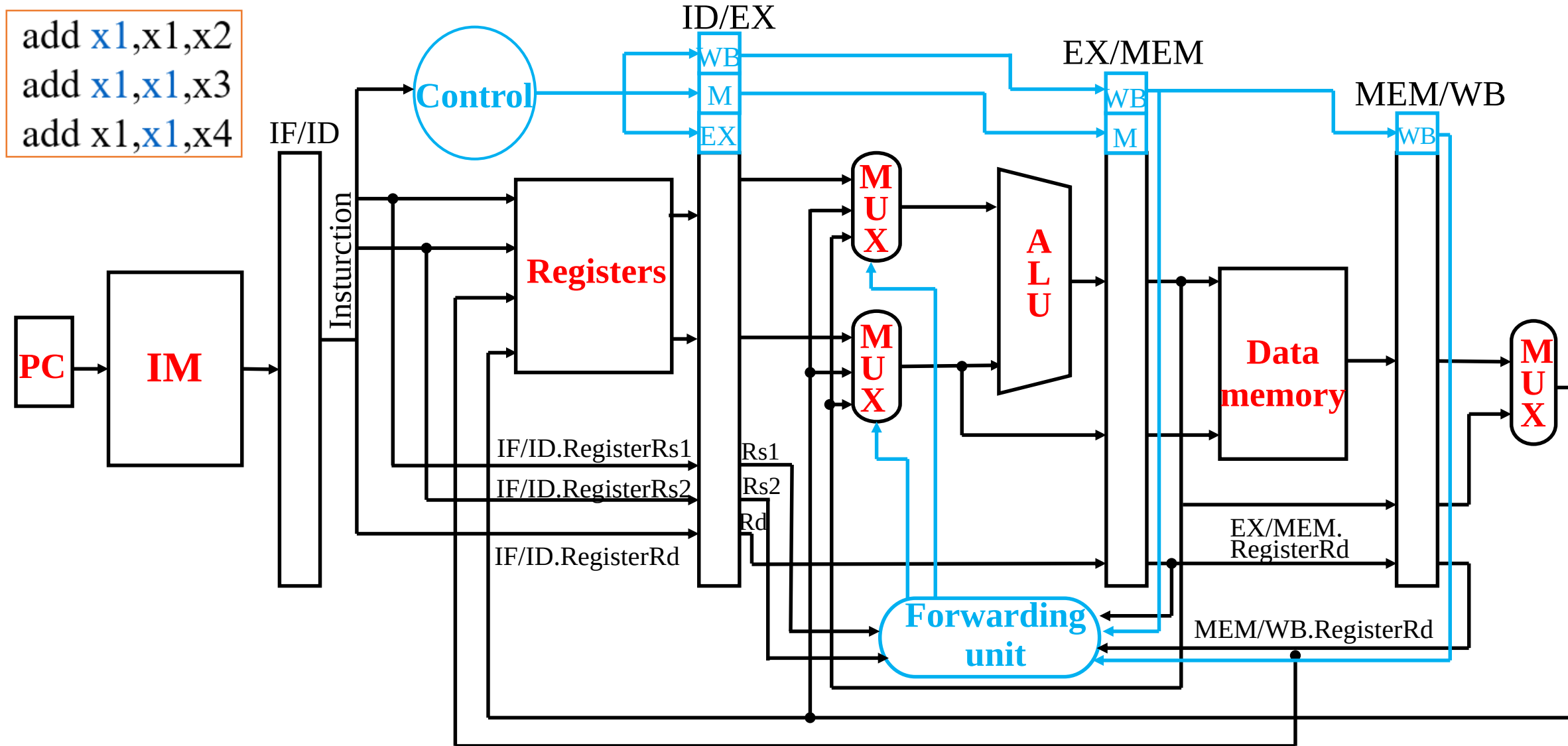
- EX冒险：EX/MEM. RegisterRd $\neq 0$
- MEM冒险：MEM/WB. RegisterRd $\neq 0$

- 二次冒险

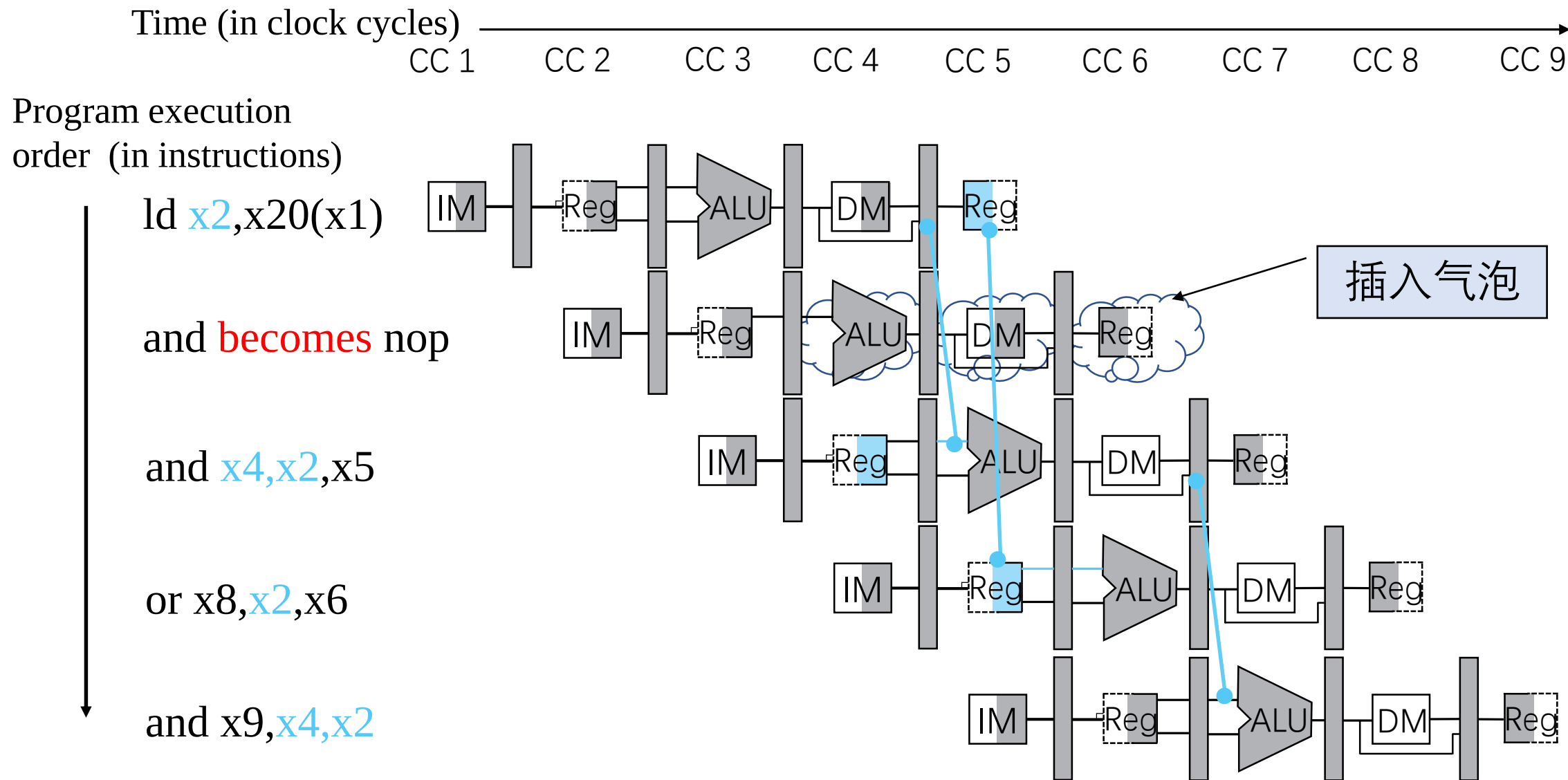
```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
```



通过前递解决冒险的数据通路

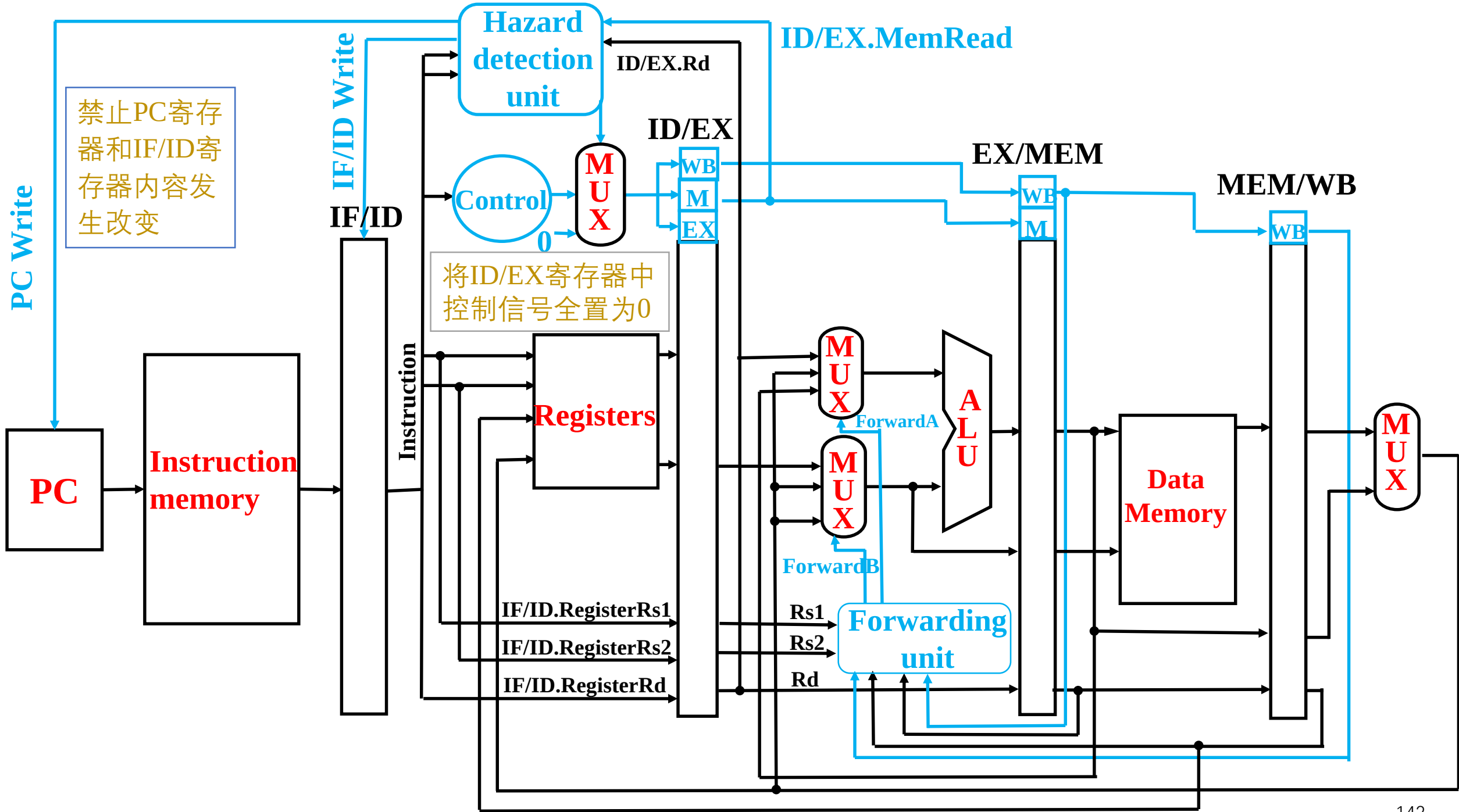


载入-使用型数据冒险



如何停顿流水线?

- 被停顿的指令正处于ID阶段
- 将ID/EX寄存器中控制信号全置为0(RegWrite, MemWrite...)
 - 被停顿的指令在EX, MEM, WB 阶段执行空指令 nop
 - 在控制值均为0时, 不会有寄存器或者存储器被写入数据
- 禁止PC寄存器和IF/ID寄存器内容发生改变
 - ID阶段的寄存器会继续使用IF/ID寄存器中相同字段读寄存器
 - 下一条指令会重新取指
 - 1个时钟周期的停顿, 能够让ld指令的MEM阶段完成
 - 就可以把取到的数据前递到EX阶段

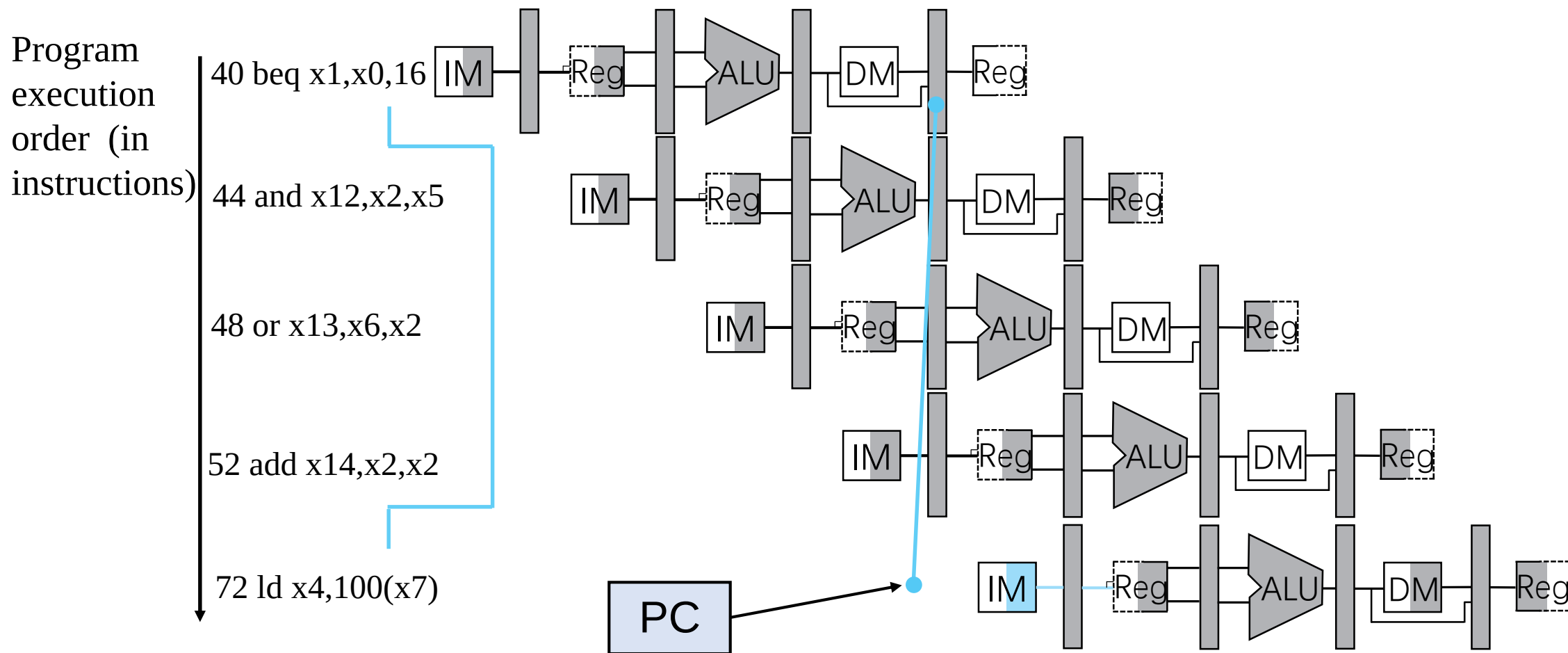


第五章 流水线处理器

- 流水线概述
- 流水线数据通路和控制
- 数据冒险：前递与停顿
- 控制冒险
- 例外

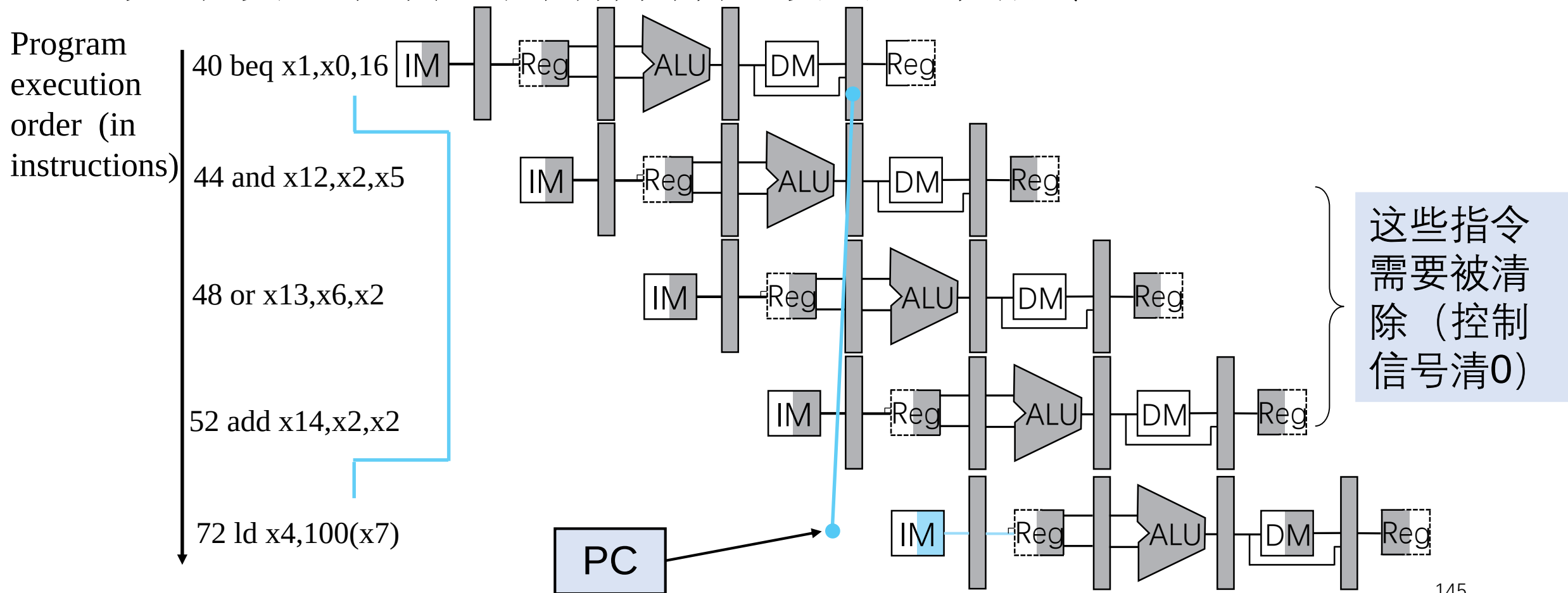
控制冒险

- 假设分支结果在MEM阶段完成确认
 - 分支指令修改PC值发生在MEM阶段



假设分支不发生

- 默认不发生分支跳转
- 如果发生跳转，则清除掉后续的两条指令



缩短分支延迟

- 移动硬件，使得分支决定提前到ID阶段
 - 需要提早计算分支目标地址（IF/ID寄存器中有PC和Immediate）
 - 需要提早判断分支条件

- 例：分支跳转发生

36: sub x10, x4, x8

40: beq x1, x3, 16 //PC-relative branch to 40+16*2=72

44: and x12, x2, x5

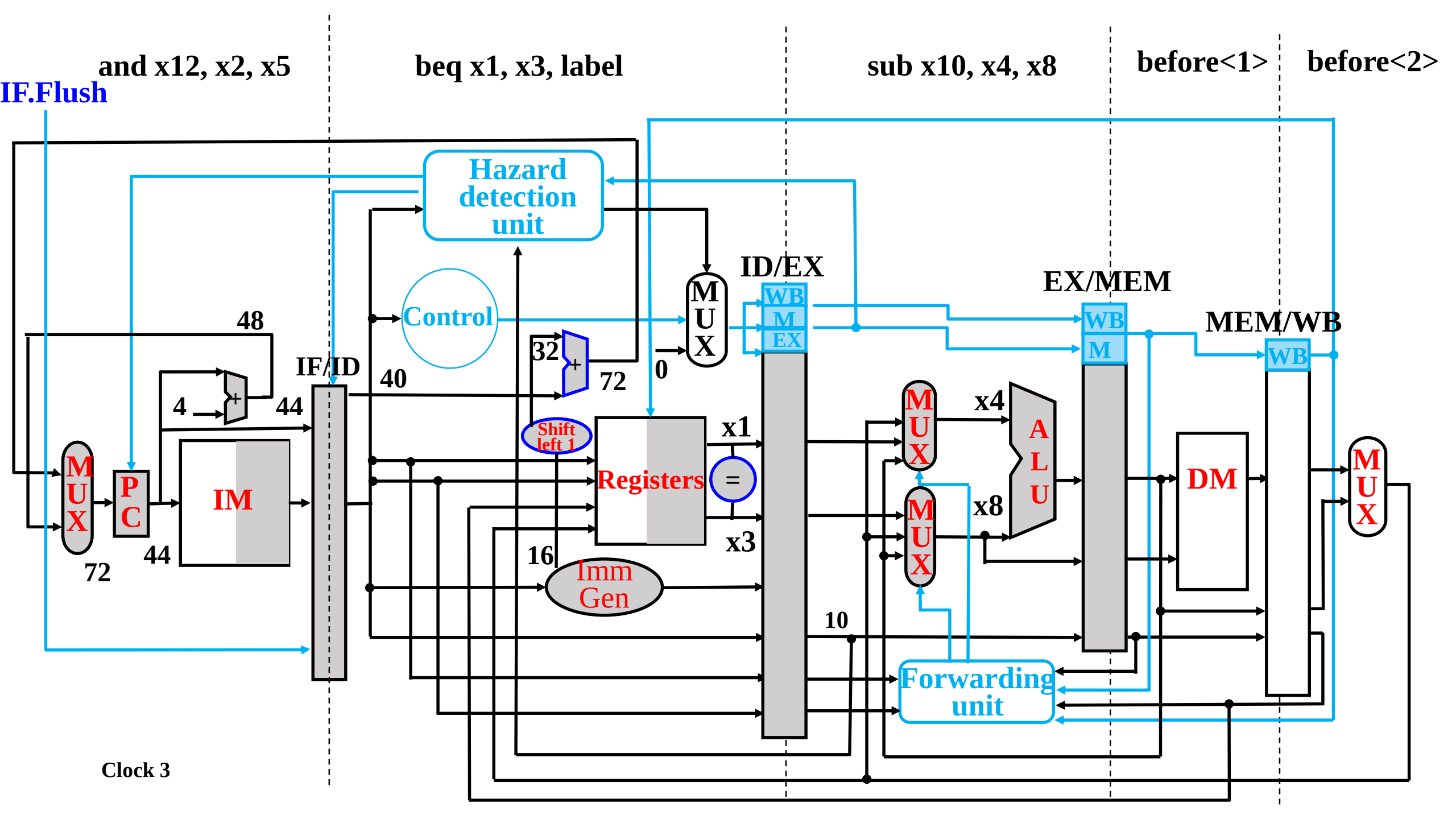
48: or x13, x2, x6

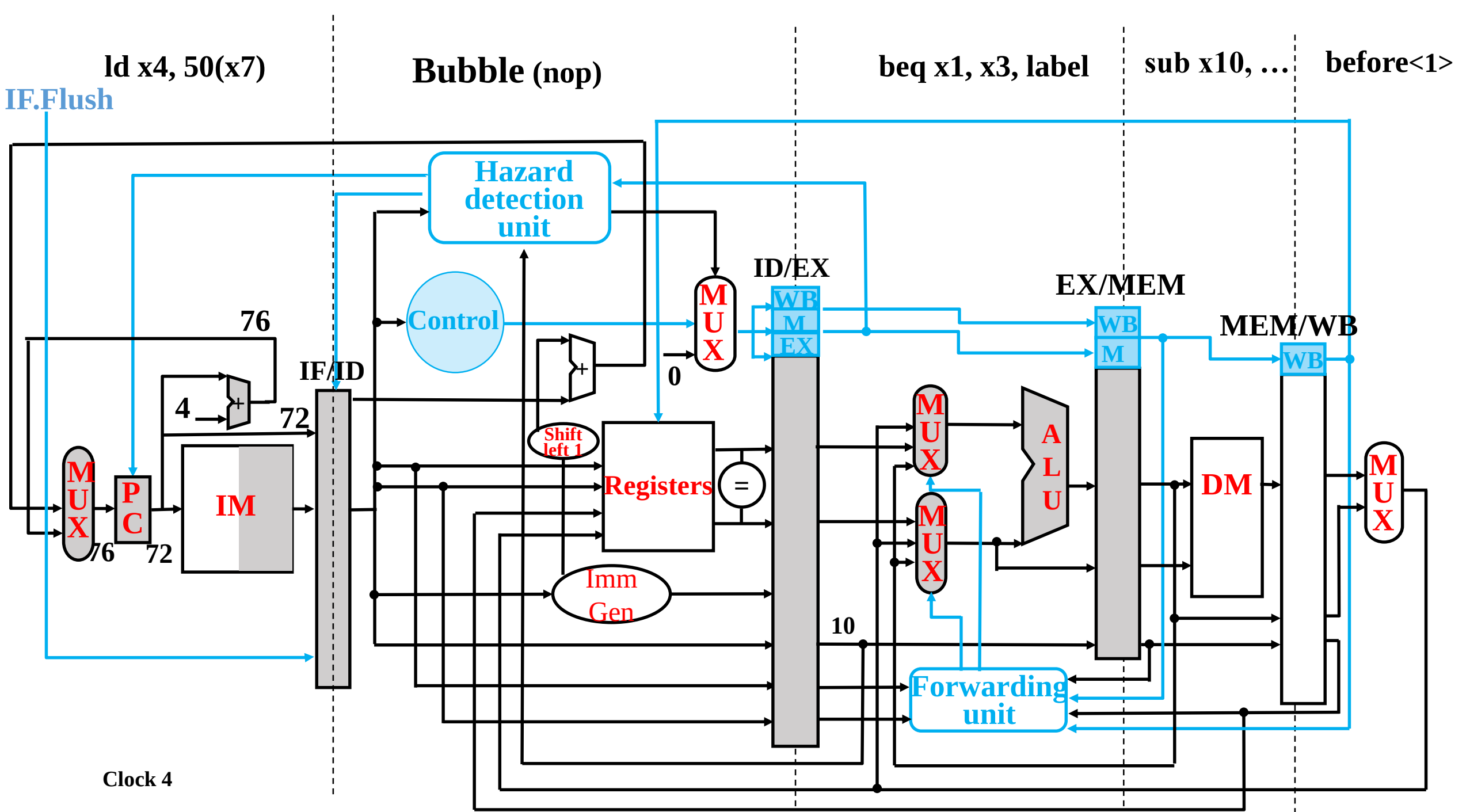
52: add x14, x4, x2

56: sub x15, x6, x7

...

72: ld x4, 50(x7)





动态分支预测

- 对于更深的流水线，从时钟周期数的角度来说，分支预测错误的代价会增大。
- 使用动态分支预测
 - 一种实现方式：采用分支预测缓存或分支历史表
 - 是一块按照分支指令的低位地址索引定位的小容量存储器
 - 包含一个或多个位（bit）以表明一个分支最近是否发生了跳转
- 1-Bit预测机制：用1位表示最近是否发生了跳转
 - 查表，使用表中记录结果作为预测结果
 - 开始取指令
 - 如果预测错误，则清掉错误指令，并更改分支预测缓存中的记录

分支目标计算

- 除了判断分支是否发生，还要计算分支目标地址
 - 发生跳转时需要一个时钟周期的代价来计算分支目标地址
- 使用分支目标缓存
 - 缓存分支目标PC值或目标指令
 - 根据当前分支指令的PC值来索引定位
 - 如果缓存中有目标PC值或指令，且分支预测跳转，就能读取缓存结果，立即使用

第五章 流水线处理器

- 流水线概述
- 流水线数据通路和控制
- 数据冒险：前递与停顿
- 控制冒险
- 例外

流水线实现中的例外处理小结

- IF阶段的指令：变为nop操作
- ID阶段的指令：增加新的逻辑部件，使译码阶段输出为0
- EX阶段的指令：需要保留x1原本的值
- 例外之前的指令，正常完成
- 设置SEPC和SCAUSE的寄存器值
- 转到例外处理程序

第六章 存储器

重点

1. 存储系统的层次结构

Cache—主存和主存—辅存层次的作用

程序访问的局部性原理与存储系统层次结构的关系

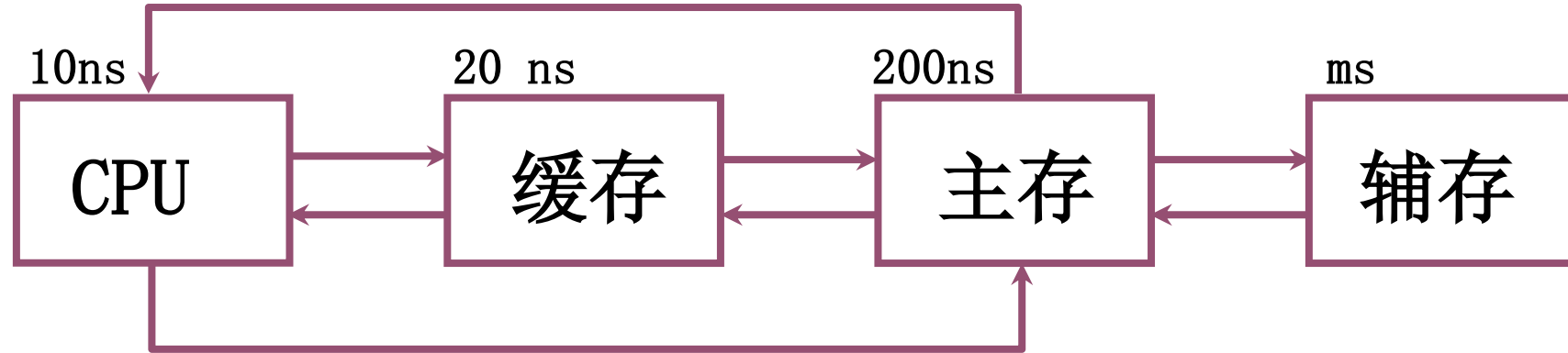
2. 主存、Cache工作原理及技术指标

3. 半导体存储芯片的外特性以及与 CPU 的连接

4. 如何提高访存速度

5. 虚拟存储器的工作原理

缓存—主存层次和主存—辅存层次



(速度) (容量)
缓存—主存 主存—辅存



主存储器	虚拟存储器
实地址	虚地址
物理地址	逻辑地址

动态RAM和静态RAM的比较

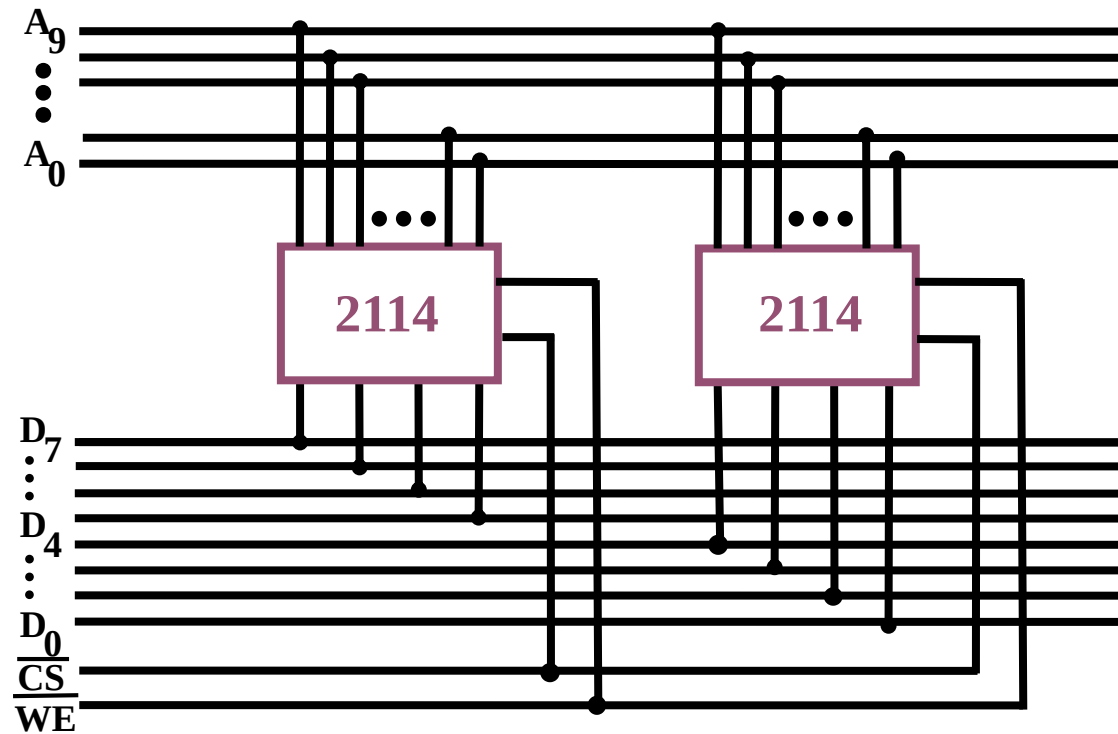
	主存 DRAM	SRAM 缓存
存储原理	电容	触发器
集成度	高	低
芯片引脚	少	多
功耗	小	大
价格	低	高
速度	慢	快
刷新	有	无

五、存储器与 CPU 的连接

1. 存储器容量的扩展

(1) 位扩展（增加存储字长）

用 **2 片** $1\text{K} \times 4$ 位 存储芯片组成 $1\text{K} \times 8$ 位的存储器

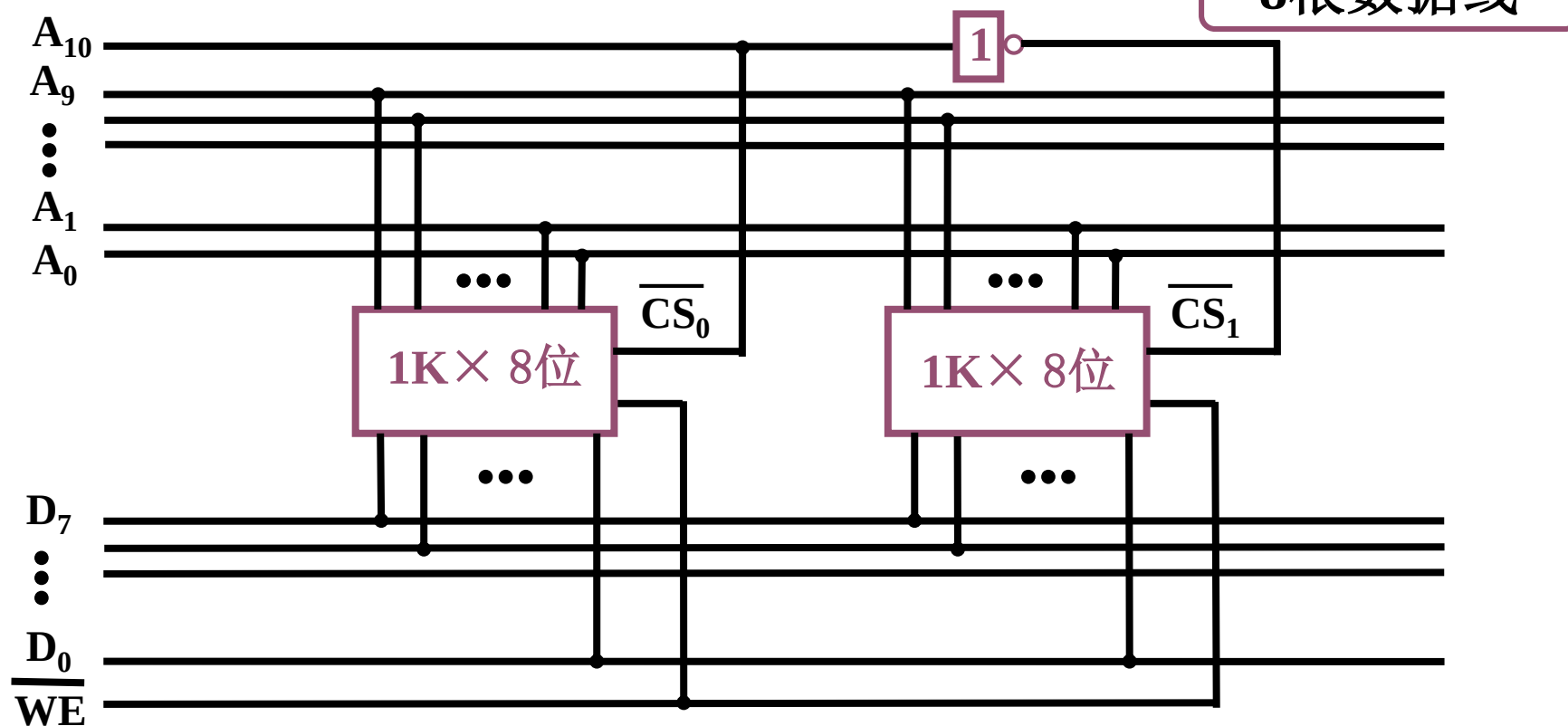


10根地址线

8根数据线

(2) 字扩展（增加存储字的数量）

用 2 片 $1\text{K} \times 8$ 位 存储芯片组成 $2\text{K} \times 8$ 位的存储器

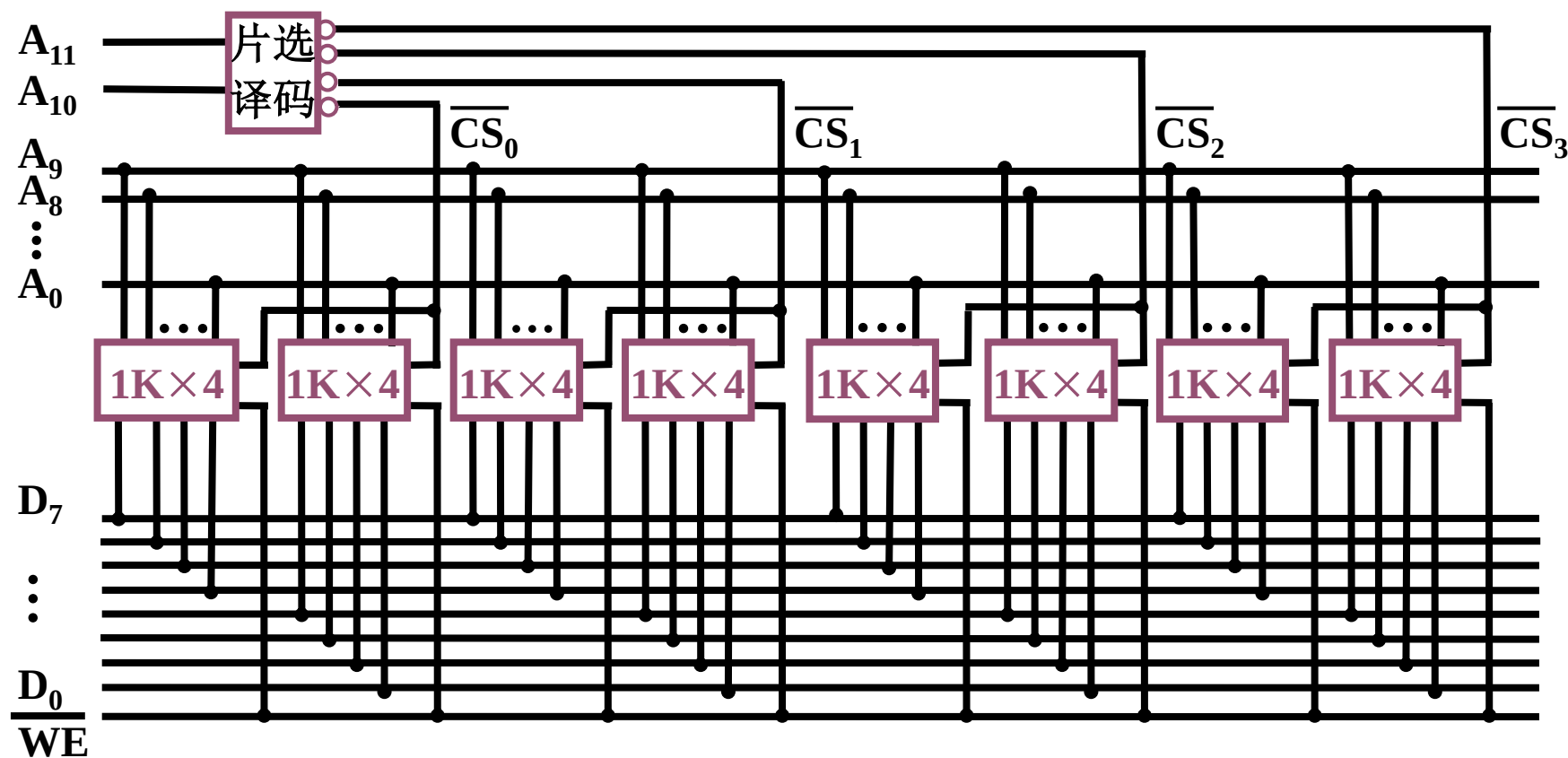


(3) 字、位扩展

用 8 片 $1\text{K} \times 4$ 位 存储芯片组成 $4\text{K} \times 8$ 位的存储器

12根地址线

8根数据线



主存储器

2. 存储器与 CPU 的连接

- (1) 地址线的连接
- (2) 数据线的连接
- (3) 读/写命令线的连接
- (4) 片选线的连接
- (5) 合理选择存储芯片
- (6) 其他：时序、负载等

例6.1

设 CPU 有 16 根地址线，8 根数据线，
 $\overline{\text{MREQ}}$ 访存控制信号（低电平有效），
 $\overline{\text{WR}}$ 读/写控制信号（高电平为读，低电平为写）
RAM：1K×4位；4K×8位；8K×8 位
ROM：2K×8位；4K×8位；8K×8 位
74LS138 译码器和各种门电路

画出 CPU 与存储器的连接图，要求

① 主存地址空间分配：

6000H~67FFH 为系统程序区；

6800H~6BFFH 为用户程序区。

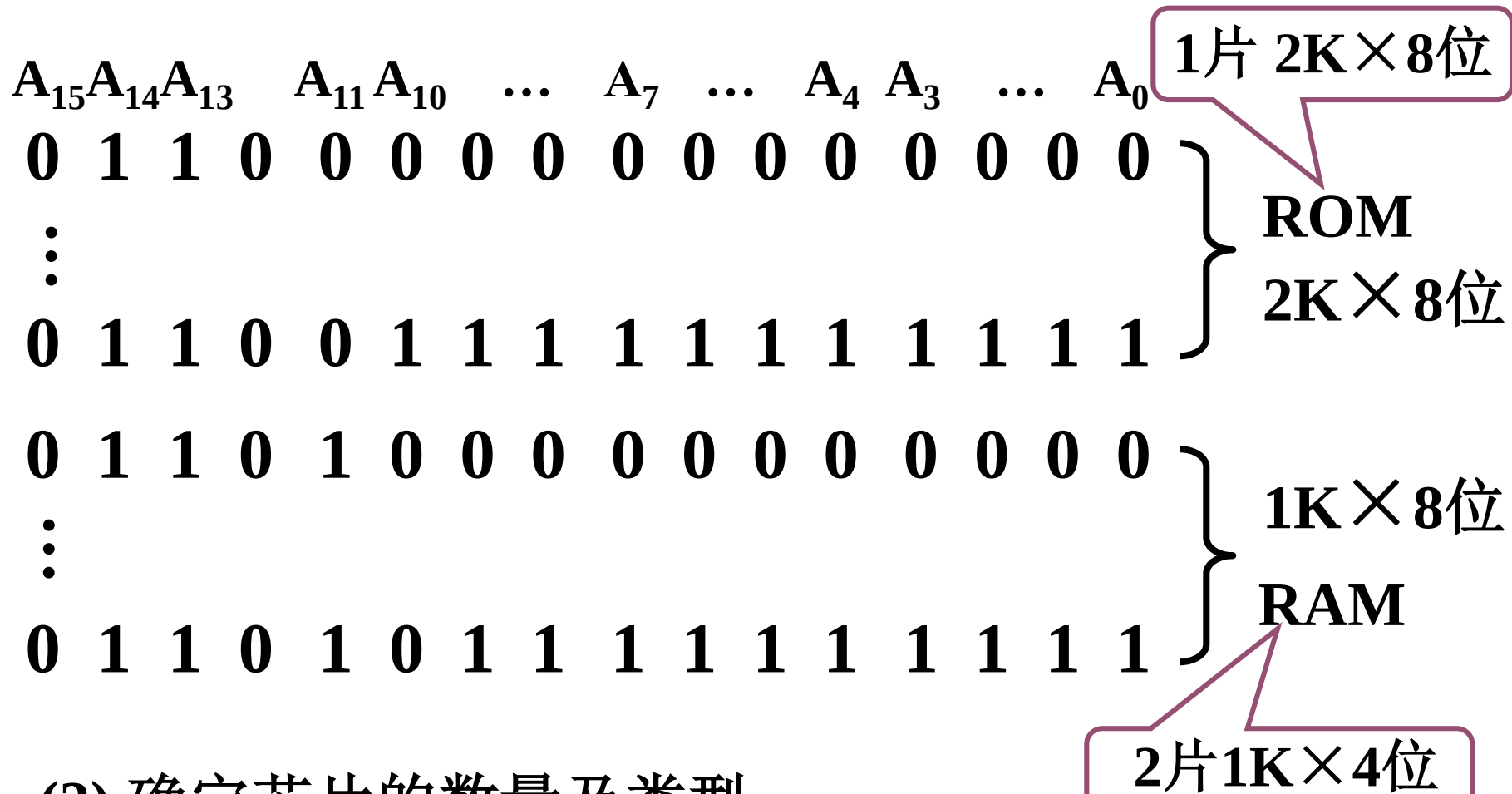
② 合理选用上述存储芯片，说明各选几片？

③ 详细画出存储芯片的片选逻辑图。

例6.1 解析

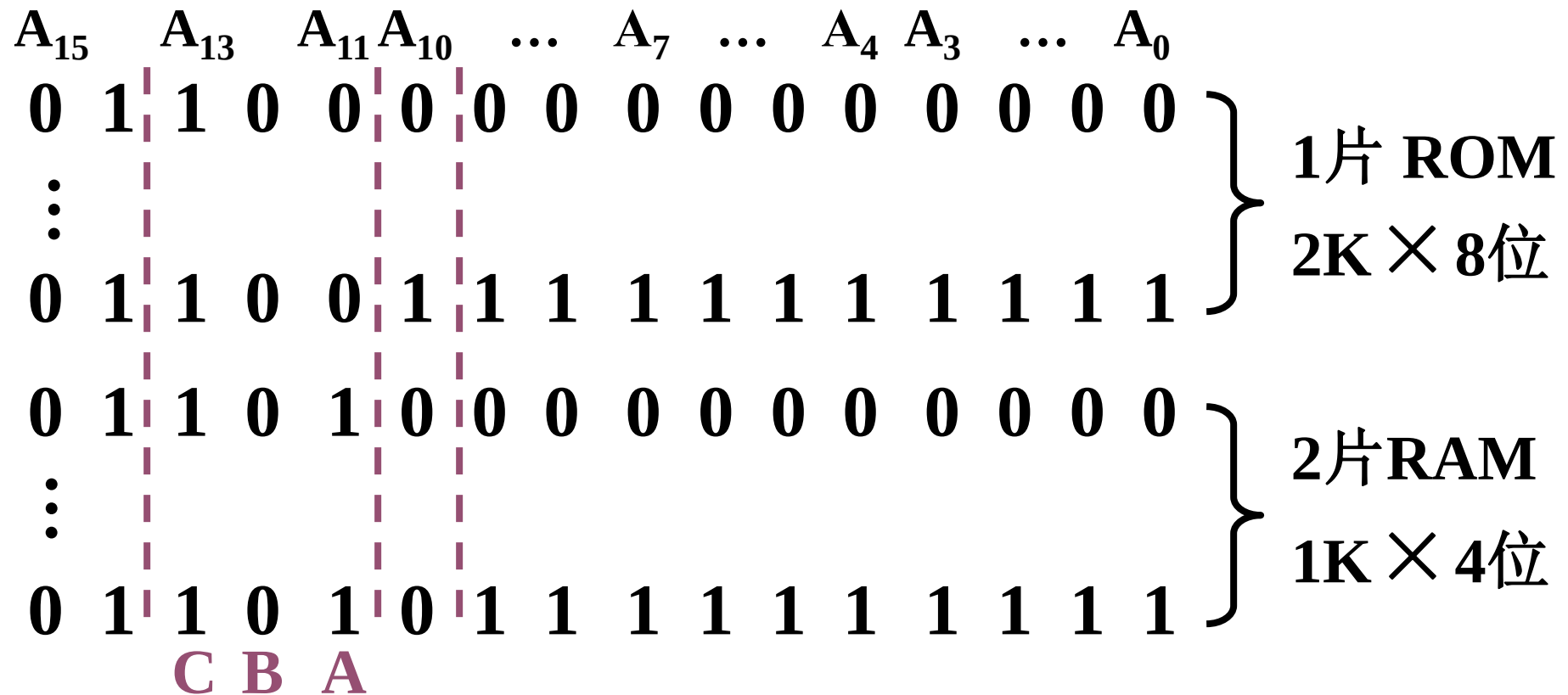
6000H~67FFH 为系统程序区；
6800H~6BFFH 为用户程序区。

(1) 写出对应的二进制地址码



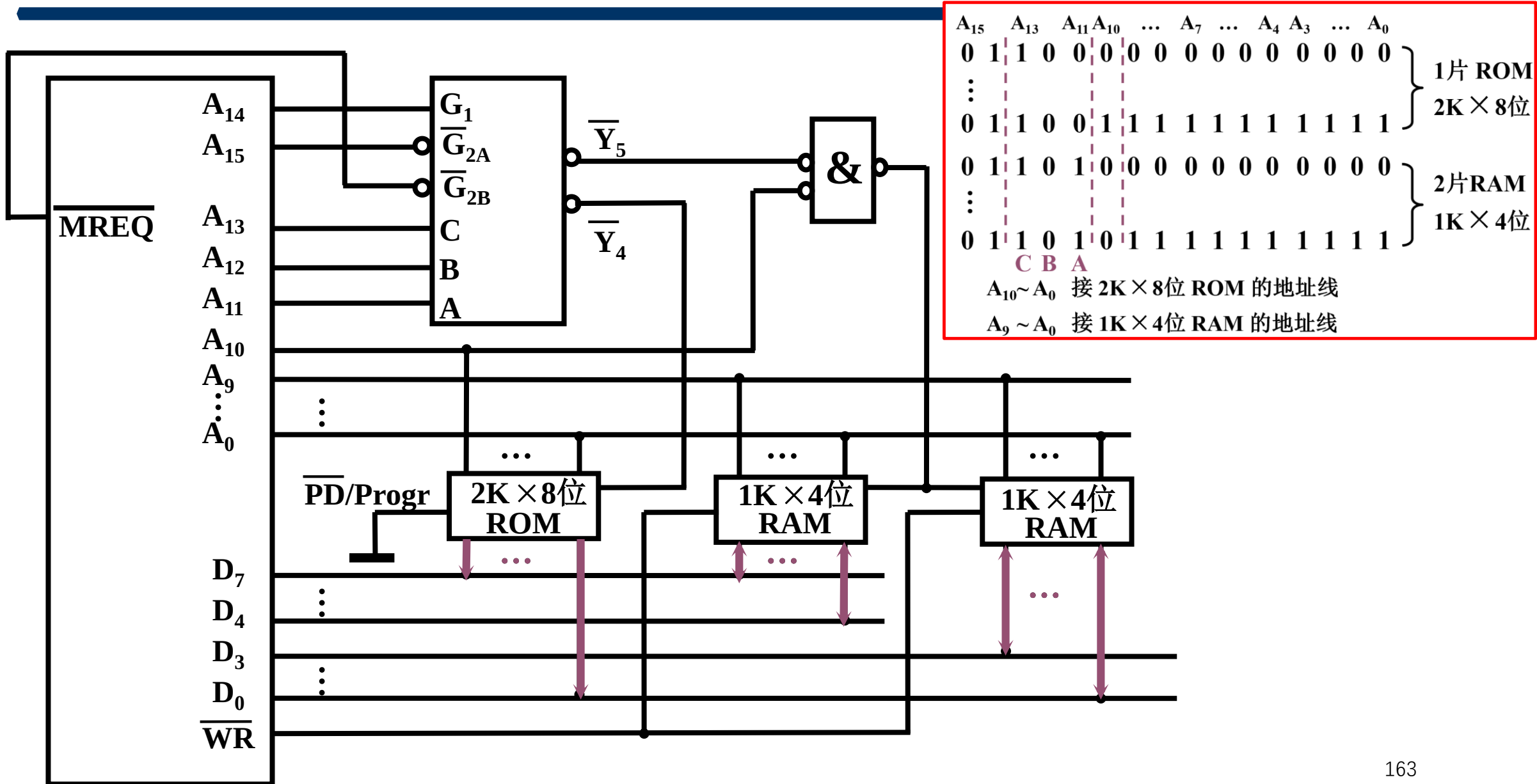
(2) 确定芯片的数量及类型

(3) 分配地址线



(4) 确定片选信号

例 6.1 CPU 与存储器的连接图



汉明码

- 组成汉明码的三要素

汉明码的组成需增添 ? 位检测位

$$2^k \geq n + k + 1$$

检测位的位置 ?

$$2^i \ (i = 0, 1, 2, 3, \dots)$$

检测位的取值 ?

与该位所在的检测“小组”中承担的奇偶校验任务有关

各检测位 C_i 所承担的检测小组

- C_1 检测 g_1 小组包含位 1,3,5,7,9,11,... (XXXX**1**)
 - C_2 检测 g_2 小组包含位 2,3,6,7,10,11,... (XXX**1**X)
 - C_4 检测 g_3 小组包含位 4,5,6,7,12,13,... (XX**1**XX)
 - C_8 检测 g_4 小组包含位 8,9,10,11,12,13,14,15,24,... (X**1**XXX)
-
- g_i 小组独占第 2^{i-1} 位
 - g_i 和 g_j 小组共同占第 $2^{i-1} + 2^{j-1}$ 位
 - g_i 、 g_j 和 g_l 小组共同占第 $2^{i-1} + 2^{j-1} + 2^{l-1}$ 位

例6.4 求 0101 按“偶校验”配置的汉明码

解：∵ $n = 4$

根据 $2^k \geq n + k + 1$

得 $k = 3$

汉明码排序如下：

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	0	C_4	1	0	1
	0	1		0			

∴ 0101 的汉明码为 **0100101**

练习1 按配偶原则配置 0011 的汉明码

解： $\because n = 4$ 根据 $2^k \geq n + k + 1$

取 $k = 3$

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	0	C_4	0	1	1
	1	0		0			

$$C_1 = 3 \oplus 5 \oplus 7 = 1$$

$$C_2 = 3 \oplus 6 \oplus 7 = 0$$

$$C_4 = 5 \oplus 6 \oplus 7 = 0$$

$\therefore$ 0011 的汉明码为 1000011

3. 汉明码的纠错过程

形成新的检测位 P_i ，其位数与增添的检测位有关，
如增添 3 位（ $k = 3$ ），新的检测位为 $P_4 P_2 P_1$ 。

以 $k = 3$ 为例， P_i 的取值为

$$P_1 = \overset{C_1}{1} \oplus 3 \oplus 5 \oplus 7$$

$$P_2 = \overset{C_2}{2} \oplus 3 \oplus 6 \oplus 7$$

$$P_4 = \overset{C_4}{4} \oplus 5 \oplus 6 \oplus 7$$

对于按“偶校验”配置的汉明码
不出错时 $P_1 = 0, P_2 = 0, P_4 = 0$

第6章 存储器

难点

1. 对于一定容量的存储器，按字节或字访问的寻址范围是不同的

6.2 主存储器—主存中存储单元地址的分配

- unsigned int value = 0x12345678

字地址

字节地址

0x4000	12	34	56	78
0x4004				
0x4008				
0x4012				

字地址

字节地址

0x4000	78	56	34	12
0x4004				
0x4008				
0x4012				

大端模式Big-Endian:低地址存放高位

小端模式Little-Endian: 低地址存放低位

如 16 MB (2^{27} 位) 的存储器

寻址范围

容量

按 字节 寻址

$$2^{24} = 16 \text{ M}$$

$$2^{24} \times 2^3 = 2^{27} \text{ 位}$$

按 字 (16位) 寻址

$$2^{23} = 8 \text{ MW}$$

$$2^{23} \times 2^4 = 2^{27} \text{ 位}$$

按 字 (32位) 寻址

$$2^{22} = 4 \text{ MW}$$

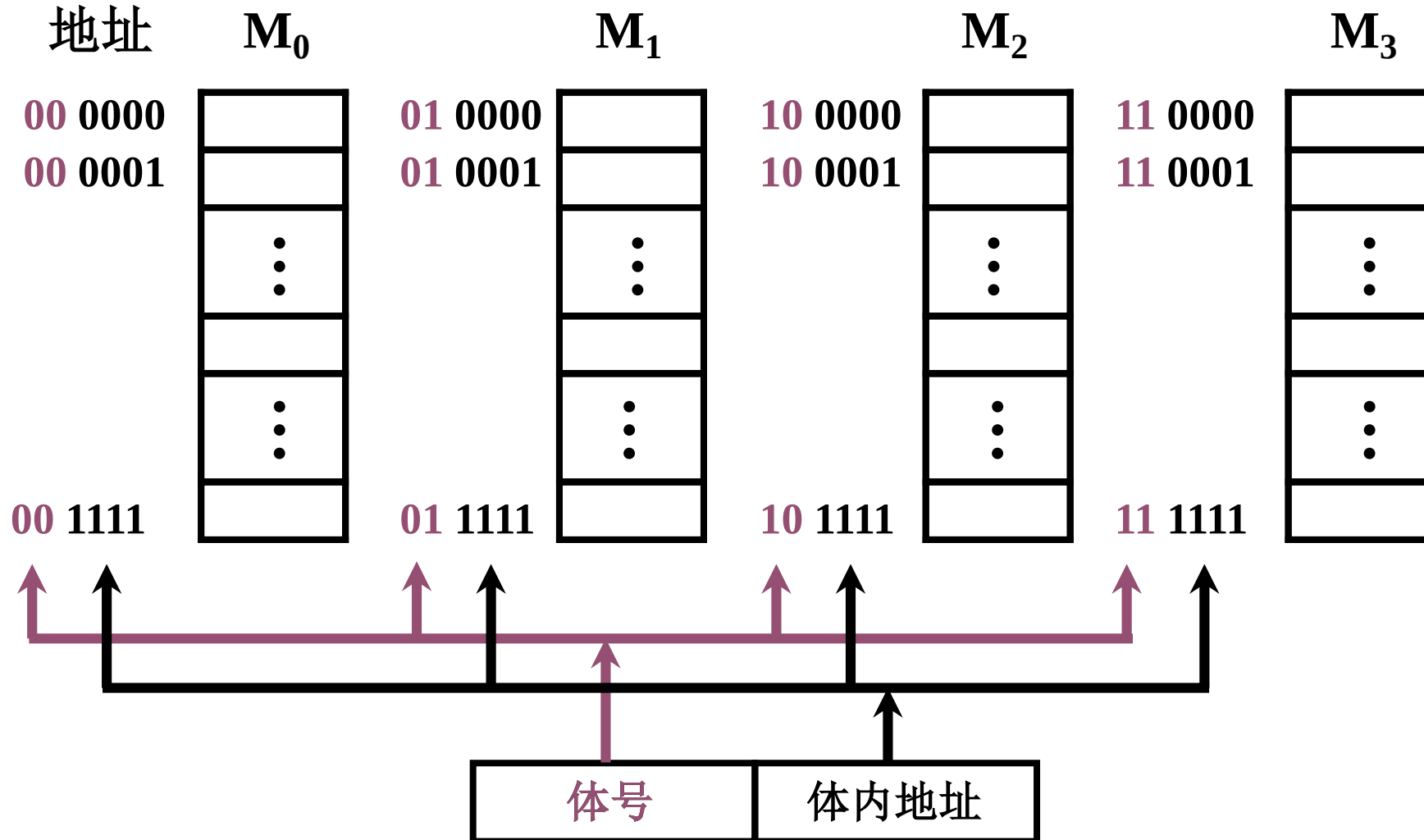
$$2^{22} \times 2^5 = 2^{27} \text{ 位}$$

第6章 存储器

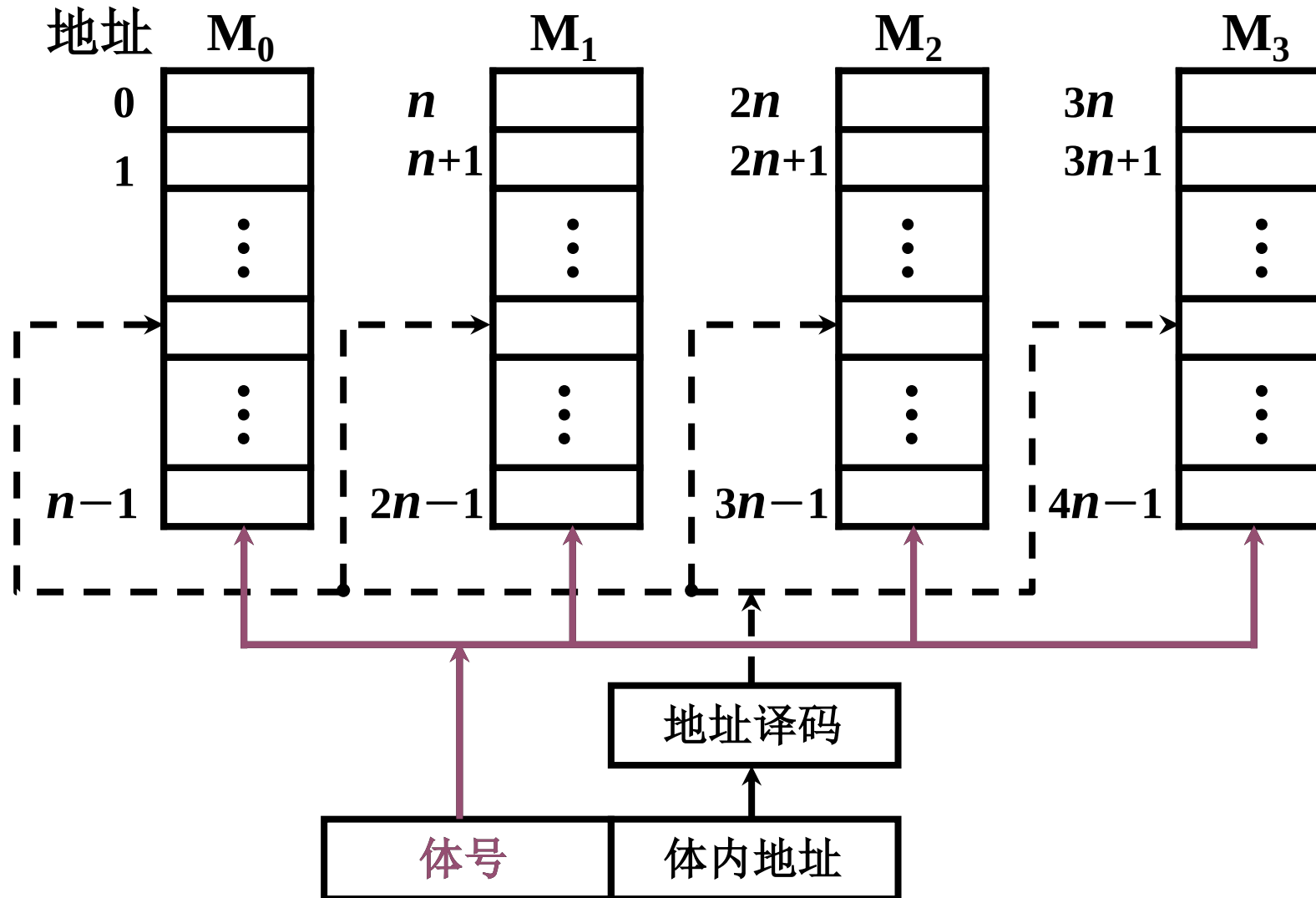
难点

1. 对于一定容量的存储器，按字节或字访问的寻址范围是不同的
2. 多体并行结构存储器顺序编址和交叉编址对访存速度的影响

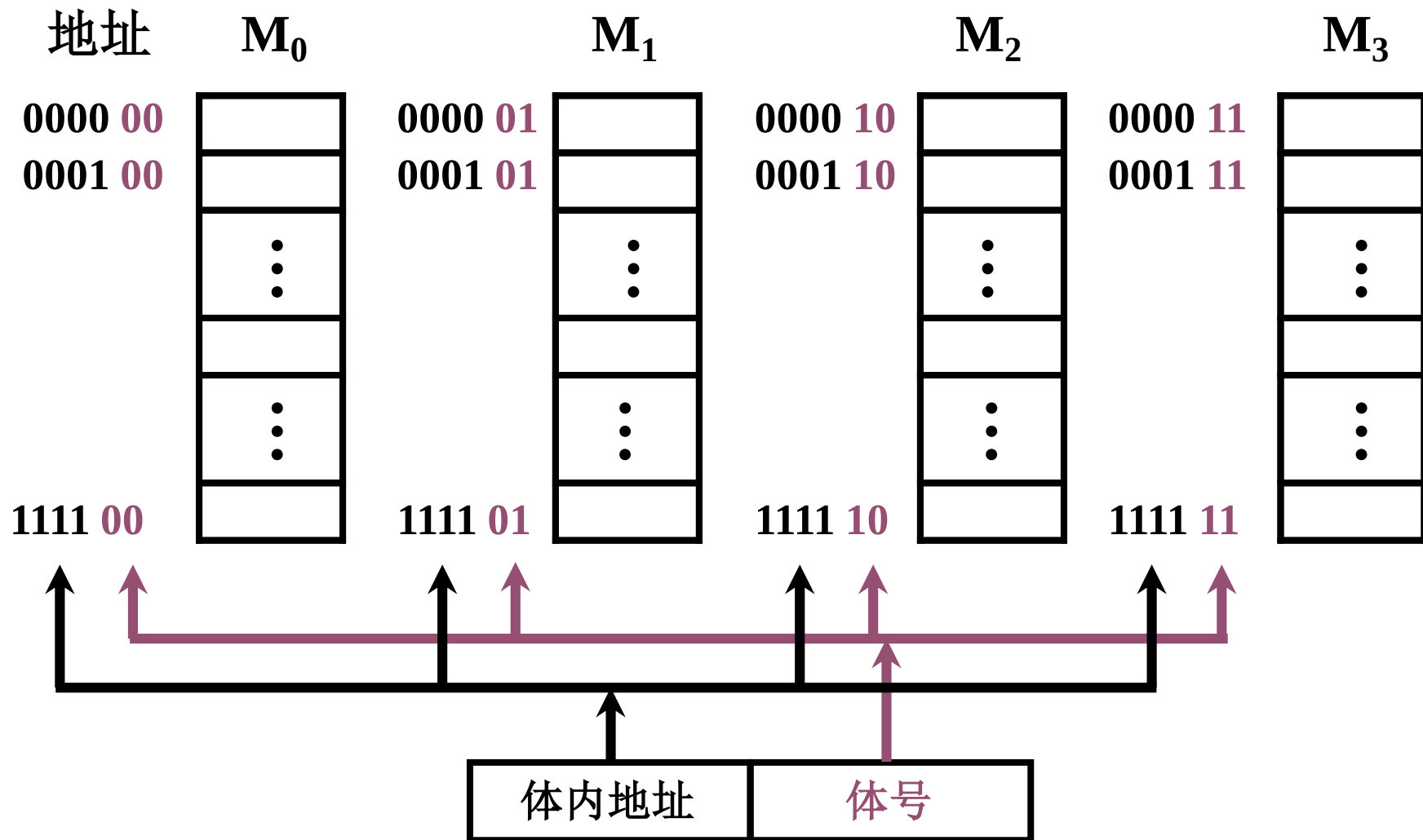
高位交叉编址多体存储器（顺序存储）



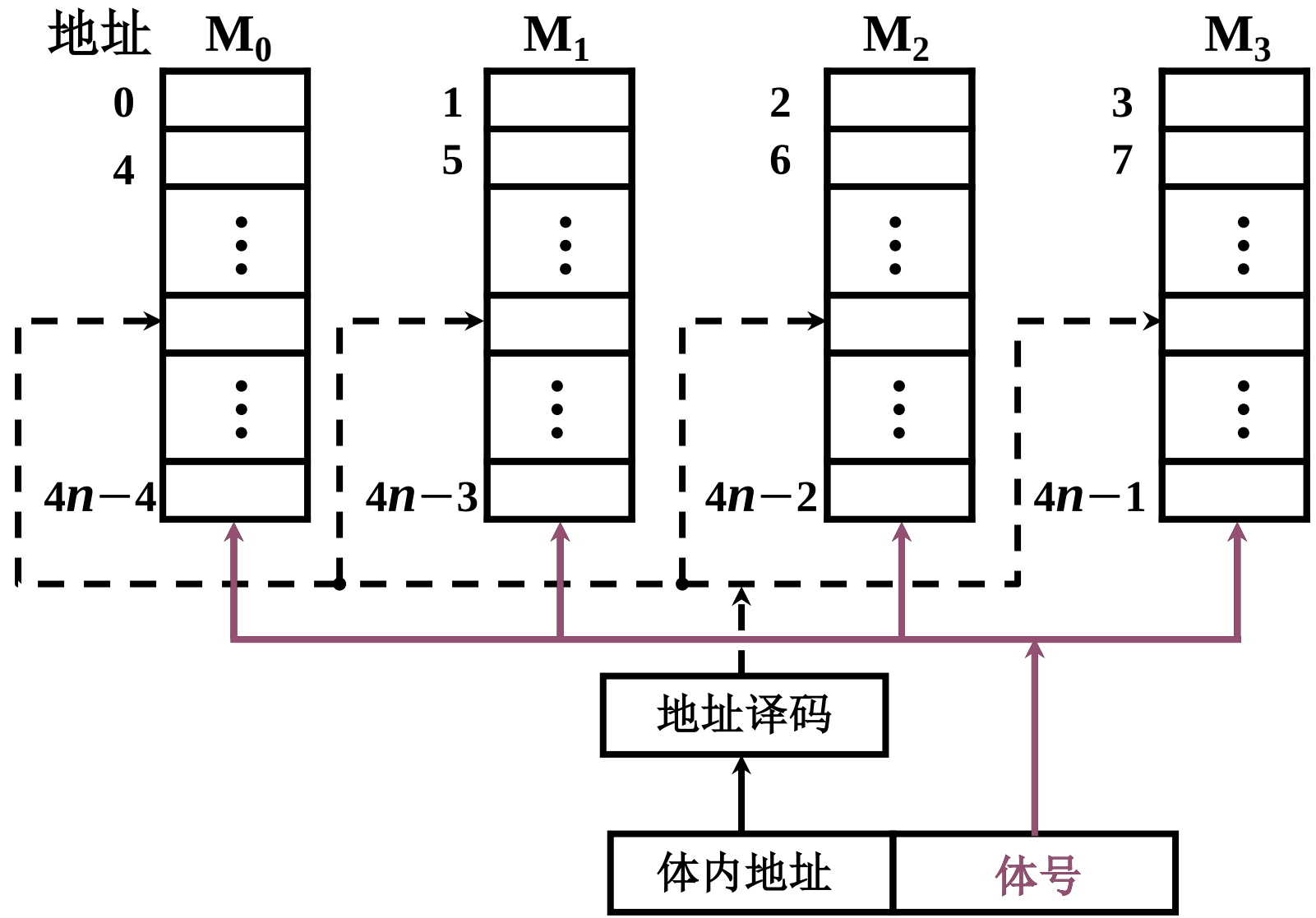
高位交叉编址——各个存储体可并行工作



低位交叉——各个体轮流编址

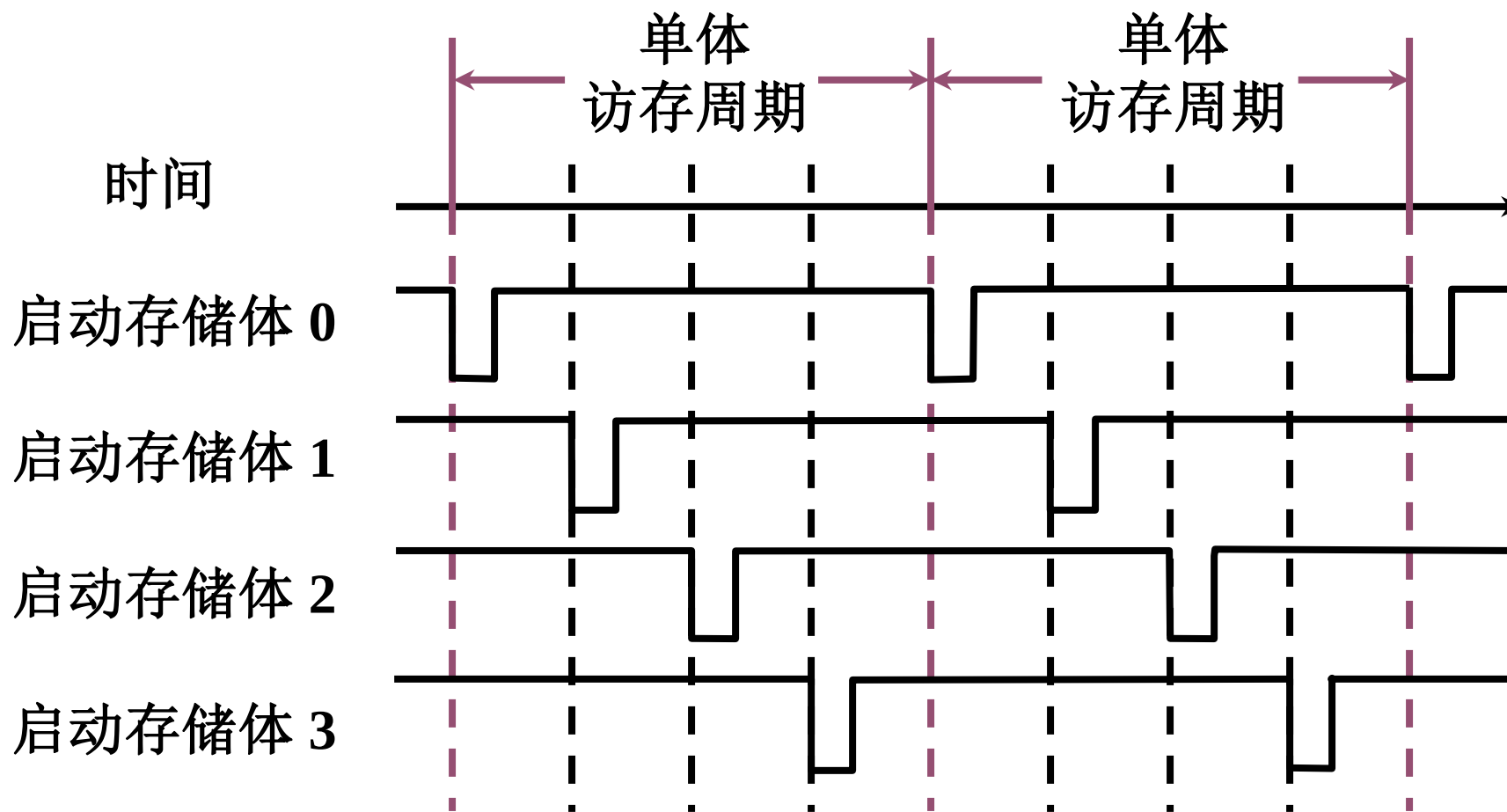


低位交叉——各个存储体轮流编址



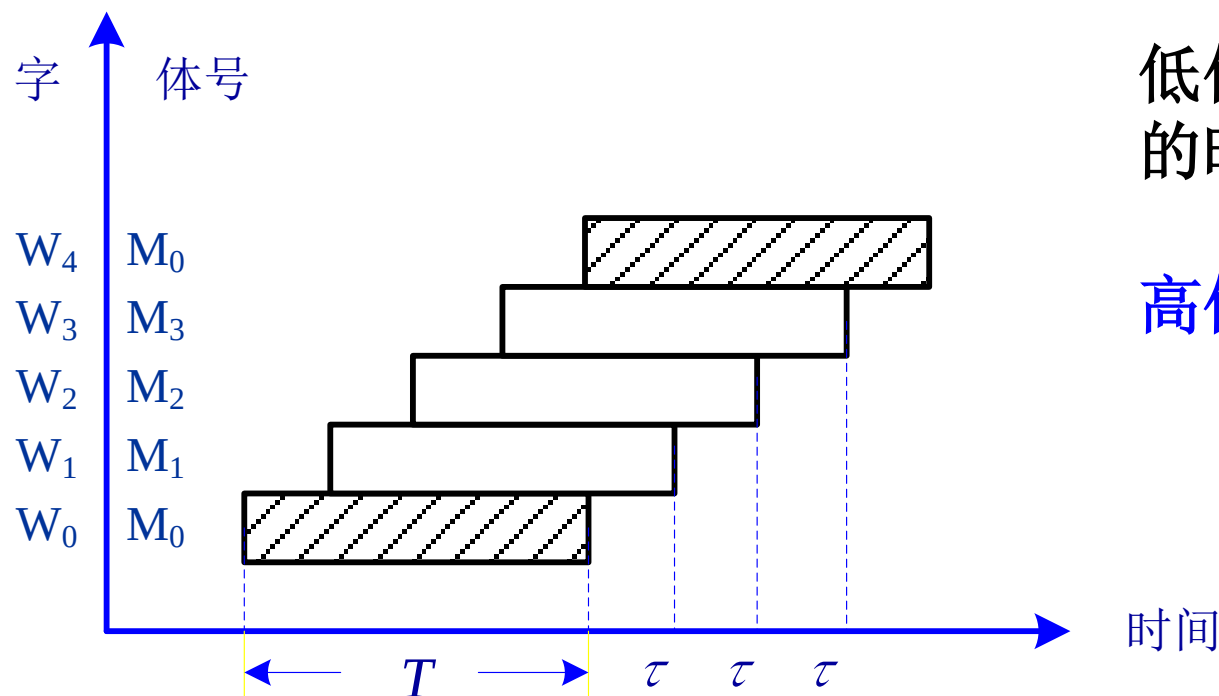
低位交叉编址的特点

在不改变存取周期的前提下，增加存储器的带宽



流水线方式存取

- 设 n 体低位交叉存储器，存取周期为 T ，总线传输周期为 τ （为实现流水线方式存取，应满足 $T = n\tau$ ）。
- （唐书例4.6中 $T=200\text{ns}$ ， $\tau=50\text{ns}$ ）



低位交叉存储器连续读取 n 个字所需的时间为

$$T + (n - 1)\tau$$

高位交叉存储读取 n 个字所需时间为

$$4T$$

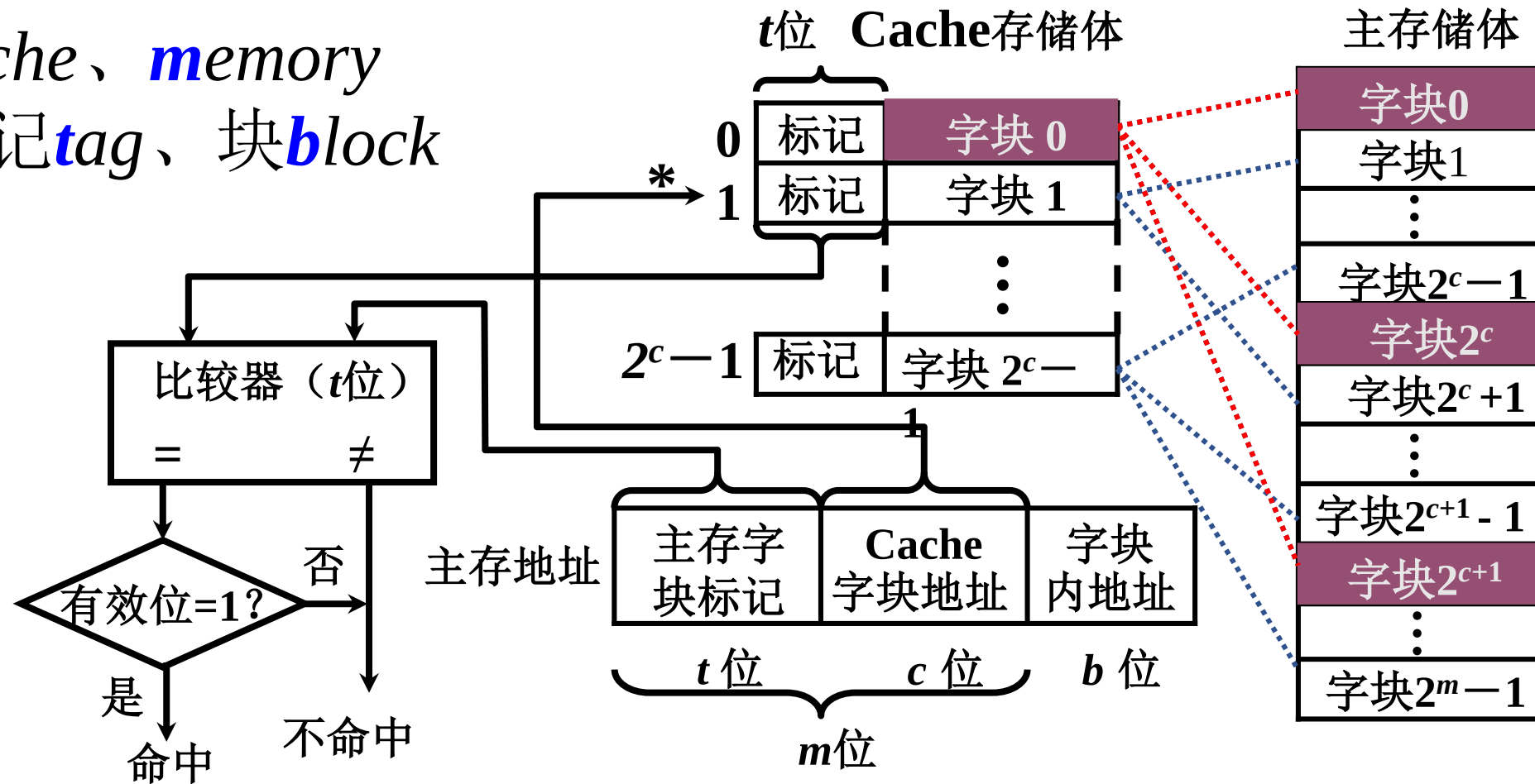
第6章 存储器

难点

1. 对于一定容量的存储器，按字节或字访问的寻址范围是不同的
2. 多体并行结构存储器顺序编址和交叉编址对访存速度的影响
3. 不同的 Cache — 主存地址映射，直接影响主存地址字段的分配、替换策略及命中率

Cache-主存直接映射

cache、*memory*
标记*tag*、块*block*



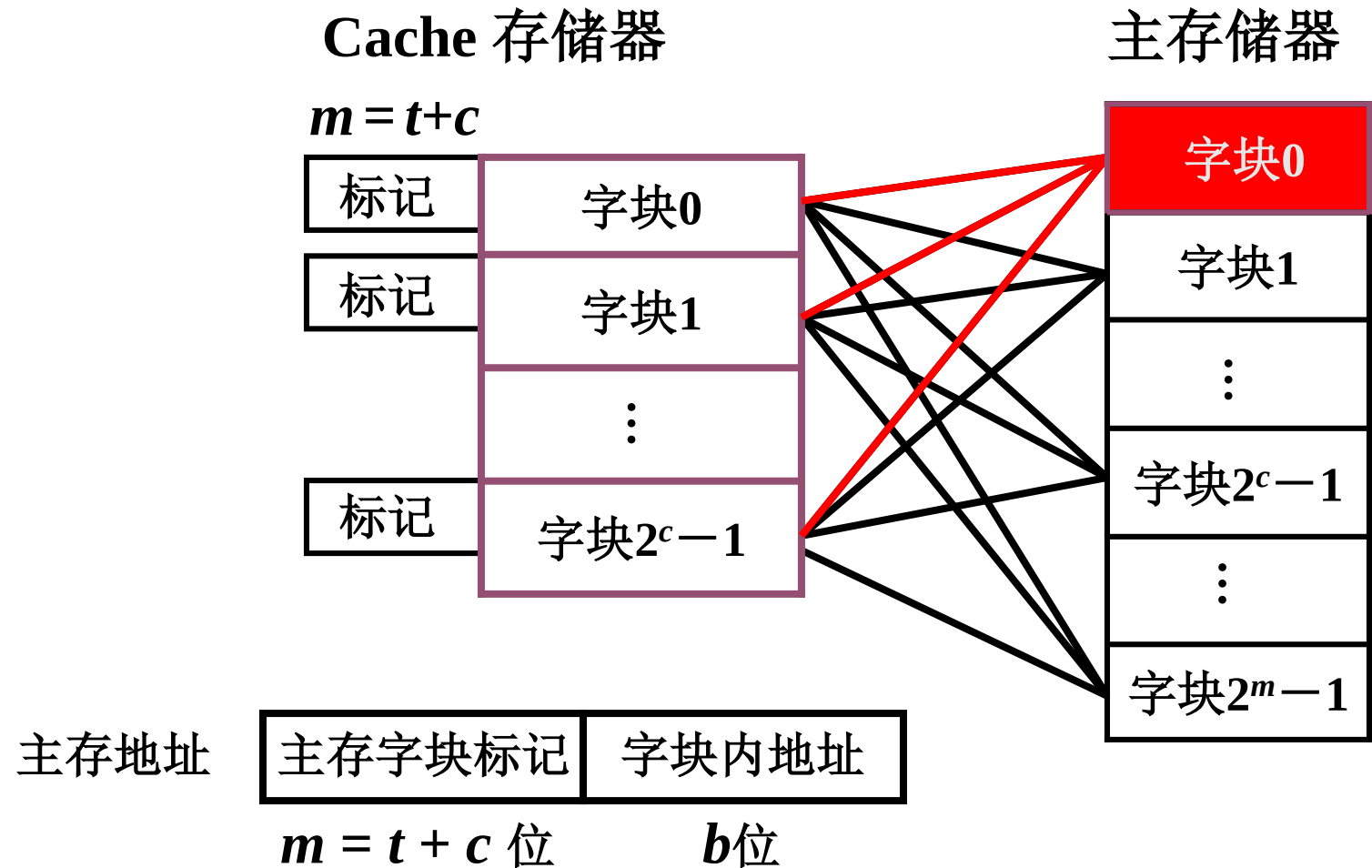
每个缓存块 i 可以和 若干个 主存块 对应

每个主存块 j 只能和 一个 缓存块 对应

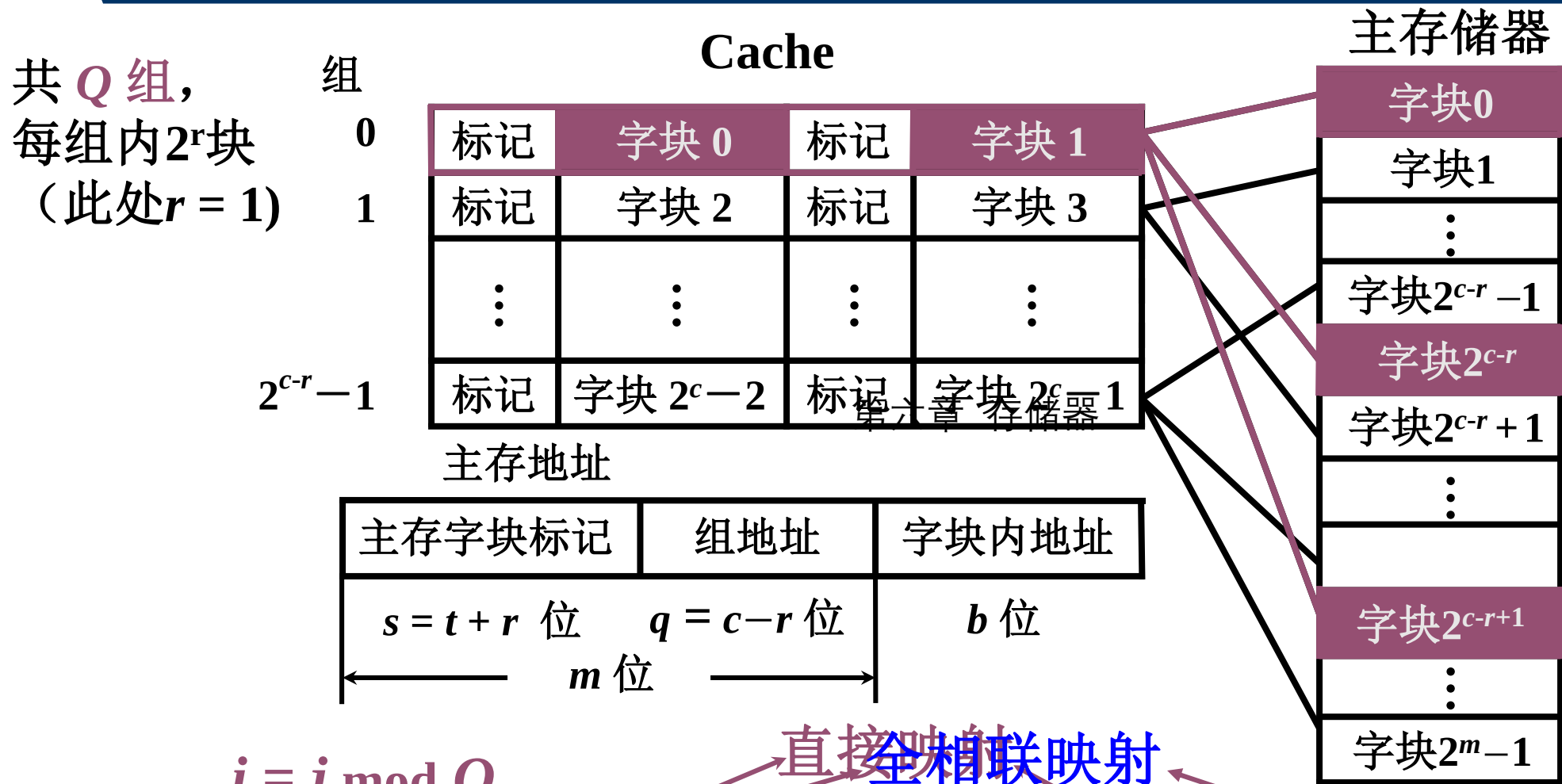
$$i = j \bmod C$$

Cache-主存全相联映射

主存 中的 任一 块 可以映射到 缓存 中的 任一 块



Cache-主存组相联映射



$$i = j \bmod Q$$

某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

6.3 高速缓冲存储器——小结

- **实际应用：**块设备缓存、硬盘缓存、web cache...

不灵活

直接映射 某一主存块 只能固定 映射到 某一缓存块

全相联映射 某一主存块 能 映射到 任一缓存块

组相联映射 某一主存块 只能映射 某一缓存组中的任一块

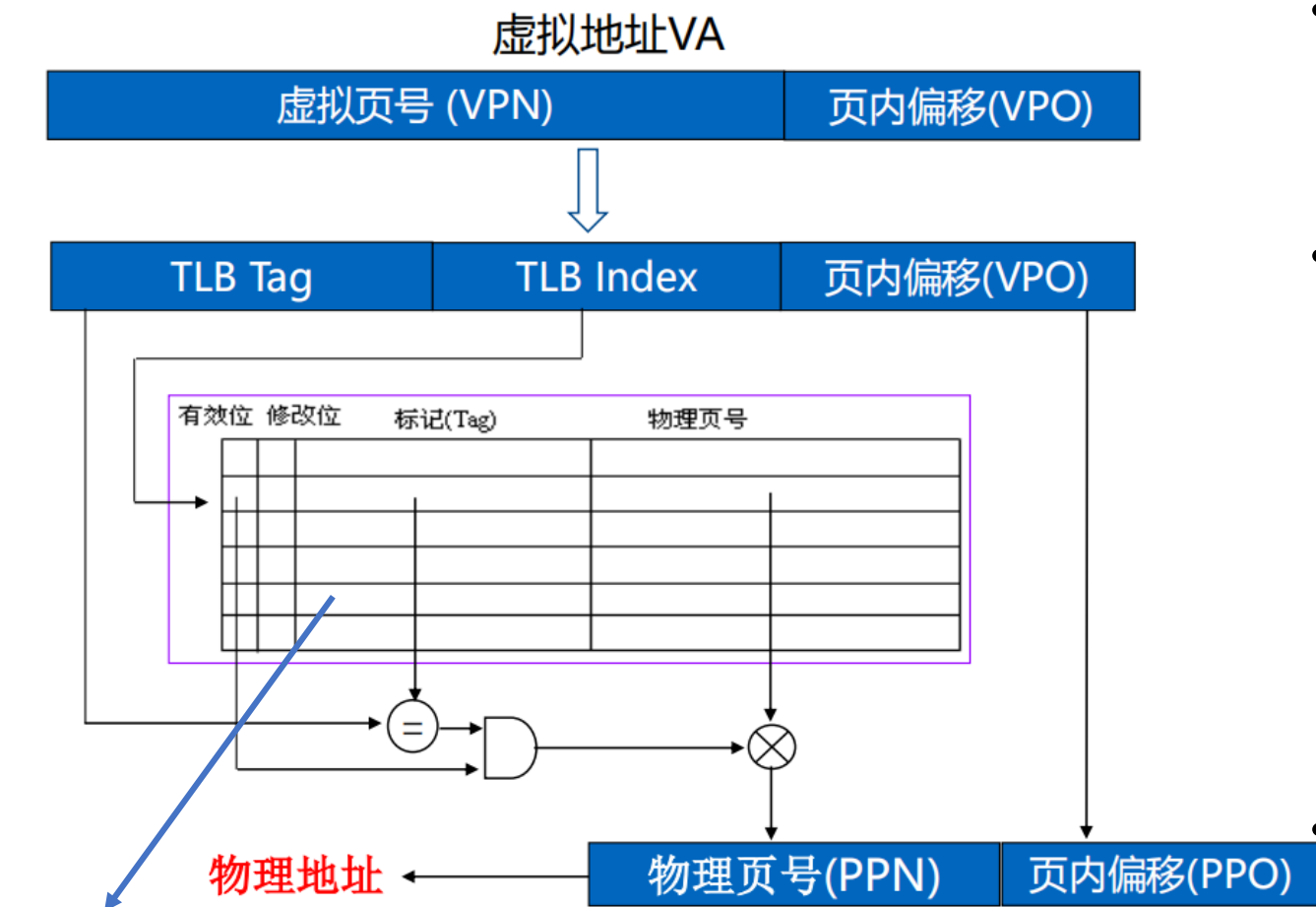
成本高

第六章 存储器

难点

1. 对于一定容量的存储器，按字节或字访问的寻址范围是不同的
2. 多体并行结构存储器顺序编址和交叉编址对访存速度的影响
3. 不同的 Cache — 主存地址映射，直接影响主存地址字段的分配、替换策略及命中率
4. TLB下虚拟存储器虚拟地址到物理地址转换

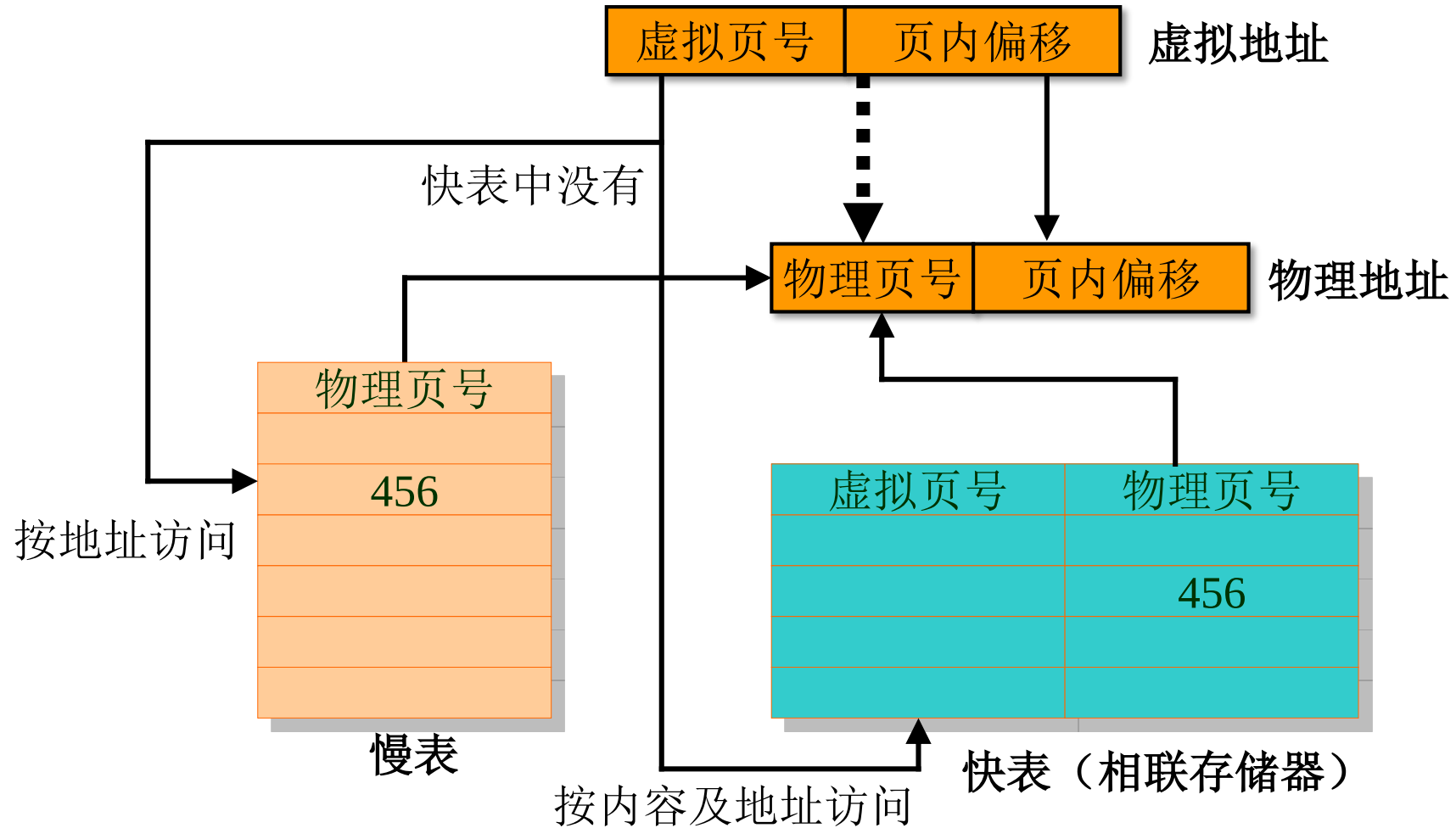
TLB虚实地址转换



TLB中的页表项和主存中的页表项的差异在于多出了一个Tag标记，用于区分不同的虚拟页。

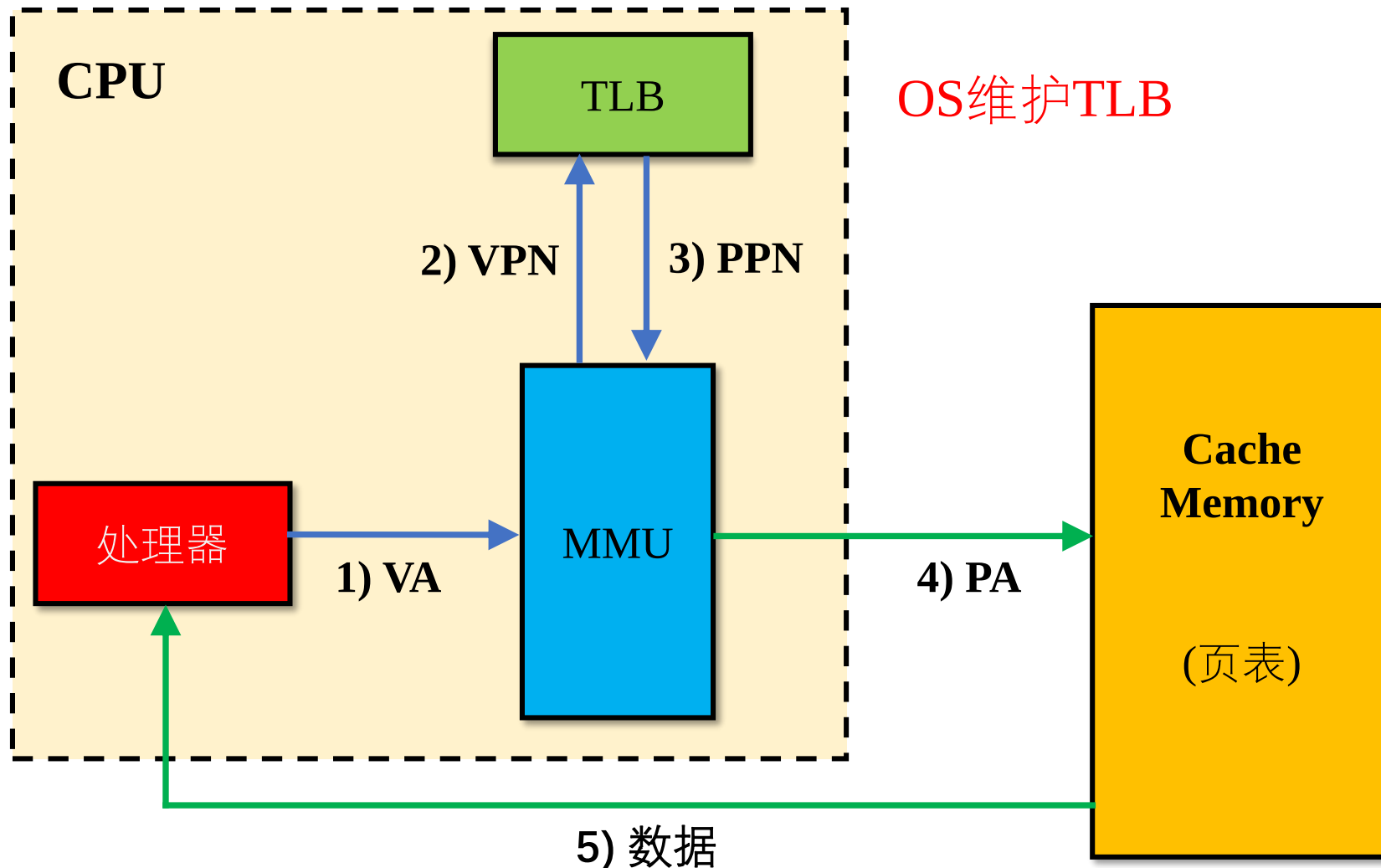
- 采用TLB后，对虚拟地址进行二次划分，主要是把VPN划分为TLB Tag和TLB index，页内偏移地址不变。
- 当CPU传送一个新的虚拟地址后，从虚拟地址中划分出TLB Tag和TLB index，使用TLB index找到快表中对应的页表项，比较当前虚拟地址的TLB Tag和页表项中存储的TLB Tag是否相同。如果相同且有效位为1，则快表命中，直接基于页表项中存储的物理页号生成物理地址；否则，快表未命中，需要访问主存中的页表。
- TLB中存放的是页表的子集，取决于TLB Index的位数。因此，不同的虚拟页可能会有相同的TLB Index，需要通过TLB Tag来区分。这类似于Cache的直接相连映射。

TLB——经快慢表实现内部地址转换

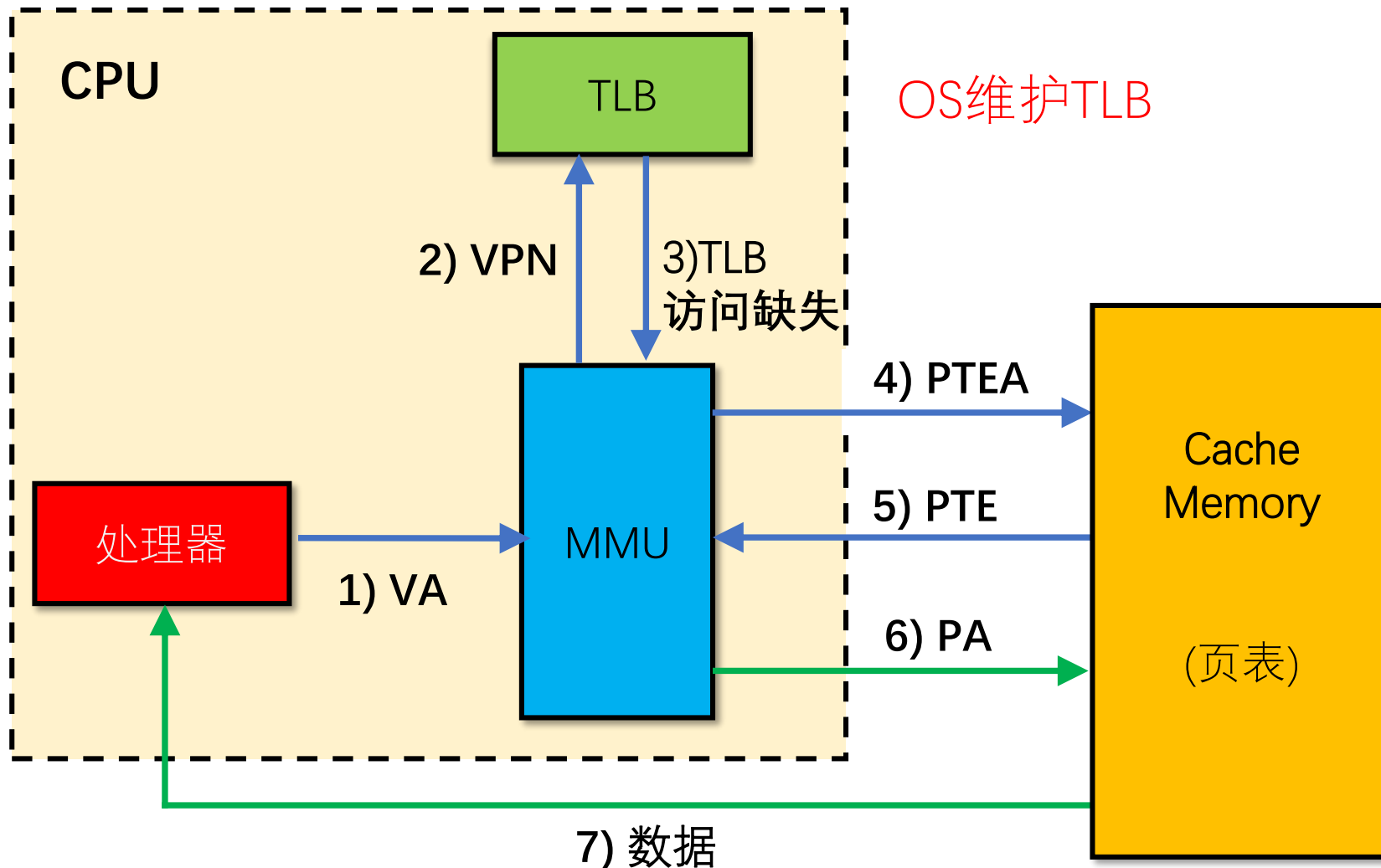


TLB命中

- CPU生成虚拟地址，传递给MMU
- MMU利用虚页号VPN查询TLB
如果TLB访问命中，即页表项在TLB中且相应的PTE中有效位为1，则TLB向MMU返回与VPN对应的物理页号PPN
- MMU利用返回的PPN构造物理地址PA，利用PA访问Cache/主存
- Cache/主存返回CPU请求的数据给CPU



TLB缺失



- MMU利用虚页号VPN查询TLB
如果TLB访问未命中，则MMU返回TLB访问缺失信息
- MMU利用页表基址寄存器PTBR和虚页号生成页表地址PTEA，访问存放在Cache/主存中的页表，请求与虚拟页号对应的页表项内容。
- Cache/主存向MMU返回页表项PTE，构成所访问信息的物理地址PA
- 若返回的PTE中有效位为1，则需要更新TLB表，同时利用返回的PTE构造PA，并利用构造出的PA访问Cache/主存
- Cache/主存返回CPU请求的数据给CPU

第七章（1）系统总线

重点

- 1.总线的基本概念与分类
- 2.总线特性和性能指标
- 3.总线控制（总线判优与总线通信）

为什么要用总线

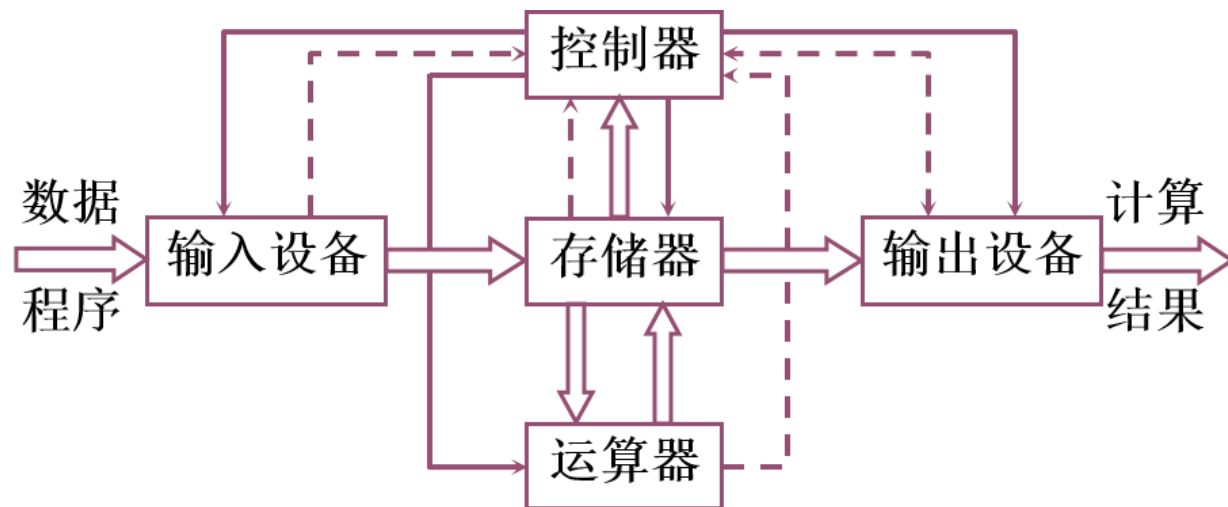
- 什么是总线？
 - 总线是连接各个部件的信息传输线
 - 是各个部件共享的传输介质
- 为什么要用总线（bus）？
- 总线上信息的传送



串行

— — — — —

并行



总线的分类

- 片内总线：芯片内部的总线
- 系统总线：计算机各部件之间的信息传输线
 - 数据总线 双向 与机器字长、存储字长有关
 - 地址总线 单向 与存储地址、I/O地址有关
 - 控制总线 有入、有出
 - 中断请求、总线请求
 - 存储器读、存储器写、总线允许、中断确认
- 通信总线：用于计算机系统之间 或 计算机系统与其他系统（如控制仪表、移动通信等）之间的通信
 - 根据传输方式可以分为：串行通信总线和并行通信总线

总线特性

机械特性 尺寸、形状、管脚数 及 排列顺序

电气特性 传输方向 和有效的 电平 范围

功能特性 每根传输线的 **功能**

{ 地址
数据
控制

时间特性 信号的 时序 关系

总线的性能指标

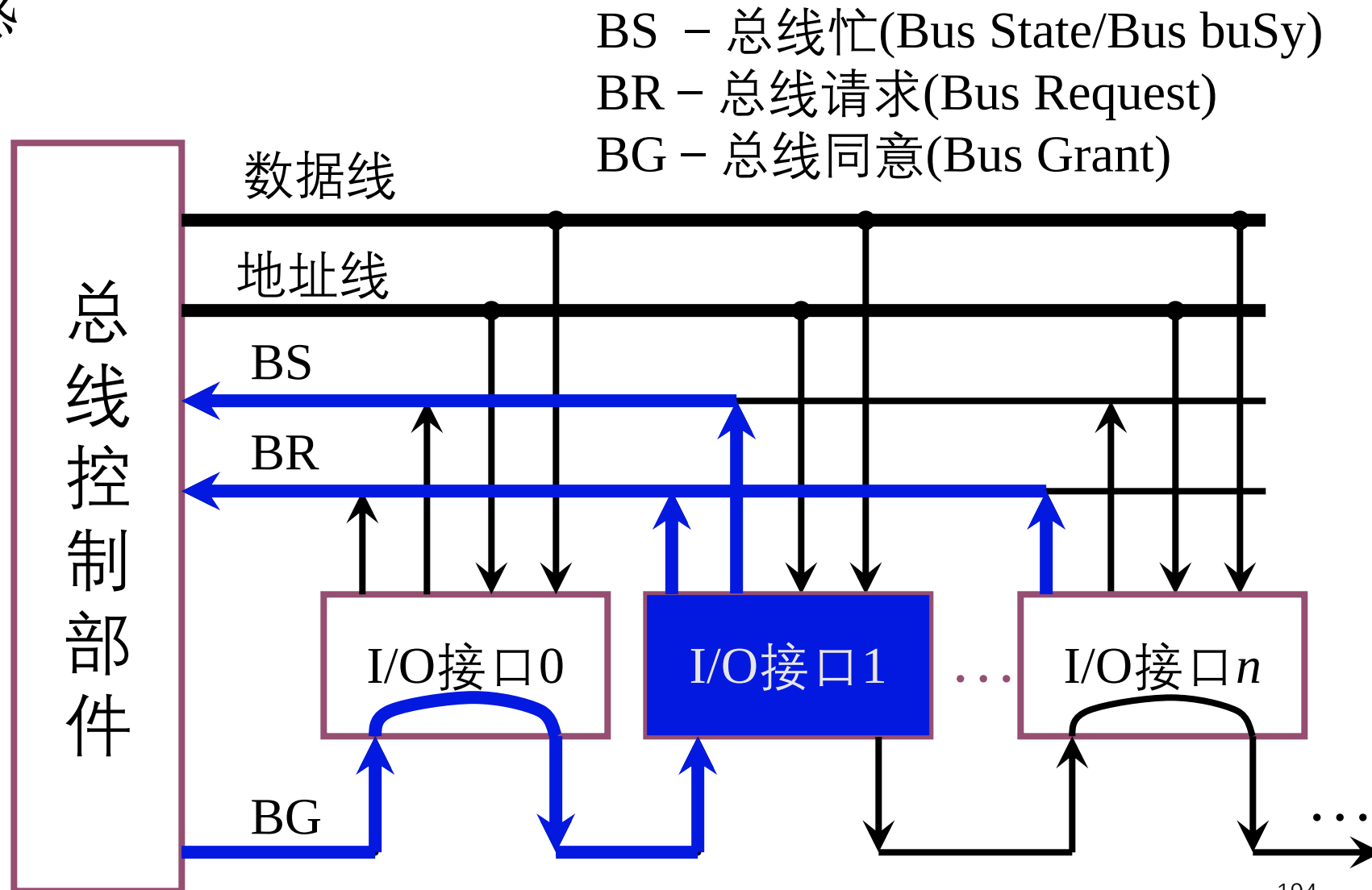
- 总线宽度 数据线 的根数
- 标准传输速率 每秒传输的最大字节数 (MBps)
- 时钟同步/不同步 同步、不同步
- 总线复用 地址线 与 数据线 复用
- 信号线数 地址线、数据线和控制线的 总和
- 总线控制方式 突发、自动、仲裁、逻辑、计数
- 其他指标 负载能力

总线控制基本概念

- 主设备(模块): 对总线有 **控制权**
- 从设备(模块): **响应** 从主设备发来的总线命令
- 总线判优(**仲裁**): 对总线的使用进行合理的分配和管理.
 - 部件要使用总线进行通信时,要向控制部件发请求
 - 控制部件按优先级来决定谁使用总线
- 总线判优控制
(仲裁)
 - 集中式
 - 链式查询
 - 计数器定时查询
 - 独立请求方式
 - 分布式

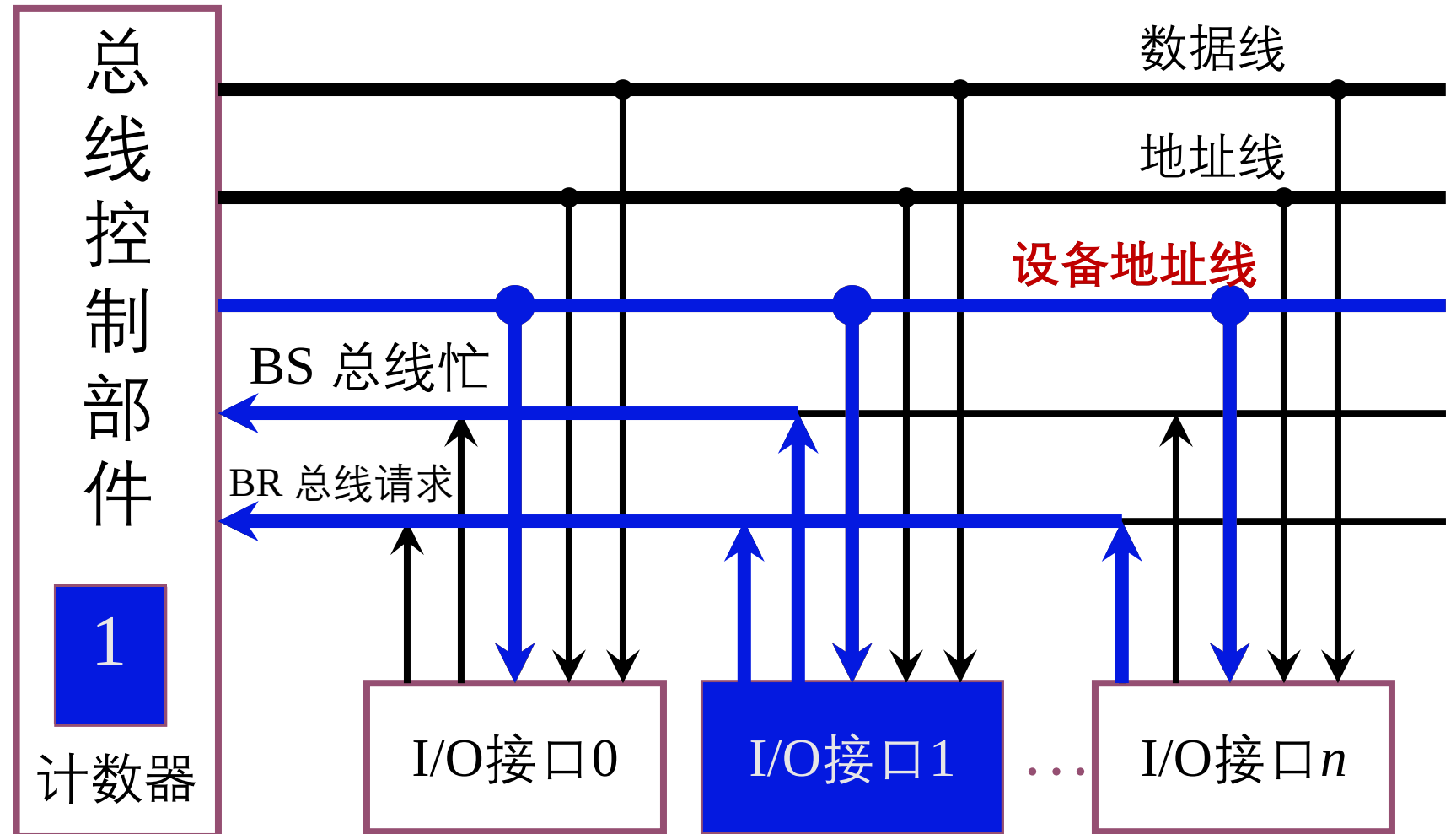
链式查询方式

- 单点故障敏感
- 优先级固定
- 响应慢



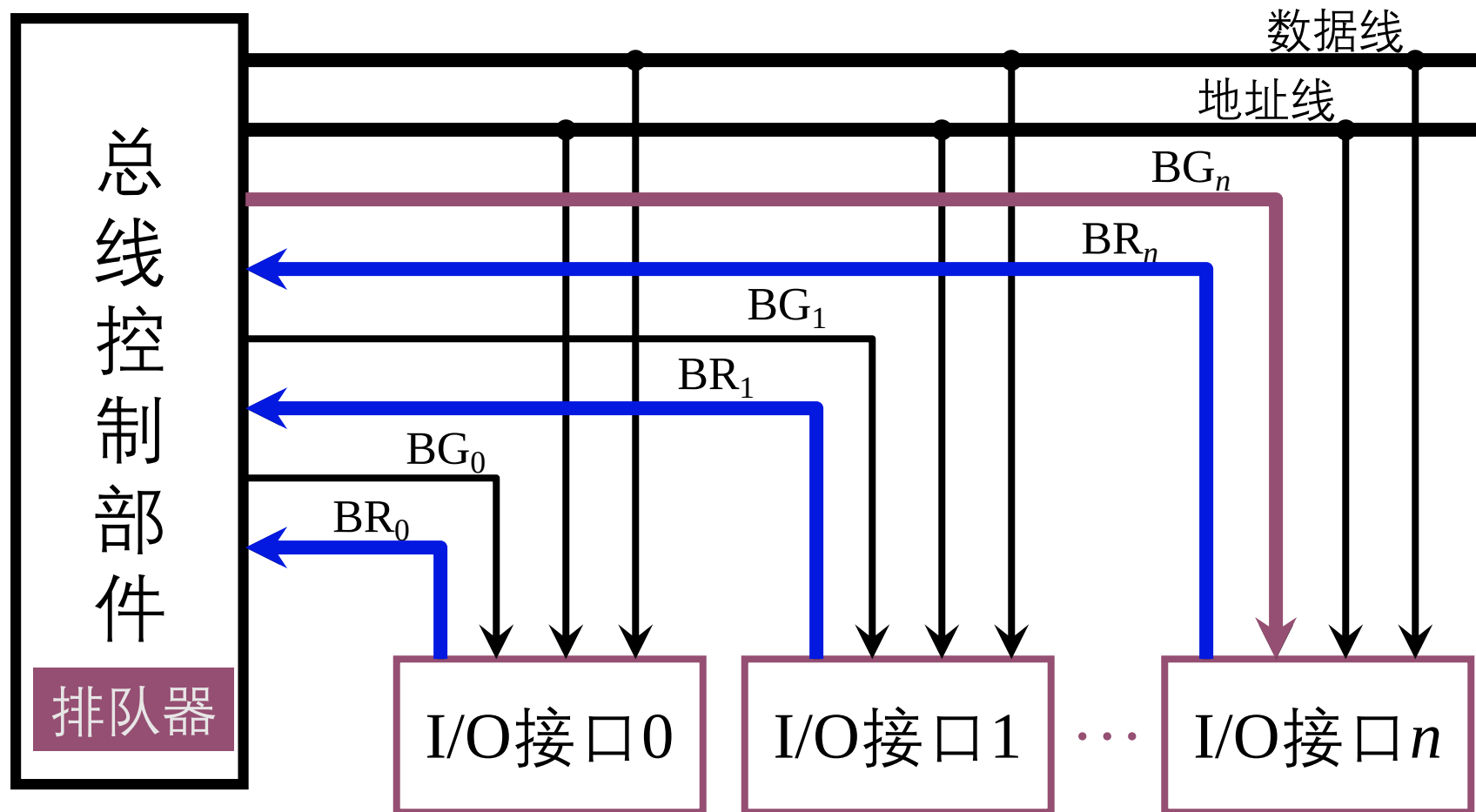
计数器定时查询方式

- 设备地址线：每个IO设备都有一个唯一的设备编号。
- 优先级可变
- 响应慢
- 故障不敏感
- 扩展困难



独立请求方式

- 优先级可灵活变化；
- 响应快；
- 故障不敏感；
- 扩展容易。



总线传输过程

目的：解决通信双方 **协调配合** 问题

总线传输周期：

申请阶段

主模块申请，总线仲裁决定

寻址阶段

主模块向从模块给出地址和命令

传输阶段

主模块和从模块 交换数据

结束阶段

主模块 撤消BR等有关信息

总线通信的四种方式

同步通信

由 **统一时钟** 控制数据传送，适合快速设备

仅适合于总线长度短，存取时间相差不大的情况

异步通信

采用**应答**方式，**没有**公共时钟标准，适合慢速设备。

半同步通信

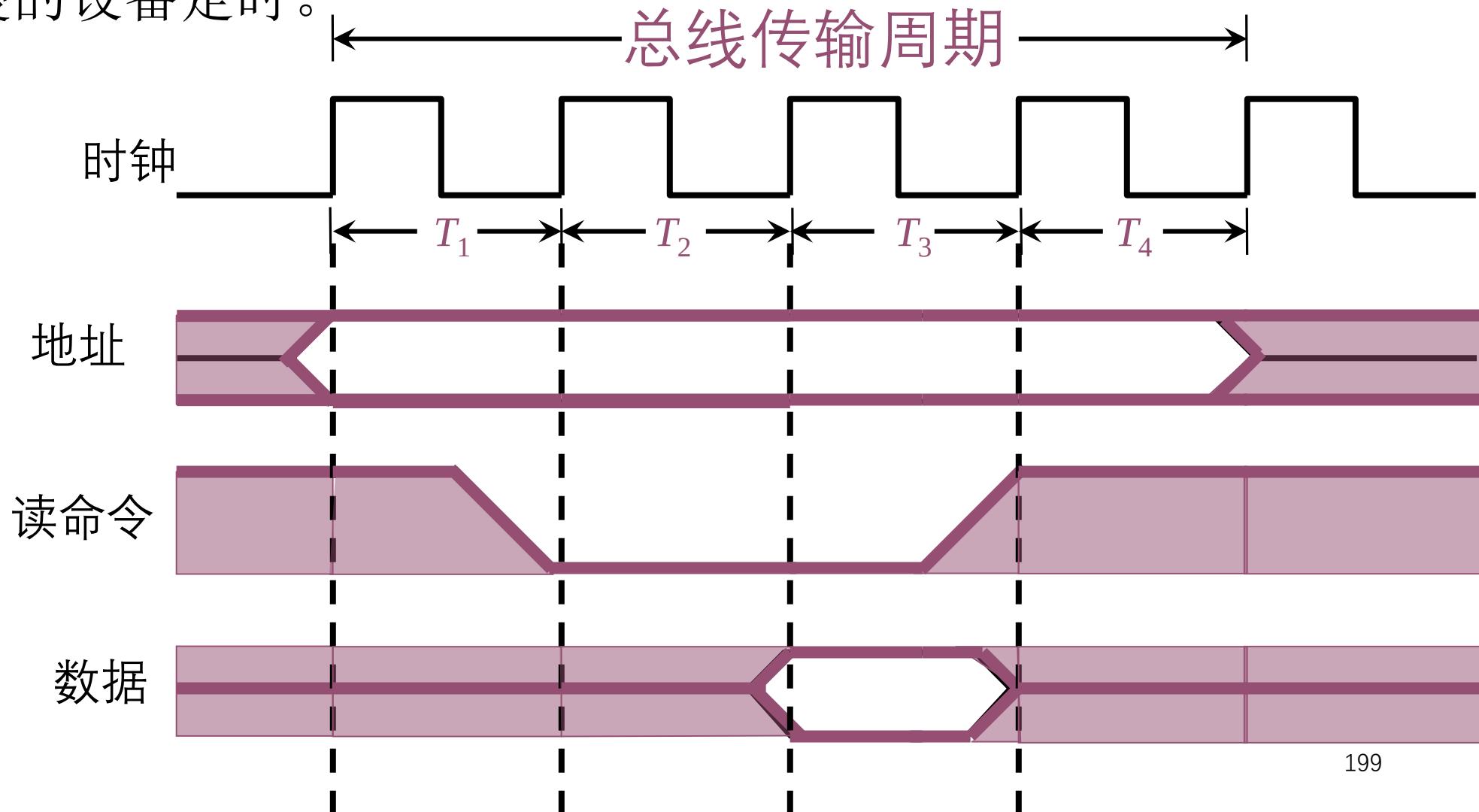
同步、异步**结合**，在同步时钟控制下进行采样和应答。

分离式通信

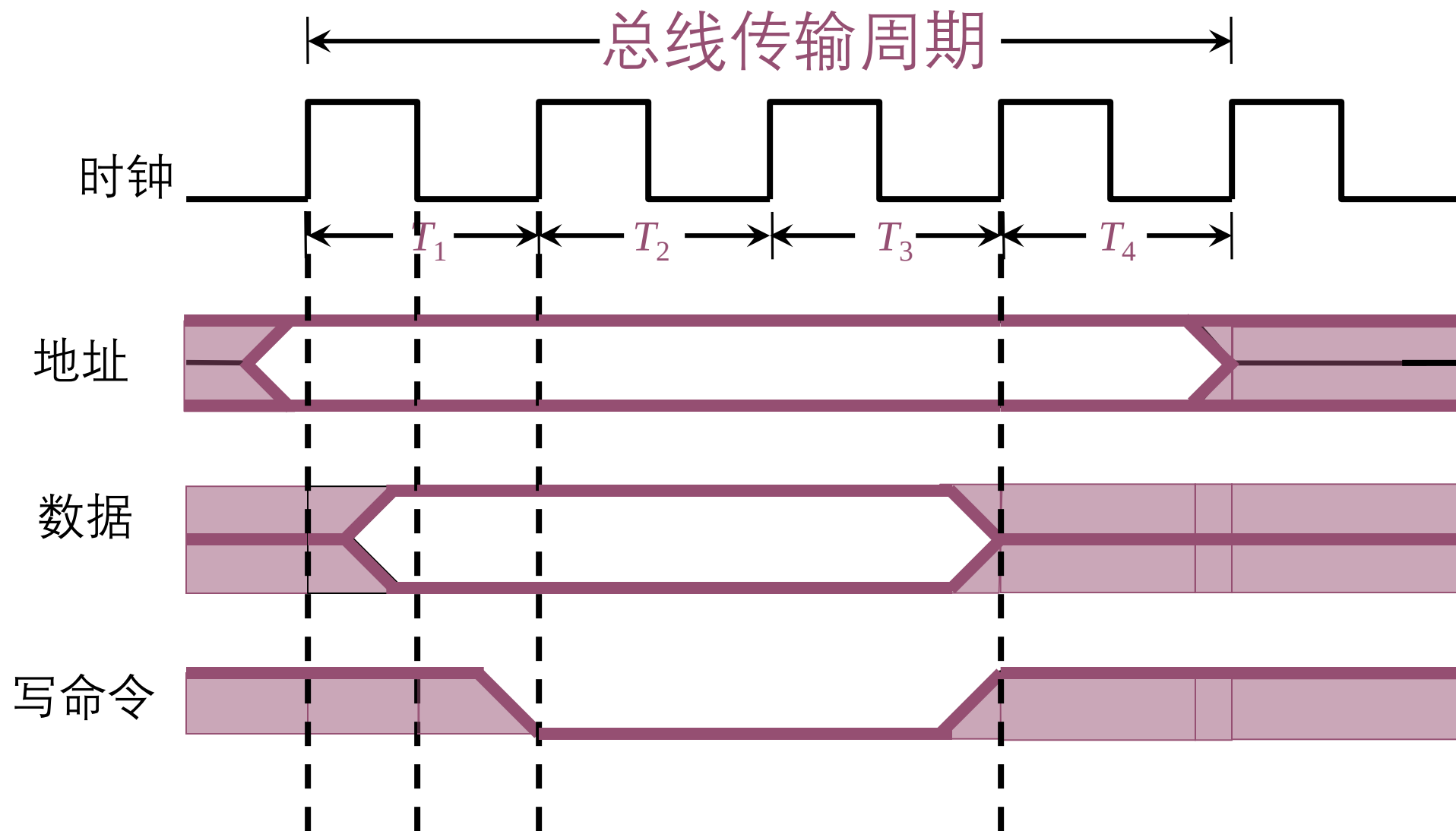
充分**挖掘系统总线**每个瞬间的**潜力**

同步式数据输入

- 仅适合于总线长度短，各功能模块存取时间相差不大的情况，必须按最慢的设备定时。

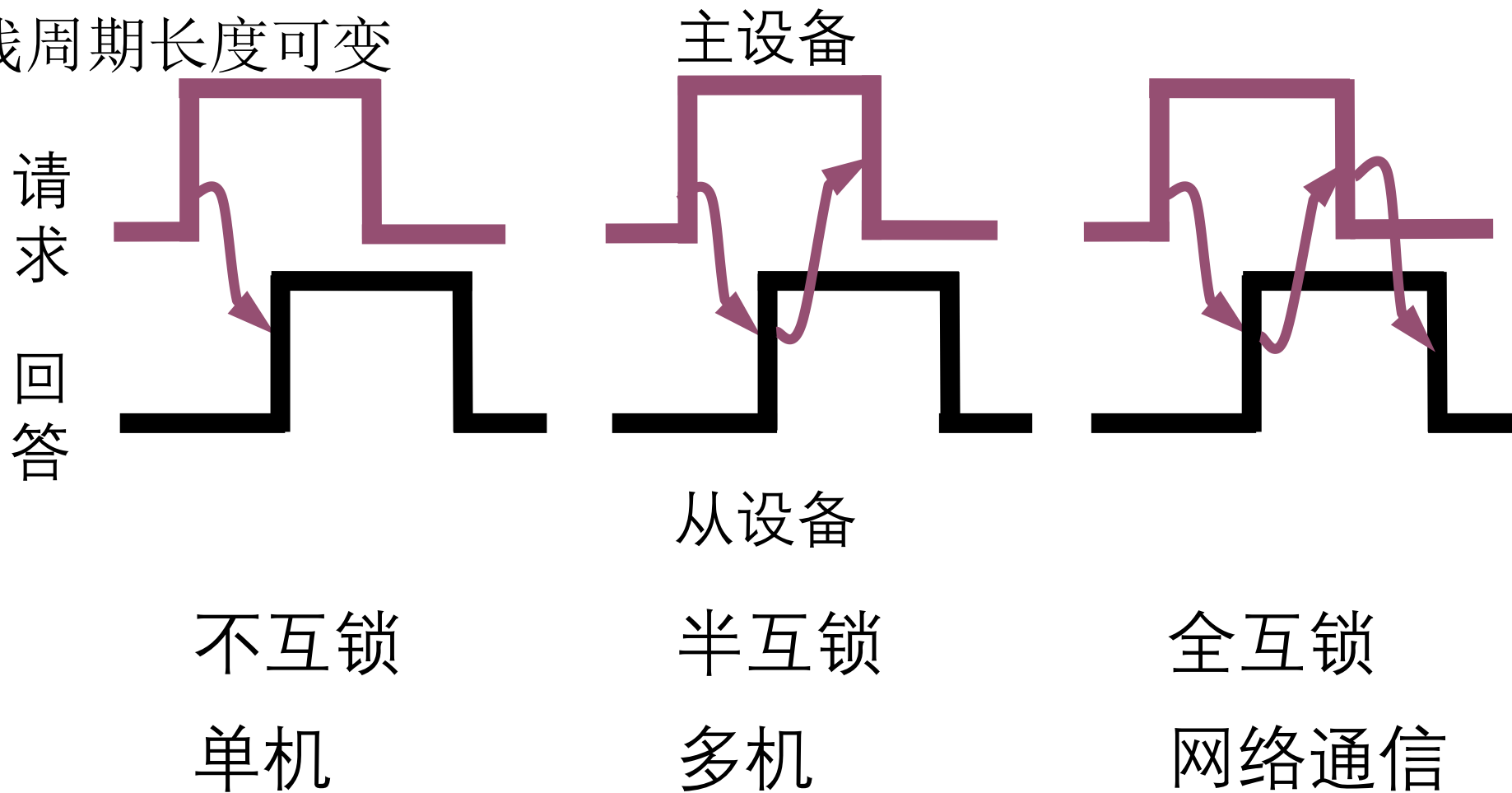


同步式数据输出



异步通信

- 不需公共时钟信号
- 快、慢速设备可连到同一总线上
- 总线周期长度可变



半同步通信（同步、异步结合）

- 同步通信

- 发送方 用系统 时钟前沿 发信号
- 接收方 用系统 时钟后沿 判断、识别

- 异步通信

- 允许不同速度的模块和谐工作
- 增加一条 “等待” 响应信号线

以输入数据为例的半同步通信时序

T_1 主模块发地址

T_2 主模块发命令

T_w 当 $\overline{\text{WAIT}}$ 为低电平时，等待一个 T

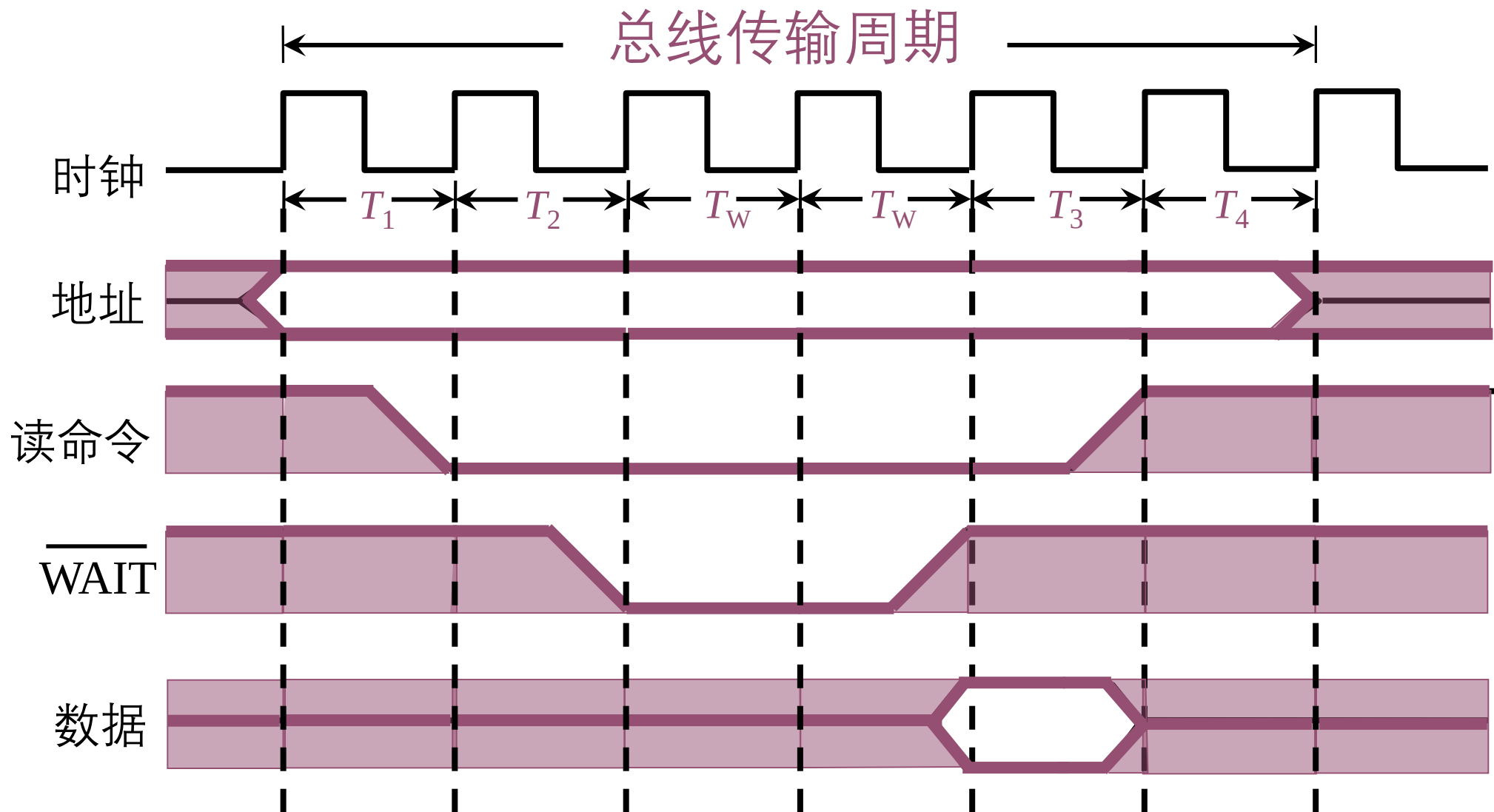
T_w 当 $\overline{\text{WAIT}}$ 为低电平时，等待一个 T

⋮

T_3 从模块提供数据

T_4 从模块撤销数据，主模块撤销命令

半同步通信（同步、异步 结合）



分离式通信

- 充分挖掘系统总线每个瞬间的潜力

总线传输周期 { 子周期1 主模块申请占用总线，用完后放弃总线使用权
子周期2 从模块申请占用总线，将信息送至总线

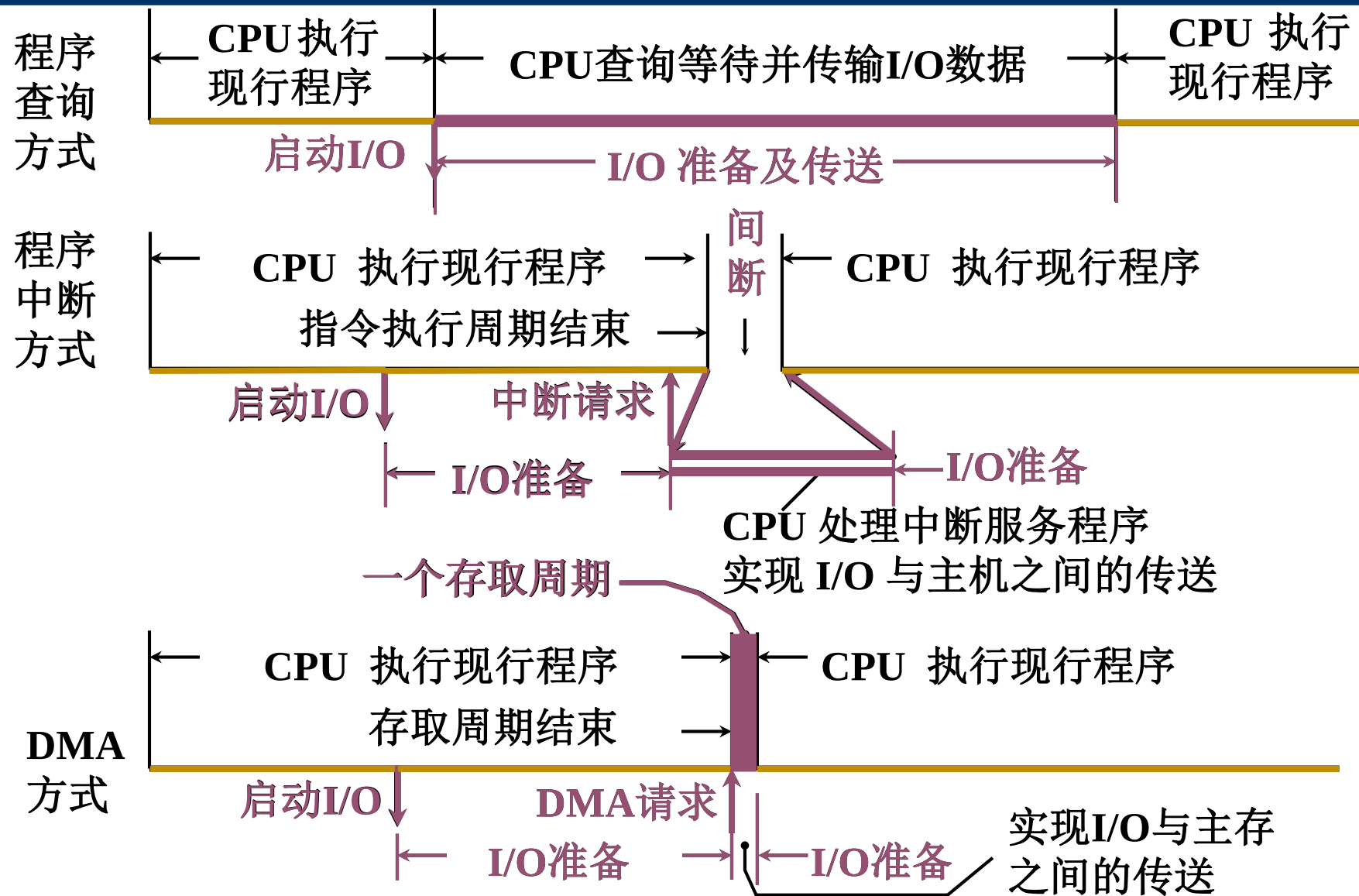
- 特点：
 - 各模块有权申请占用总线
 - 采用同步方式通信，不等对方回答
 - 各模块准备数据时，不占用总线
 - 总线被占用时，无空闲
 - 充分提高了总线的利用率

第七章（2）输入输出系统

重点

1. 主机与 I/O 交换信息的三种控制方式

三种方式的CPU工作效率比较

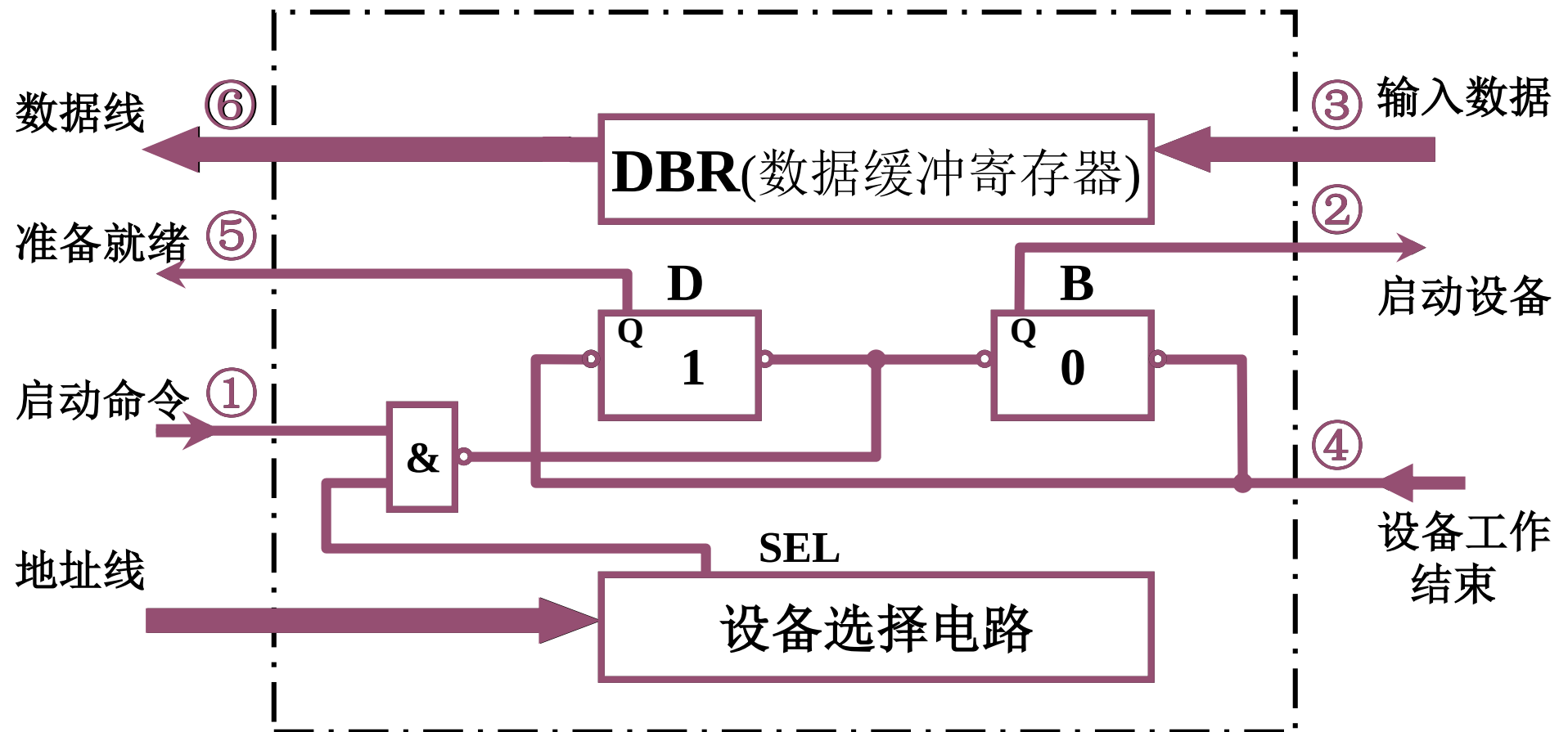


第8章 输入输出系统

重点

1. 主机与 I/O 交换信息的三种控制方式
2. 程序查询方式特点、接口电路、工作原理

程序查询方式的接口电路——以输入为例

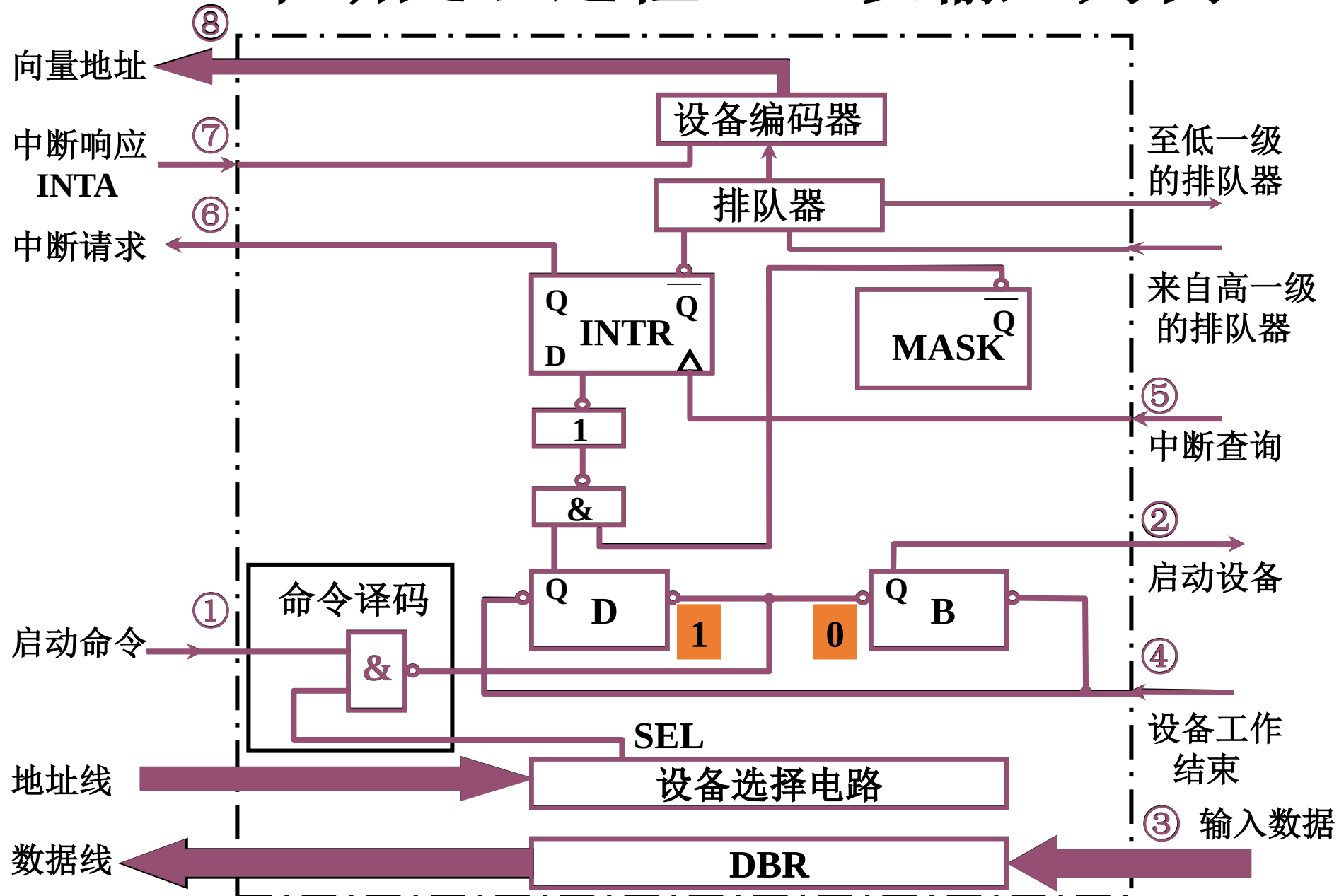


第七章（2）输入输出系统

重点

1. 主机与 I/O 交换信息的三种控制方式
2. 程序查询方式特点、接口电路、工作原理
3. 程序中断方式特点、接口电路、工作原理

I/O 中断处理过程——以输入为例

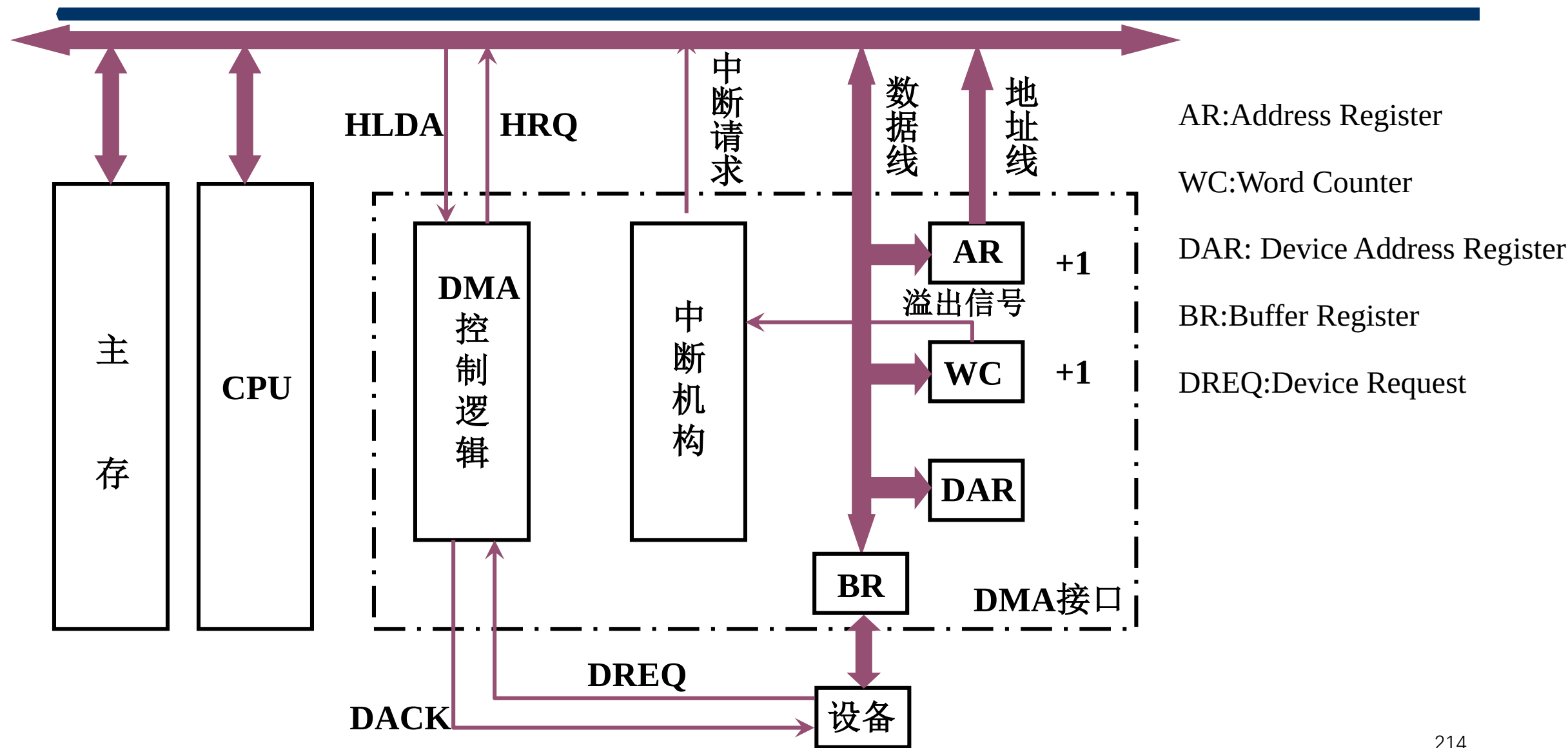


第七章（2）输入输出系统

重点

1. 主机与 I/O 交换信息的三种控制方式
2. 程序查询方式特点、接口电路、工作原理
3. 程序中断方式特点、接口电路、工作原理
4. DMA 方式特点、接口电路、工作原理

DMA 接口的组成



第八章 输入输出系统

难点

1. 处理 I/O 中断的各类软、硬件技术的运用，中断屏蔽字

中断请求（INTR）触发器

- 一个中断请求源对应一个 INTR 中断请求标记触发器
 - INTR 分散 在各个中断源的 接口电路中
 - INTR 集中 在 CPU 的中断系统 内
- 多个 INTR 组成中断请求寄存器

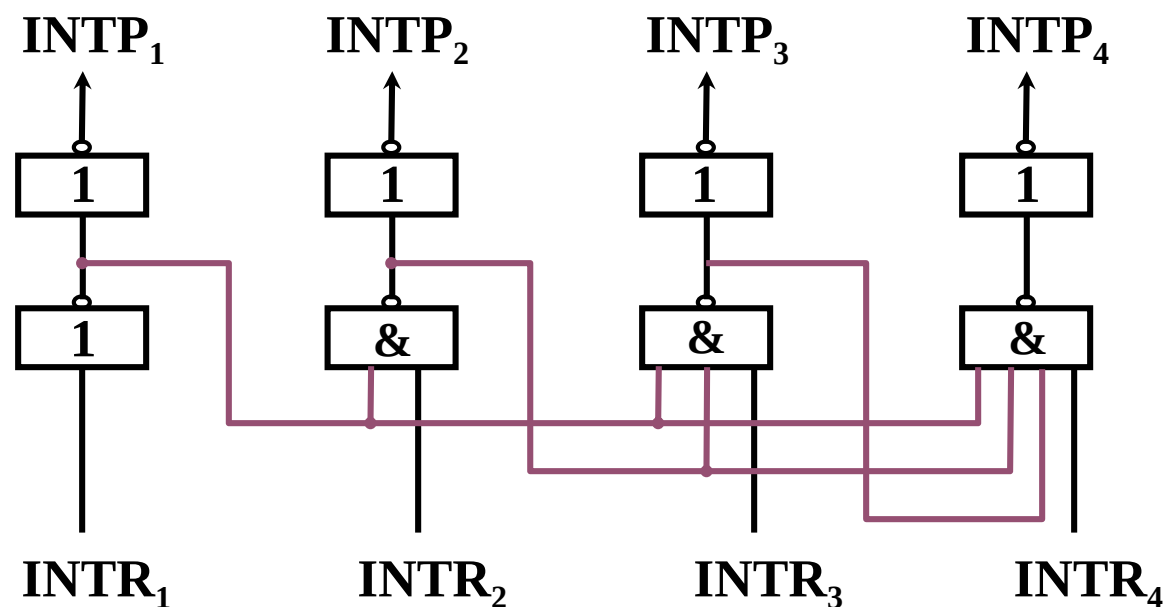
1	2	3	4	5			<i>n</i>
掉电	过热	主存读写校验错	阶上溢	非法除法		键盘输入	打印机输出

中断判优逻辑（CPU端）

1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中 链式排队器

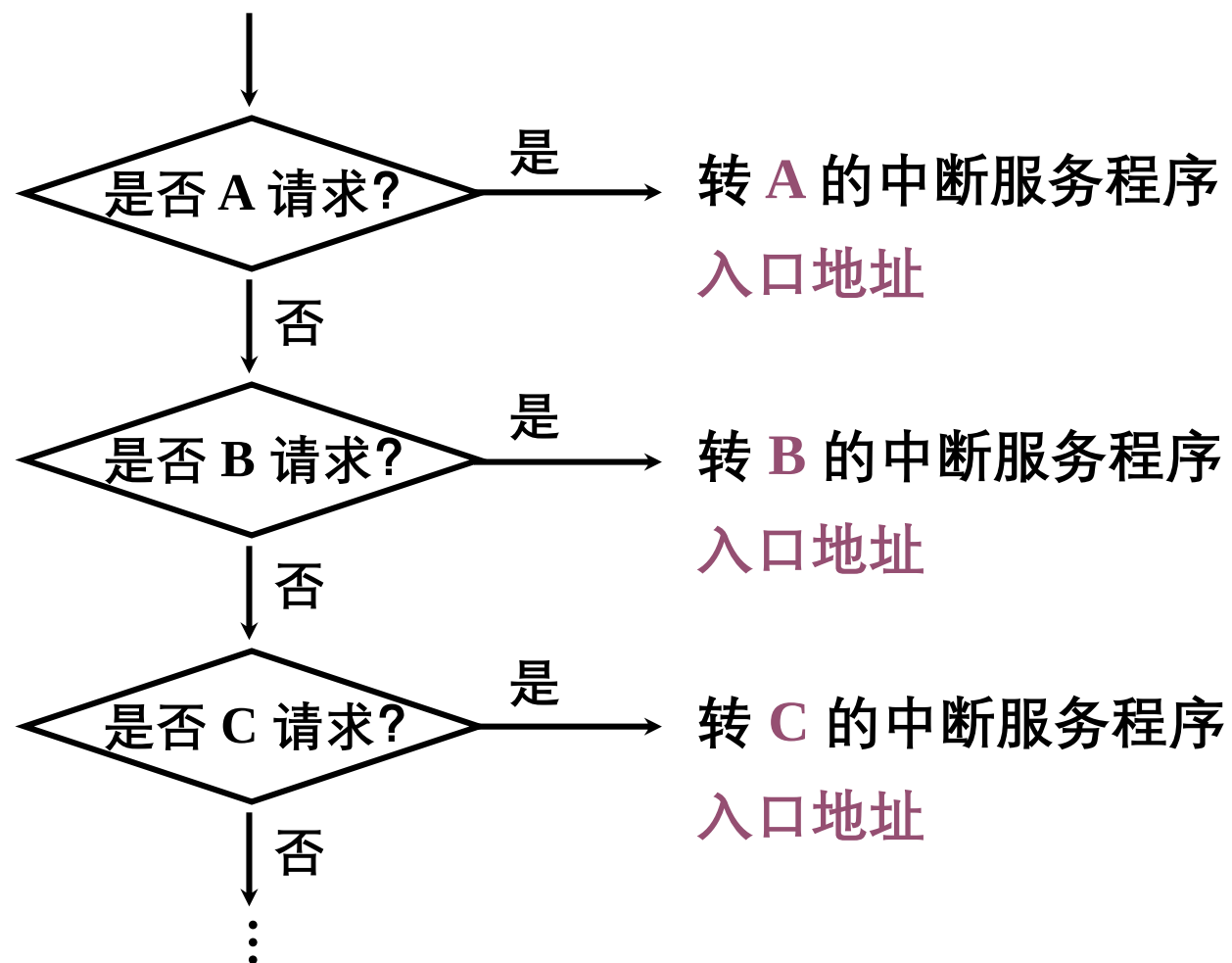
② 集中 在 CPU 内



$INTR_1$ 、 $INTR_2$ 、 $INTR_3$ 、 $INTR_4$ 优先级按降序排列

（中断）排队器——软件实现（程序查询）

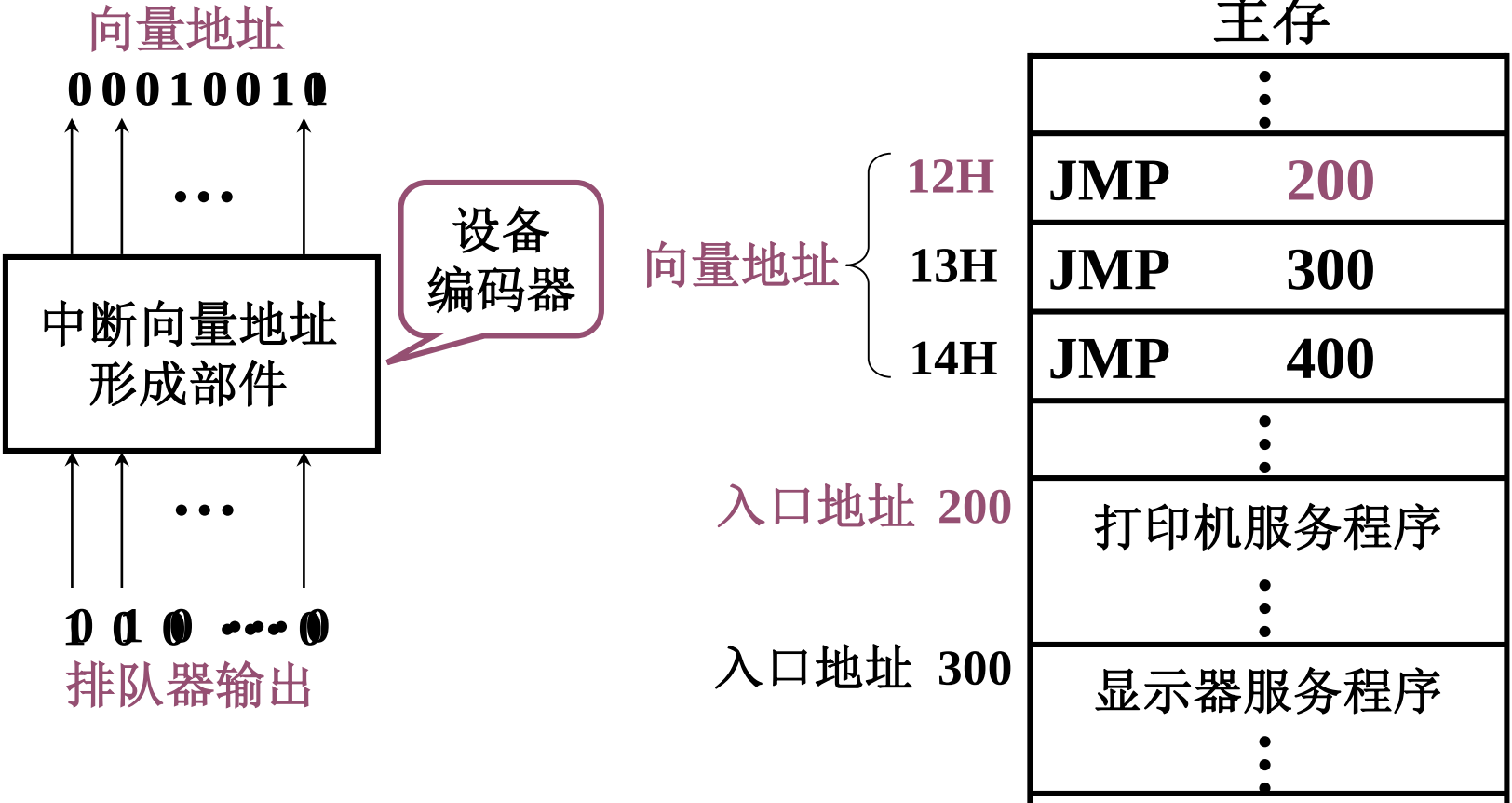
假设A、B、C 优先级按 降序 排列



中断向量地址形成部件

中断服务程序入口地址 { 由软件产生
 硬件向量法 由 **硬件** 产生 **向量地址**

再由 **向量地址** 找到 **入口地址**
主存



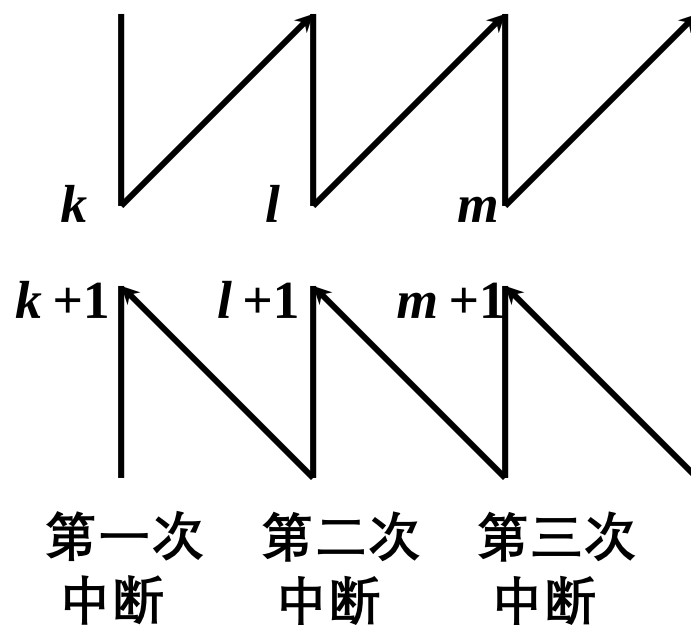
中断服务程序入口地址——软件查询法

八个中断源 1, 2, ... 8 按 降序 排列
中断识别程序 (入口地址 M)

地 址	指 令	说 明
M	SKP DZ 1 [#]	1 [#] D = 0 跳 (D为完成触发器)
	JMP 1 [#] SR1	1 [#] D = 1 转1 [#] 服务程序
	SKP DZ 2 [#]	2 [#] D = 0 跳
	JMP 2 [#] SR2	2 [#] D = 1 转2 [#] 服务程序
	⋮	
	SKP DZ 8 [#]	8 [#] D = 0 跳
	JMP 8 [#] SR	8 [#] D = 1 转8 [#] 服务程序

中断屏蔽技术

1. 多重中断的概念

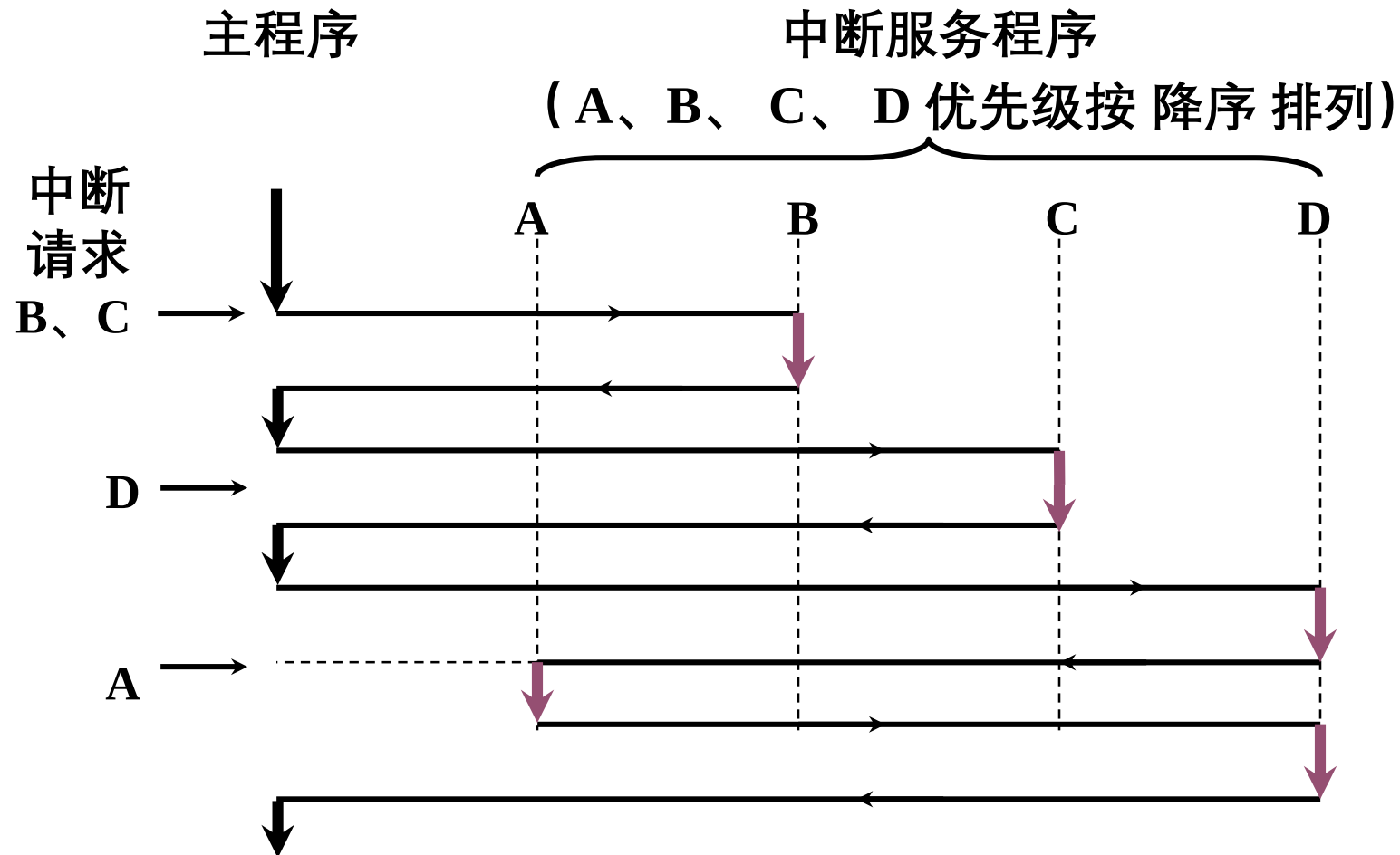


程序断点 $k+1$, $l+1$, $m+1$

2. 实现多重中断的条件

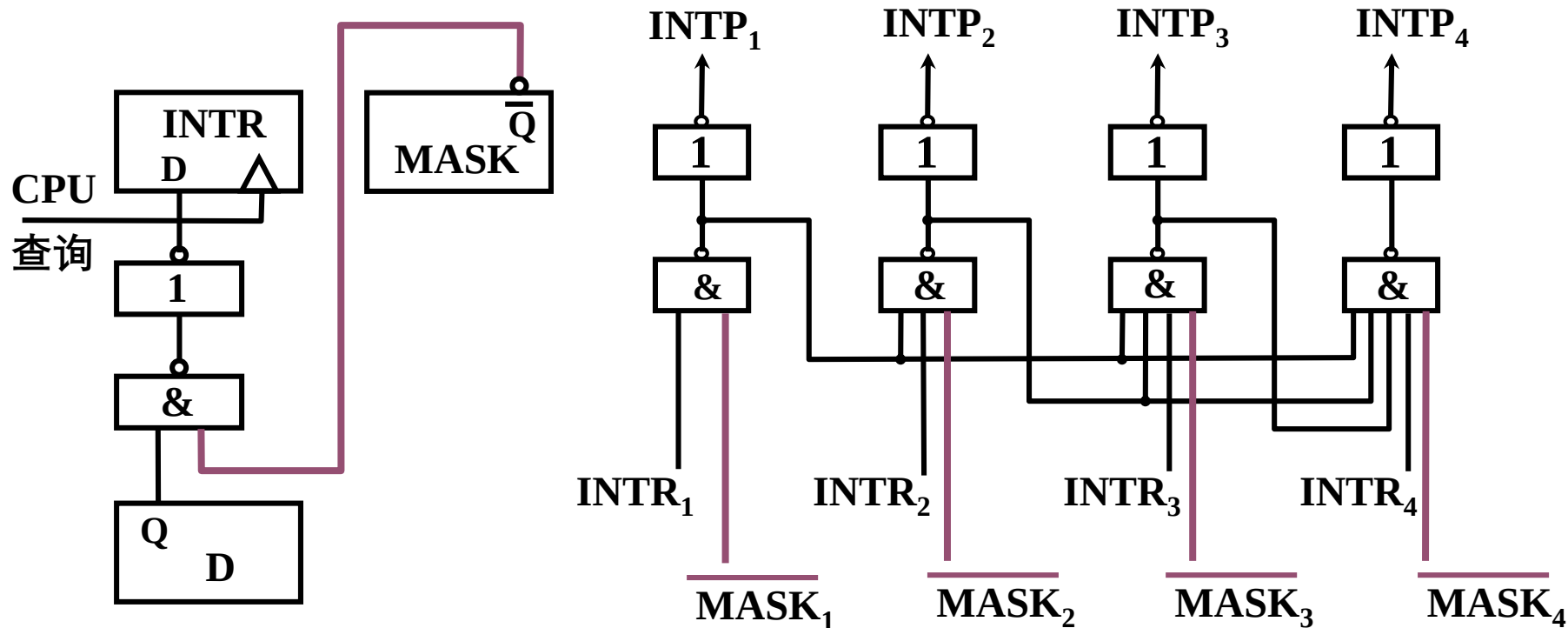
(1) 提前 设置 开中断 指令

(2) 优先级别高 的中断源 有权中断优先级别低 的中断源



3. 屏蔽技术

(1) 屏蔽触发器的作用



$MASK = 0$ (未屏蔽)

INTR 能被置“1”

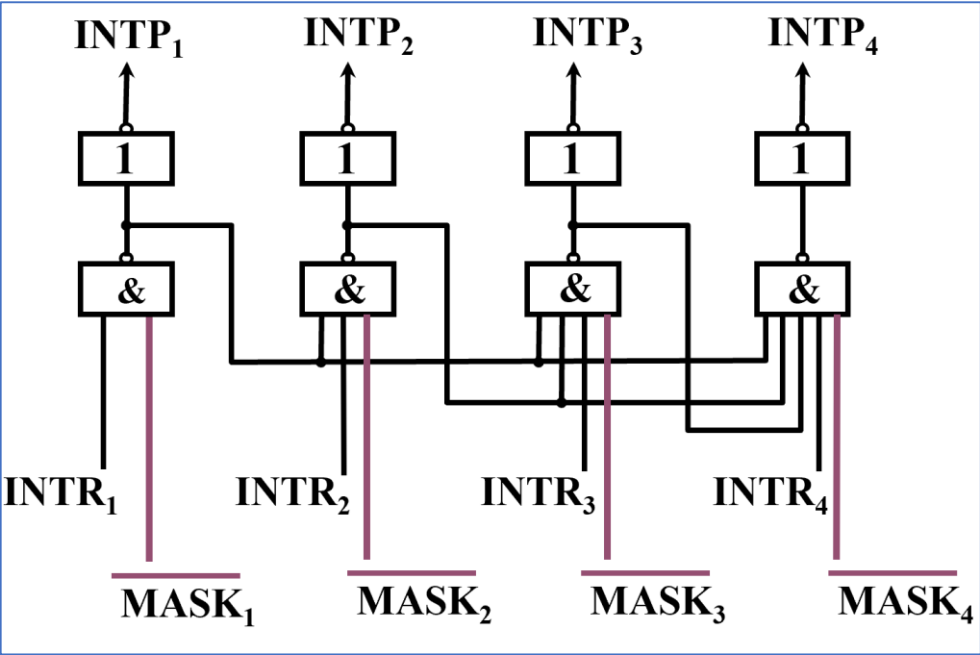
$MASK_i = 1$ (屏蔽)

$INTP_i = 0$ (不能被排队选中)

(2) 屏蔽字

16个中断源 1, 2, 3 , ... 16 按 降序 排列

优先级	屏蔽字															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
5	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1
⋮	⋮															
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1



(3) 屏蔽技术可改变处理优先等级

响应优先级 不可改变

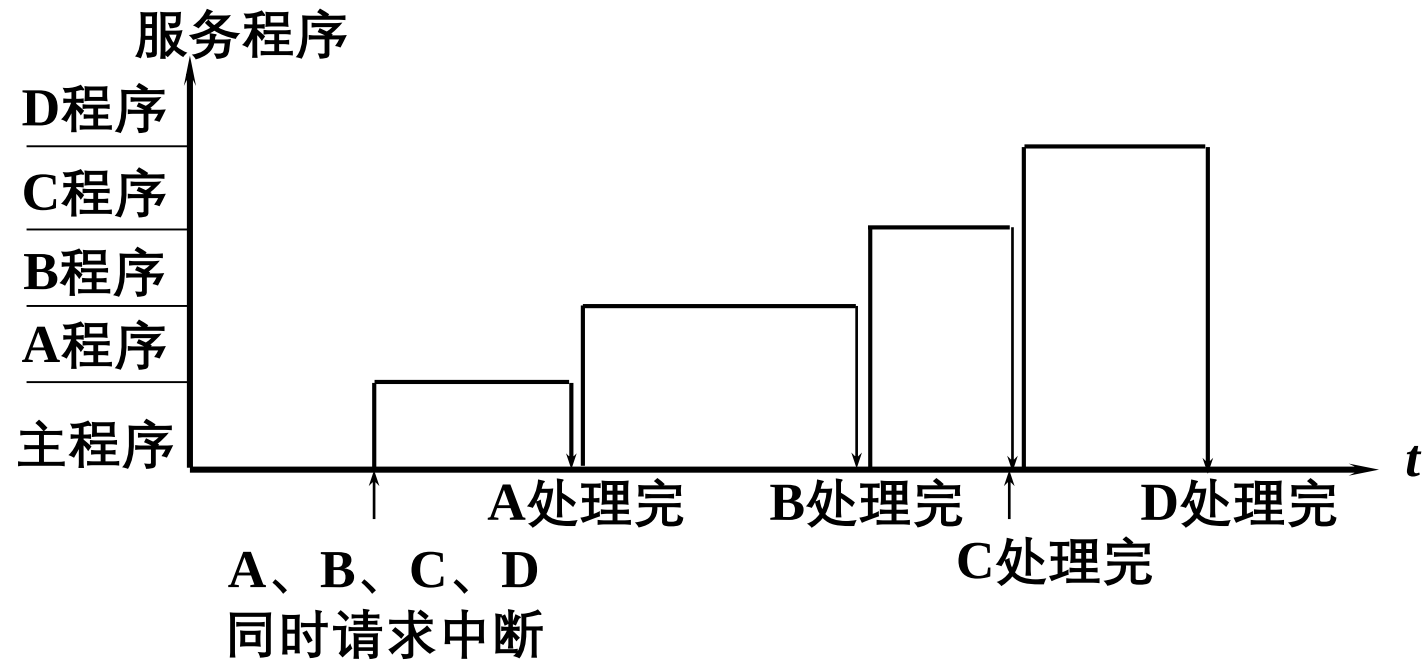
处理优先级 可改变（通过重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级 A→B→C→D 降序排列

处理优先级 A→D→C→B 降序排列

(3) 屏蔽技术可改变处理优先等级

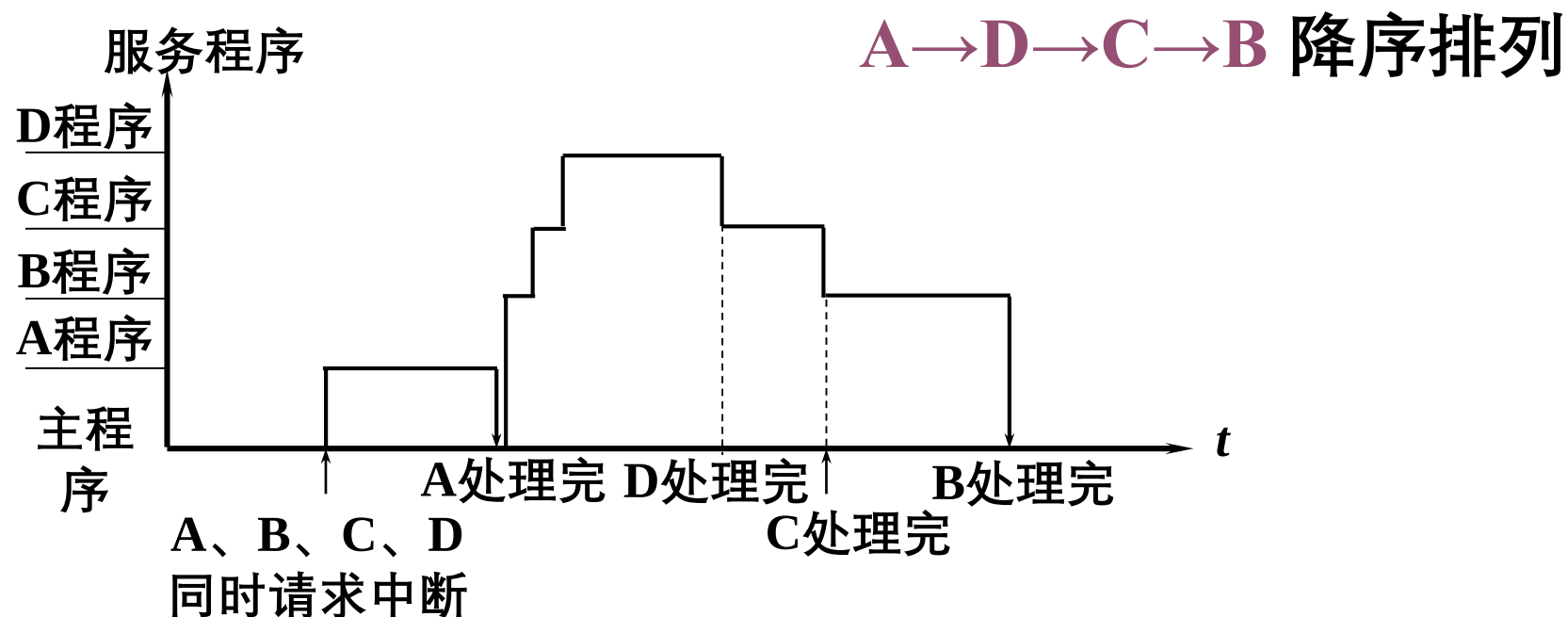


CPU 执行程序轨迹（原屏蔽字）

A→B→C→D 降序排列

(3) 屏蔽技术可改变处理优先等级

中断源	新屏蔽字
A	1 1 1 1
B	0 1 0 0
C	0 1 1 0
D	0 1 1 1



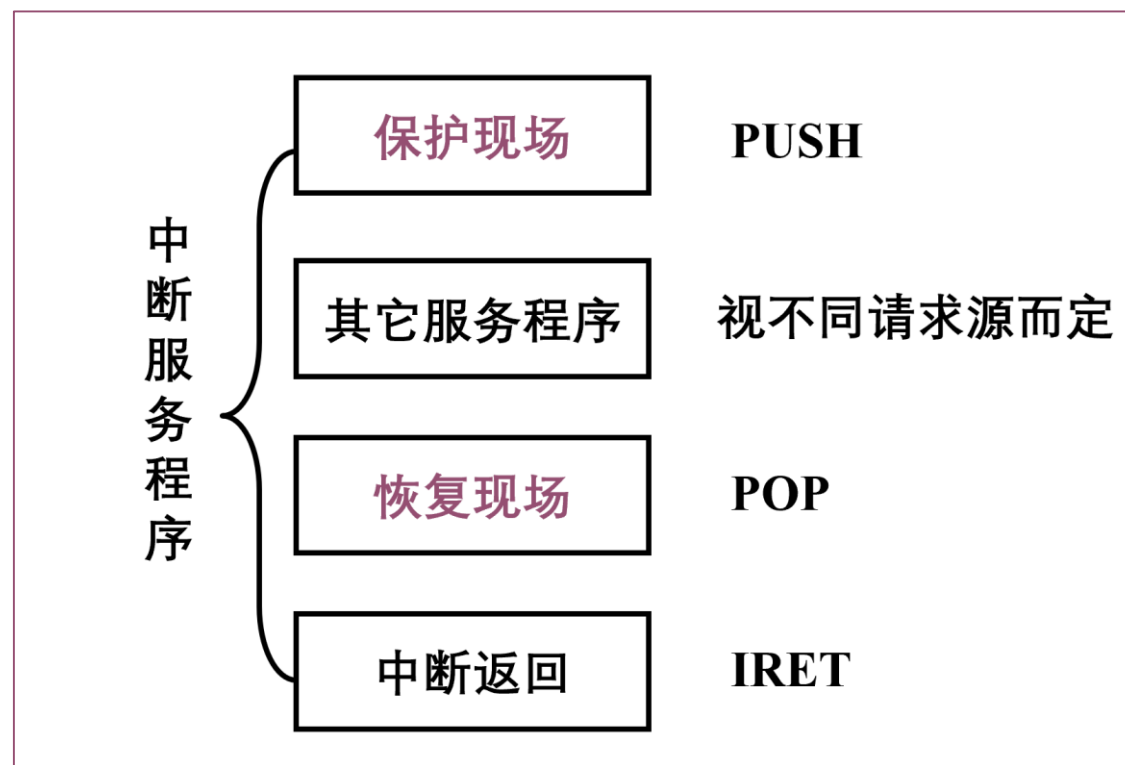
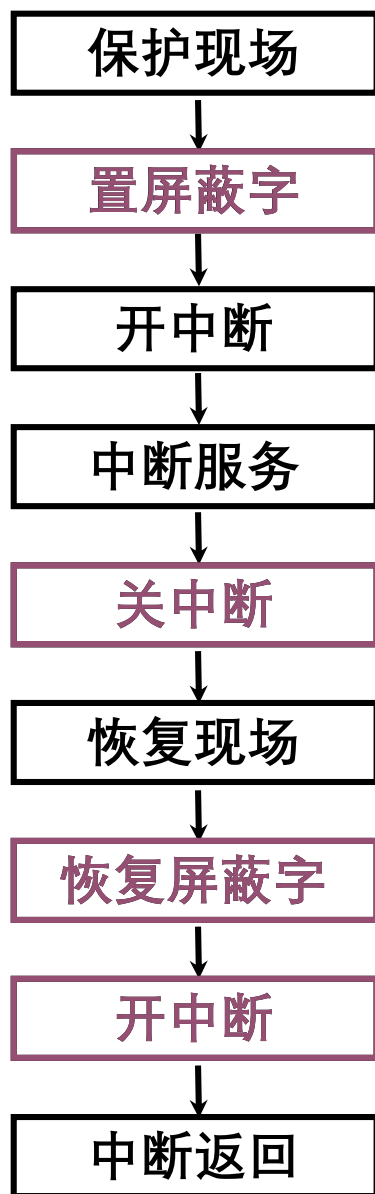
CPU 执行程序轨迹（新屏蔽字）

注意：不能改变相应优先级，只能改变执行优先级。

(4) 屏蔽技术的其他作用

可以 **人为地屏蔽** 某个中断源的请求
便于程序控制

(5) 新屏蔽字的设置



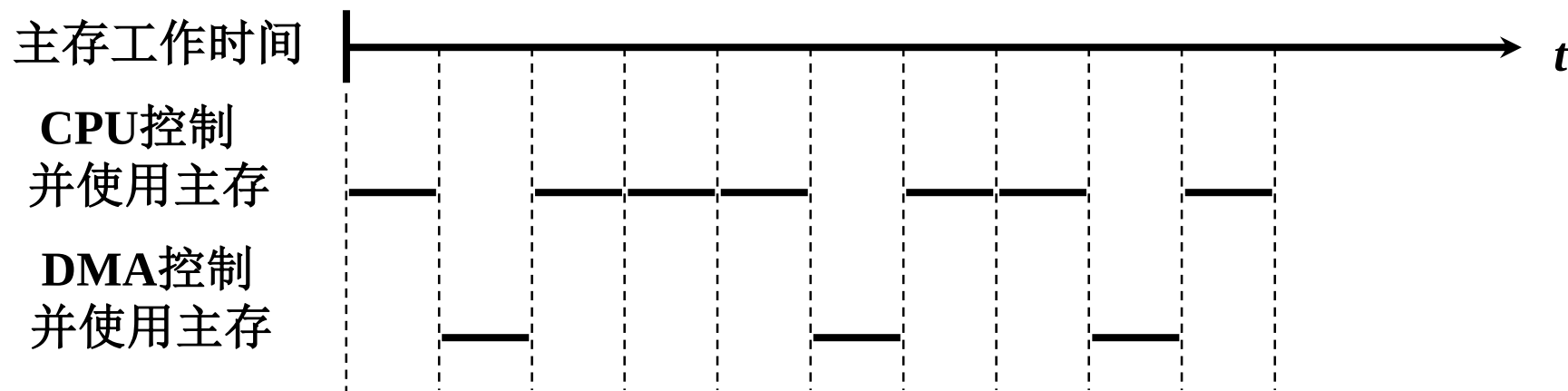
第七章（2）输入输出系统

难点

1. 处理 I/O 中断的各类软、硬件技术的运用
2. DMA 与主存交换数据的三种方法各自的特点
3. 周期窃取的含义

周期挪用

- DMA要求访存时，CPU暂停一个或多个存储周期，将总线控制权让给 DMA，数据传送结束后，CPU继续运行。
- CPU现场没有变动，仅延缓了指令的执行
 - 周期挪用或称周期窃取。



第七章（2） 输入输出系统

难点

1. 处理 I/O 中断的各类软、硬件技术的运用
2. DMA 与主存交换数据的三种方法各自的特点
3. 周期窃取的含义
4. CPU 响应中断请求和 DMA 请求的时间

DMA 方式与程序中断方式的比较

	中断方式	DMA 方式
(1) 数据传送	程序	硬件
(2) 响应时间	指令执行结束	存取周期结束
(3) 处理异常情况	能	不能
(4) 中断请求	传送数据	后处理
(5) 优先级	低	高